Fast Wait-Free Parallel Breadth-First Search

Aayush Kumar
aayush.k4562@gmail.com

Abstract

The fastest parallel implementations of breadth-first search (BFS) for shared memory are direction-optimizing and blocking: every level or round ends at a barrier or a join, so a thread that is preempted or stopped by a page fault delays all the others. A BFS is wait-free if every call returns after a finite number of the caller's own steps, which is easy if each thread searches the whole graph alone. The challenge is to share a search among threads without waiting, while repeating little work and competing in speed with the best blocking implementations.

We present such a method. The vertices that a thread discovers are its share of the next level; it expands its own share, then helps with the rest, and every write that lets other threads skip work comes after that work, so a thread that stops leaves nothing that the others cannot redo. We prove the method correct and wait-free under the memory model of x86, given that allocation takes a bounded number of steps, bound the work of a top-down level by that level and the number of threads, and give a variant whose memory depends on the largest levels, plus a constant per level.

On five standard graphs and 128 cores, our method examines about as many neighbors as the sequential baseline at 16 threads, and at most 30% more at 112; the best times of GAPBS, GBBS and PASGAL are 1.58, 2.77 and 2.73 times its own in the geometric mean. With one of 112 threads stalled for 10 ms at every level, or 1 ms on the road network, its median time grows by at most 7%, and theirs up to 19.5-fold. Since every thread takes part in every level, it is slow on long paths and other graphs with many tiny levels.

CCS Concepts

• **Computing methodologies → Shared memory algorithms**.

Keywords

Breadth-first search (BFS), Direction-optimizing BFS, Wait-free algorithms, Helping, Parallel graph algorithms, Shared memory

1 Introduction

Breadth-first search (BFS) is one of the most basic graph algorithms. Starting from a source vertex, it visits the graph level by level, where level d holds the vertices at distance d from the source, and so finds shortest paths in unweighted graphs. It is a building block of many other graph algorithms, such as connectivity, betweenness centrality and maximum flow, and it is the search kernel of the Graph500 and GAP benchmarks [6, 43].

In most parallel BFS algorithms for shared memory, the threads expand the vertices of one level together, which discovers the next level, and move on to it only once the whole level is done; such algorithms are *level-synchronous* [4, 38]. Beamer et al. [5] observed that on low-diameter graphs, a few middle levels hold most of the vertices, and that on those levels it is cheaper to search *bottom-up*: every undiscovered vertex looks for a neighbor in the current level and stops at the first one it finds, instead of every vertex of the level checking all of its neighbors. Their *direction-optimizing* BFS chooses between the two directions at every level. It underlies the fastest shared-memory BFS implementations today, including those of GAPBS [6], Ligra [51], GBBS [16] and PASGAL [17].

These implementations update vertices with atomic instructions rather than locks, but their BFS methods are still *blocking*: GAPBS ends every step with an OpenMP barrier, and GBBS and PASGAL end every round when its parallel loop joins, so a level is over only once its slowest thread has finished its share. A thread delayed while it holds part of a level, because the operating system preempted it, or an interrupt or a page fault stopped it, keeps every other thread waiting at the end of that level; dynamic scheduling and work stealing [9] hand over the work that a delayed thread has not started, but not the work it is in the middle of. The cost recurs at every level that a delay overlaps, much as short delays on single processors, amplified by synchronization, slow down whole applications on large parallel machines [26, 48], and the number of levels grows with the diameter of the graph: a BFS of the USA road network in our experiments has 6,262 levels, and on such graphs the synchronization alone can cost more than the work [17].

Wait-freedom removes this dependence. A method is *wait-free* if every call to it finishes in a finite number of the calling thread's own steps, however slow the other threads are [23, 25]. Our BFS is a method that every thread calls with the same source, and that returns only once all distances are final; if it is wait-free, no call waits for other threads, and the search is complete as soon as the first call returns, even if the other threads have stopped for good.

A wait-free BFS is easy to obtain if work does not matter: every thread can search the whole graph by itself, all writing the same distances, at T times the work of a sequential search with T threads. Do-All and Certified Write-All (CWA), in which asynchronous or failing processors must complete a set of tasks [21, 32, 41], show how to do far less, but the simplest way to use a CWA for BFS, one run per level over all n vertex indices, costs at least n in every level, however few vertices the level holds (Section 2.3). What has been missing is a wait-free BFS whose threads share each top-down level at a cost that depends on that level, and that is about as fast as the heavily tuned blocking implementations when no thread is delayed; wait-free implementations of concurrent data structures have long been considered too slow for practice [35].

Three problems make this hard. First, without barriers, every thread must decide for itself when a level is complete, and a thread that has fallen behind, possibly by several levels, can still write to shared state at any moment; such late writes must never undo or corrupt the work of others, and since the tasks of a level are discovered during the level before, by the same threads, there is no array of tasks to certify. Second, helping must be cheap: when no thread is delayed, threads should rarely repeat each other's work, and when one is, the others should split what it has left rather than each redo all of it, even when that is a vertex with millions of neighbors. Third, direction optimization adds decisions for a whole level: all threads must expand a level in the same direction, chosen

from its size, which threads may see differently, and the bottom-up step needs a test of membership in the current level that stays exact for threads that are behind.

This paper presents a wait-free method for direction-optimizing BFS that solves these problems. Its contributions are:

- **A wait-free direction-optimizing BFS** (Sections 3 to 5, Figures 1 and 3). Each level holds one bucket per thread; a thread expands its own bucket and then helps with the others in random orders, and every write that lets threads skip work comes after that work. One successful compare-and-swap (CAS) per level, with at most one attempt per thread, fixes its direction. The distances are final once any call returns, and every call returns within a bounded number of its own steps, under total store order and allocation in a bounded number of steps.
- **Bounds on work and memory** (Sections 5.7 and 6, Figure 4). The work of a top-down level depends only on that level and on T. Any piece of work certified as a whole can be repeated by every thread, so finer certificates can lower the repetition; with uniformly random orders and sequentially consistent or fenced marks, threads expand $O((T+R)\log n)$ positions of a bucket of R positions with high probability, whatever the schedule. A variant that reuses two level nodes needs memory that depends only on the largest levels, plus a constant per level.
- **An implementation and evaluation** (Sections 7 and 8). On a 128-core machine, our method examines at most 30% more neighbors than the sequential baseline at 112 threads on four real-world graphs and a Kronecker graph, and the best times of GAPBS, GBBS and PASGAL there are 1.58 to 2.77 times its own in the geometric mean. With one of 112 threads stalled for 10 ms at every level, or 1 ms on the road network, its median time grows by at most 7%, and theirs up to 19.5-fold. Graphs with many levels of few vertices, such as long paths, are slow for it, and the reset before a search waits for every thread. Our code, the scripts of every experiment and their data are available at github.com/Random-Bee/WFGT/tree/main/artifact.

To our knowledge, ours is the first wait-free BFS in which threads share the work of a search at a cost per top-down level that depends on the level rather than on the graph. On the social networks, web graph, road network, Kronecker graph and synthetic graph of hubs that we evaluate, its best times are below those of GAPBS, GBBS and PASGAL; on a long path, PASGAL's best is below that of our method.

Section 2 gives background and states our model. Sections 3 and 4 present the method, Section 5 proves it correct and wait-free and bounds its work, and Section 6 describes the variant that reuses level nodes to reduce space. Section 7 describes our implementation, Section 8 evaluates it, Section 9 discusses related work, and Section 10 concludes.

2 Background and Model

This section reviews level-synchronous and direction-optimizing BFS, defines the progress guarantees we use, relates a wait-free search to the Do-All problems of asynchronous parallel computation, and states the model under which our method is wait-free. Throughout the paper, we say *vertex* for an element of a graph and *node* for an element of a linked list.

2.1 Breadth-First Search

We treat every graph as undirected. A graph $G = (V, E)$ has n vertices, numbered 0 to $n-1$, and we store each of its edges $\{u, v\}$ as the two directed edges (u, v) and (v, u); m counts these directed edges. The graph is kept in compressed sparse row (CSR) form: the neighbors of a vertex u are $adj[offsets[u]]$ up to, not including, $adj[offsets[u+1]]$, and the *degree* of u is $offsets[u+1] - offsets[u]$.

A BFS from a source vertex s computes the distance $\delta(v)$ of every vertex v: the number of edges on a shortest path from s to v, or -1 if there is no such path. Level d is the set $L_d = \{v : \delta(v) = d\}$, so $L_0 = \{s\}$, and D is the largest distance. The search keeps the distances it has found in an array *dist*, which holds -1 for every undiscovered vertex. A vertex is *discovered* when its distance is set, and *expanded* when its neighbors are examined; the level being expanded is the *frontier*.

A top-down step expands the frontier L_d: it examines every neighbor v of every vertex in L_d, and discovers each v that is still undiscovered, setting $dist[v] = d + 1$ and adding v to L_{d+1}. A level-synchronous parallel BFS [4, 38] divides each frontier among the threads; when several threads find the same undiscovered vertex, a compare-and-swap (CAS) on its distance lets only one of them add it to the next level. The step ends at a barrier, since no thread may expand L_{d+1} before all of it has been discovered: otherwise a thread could reach a vertex of L_{d+1} that is still undiscovered from another vertex of L_{d+1}, and give it the distance $d + 2$.

A bottom-up step [5] works the other way around: every undiscovered vertex v scans its own neighbors for one in the frontier, and if it finds one, it sets $dist[v] = d + 1$, adds v to L_{d+1}, and stops scanning. Only the thread that scans v writes its distance, so the step needs no CAS, but it needs a fast test of whether a vertex is in the frontier, such as the bitmap of GAPBS, and it too ends at a barrier. When the frontier is large, a bottom-up step examines far fewer edges than a top-down step: most undiscovered vertices find a frontier neighbor among their first few neighbors, while a top-down step examines every edge of the frontier, most of which lead to vertices that are already discovered.

A direction-optimizing BFS chooses the direction of every step. The rule of Beamer et al., as GAPBS implements it, keeps two counts: m_f, the number of edges of the frontier, which is the sum of the degrees of its vertices, and m_u, an estimate of the edges not yet examined, which starts at m and drops by m_f with every top-down step. A search going top-down switches to bottom-up once $m_f > m_u/\alpha$. A search going bottom-up continues while the frontier does not shrink, or holds more than n/β vertices, and otherwise tests $m_f > m_u/\alpha$ with $m_f = 1$, which switches it back to top-down unless $m_u < \alpha$. GAPBS sets $\alpha = 15$ and $\beta = 18$; our method uses the same rule, which Algorithm 3 states in full (Rule), and our implementation the same values.

2.2 Progress Guarantees

Progress guarantees are properties of methods [25]. A method is *blocking* if a delay of one thread can keep the calls of other threads from finishing, as when a call waits for a lock that a delayed thread holds, or at a barrier that a delayed thread has not reached; a level-synchronous BFS is blocking, since a thread delayed before the barrier of a step delays every other thread there, and one that stops for good keeps them waiting forever. A method is *lock-free* if it guarantees that infinitely often some call finishes in a finite number of steps: the system as a whole makes progress, although a particular call may never finish. A method is *wait-free* if every call finishes in a finite number of the calling thread's own steps, whatever the other threads do [23], and *bounded wait-free* if one bound on that number holds in every execution [23, 25]. The bound may depend on fixed parameters, such as the number of threads, but not on the schedule; in a method that is wait-free but not bounded, a call can take more steps the more other threads have done, as long as it finishes. A call of a wait-free method therefore never waits for another thread, and even a thread that stops for good delays no other. The usual way to make a method wait-free is *helping*: instead of waiting for a delayed thread to finish its part of the work, a thread finishes that part itself [25, 35].

2.3 Wait-Free Algorithms and Do-All

Wait-freedom is hard to achieve when the threads hold inputs that the others do not know: the operations of a concurrent object, or the proposals of a distributed task such as consensus, which no wait-free protocol can solve with reads and writes [18, 23, 24]. A search is different. Every thread knows the whole input, the graph and the source, and the output is a function of it. Wait-freedom alone is then easy to have: each thread runs a sequential BFS by itself and writes the distances, all threads writing the same values, so that every call returns after $O(n + m)$ of its own steps, whatever the other threads do. This trivial method does T times the work of a sequential search, and is no faster than one. The question is how much less work, and how much more speed, wait-freedom allows.

That question has been studied for a basic problem of the same kind. In *Write-All*, P asynchronous or crash-prone processors must write 1 into every cell of an array of N cells [32]; *Do-All* generalizes the cells to N tasks [21], and *Certified Write-All* (CWA) adds a flag that may be set only once every cell is written, which abstracts telling that a set of tasks is complete [41]. The measure is *work*, the total number of steps of all processors [40], by which Anderson and Woll also measured wait-free union-find [2]. Letting every processor do every task takes work $\Theta(PN)$; deterministic and randomized algorithms do far better, and lower bounds are known for several models [3, 13, 31, 36, 41]. Anderson and Woll let each processor visit the cells in a permutation of its own, and bound the redundant work by a property of the permutations that they call their *contention* [3]; Section 5.7 uses their argument. A CWA per step runs a synchronous PRAM program on asynchronous processors [33, 40], as asynchronous PRAM models, which make the speeds of processors part of the model, call for [14, 22, 46], and iterating Do-All in this way has been analyzed as a problem of its own [20].

A level of a BFS is an instance of Do-All: its tasks are the vertices of the level, each weighted by its degree, and the next level may start only once all of them are done. Run with one CWA per level over the n vertices, a search does work at least n in every level, $\Omega(Dn)$ in all, however few vertices the levels hold; on the road network of our experiments, with 6,262 levels and 24 million vertices, that is 1.5×10^{11}, against 58 million directed edges. This cost belongs to that mapping, not to every construction built on CWA. Three things set a level apart from the instances that this literature studies. Its tasks are discovered during the level before, by the same threads, so there is no array of tasks when the level begins. Its tasks have the degrees of the graph as their weights, as skewed as the graph's degrees are. And with direction optimization, the threads must also agree on how to expand it. Section 3 shows how our method handles the first, Section 5.7 what the second costs, and Section 4 how it handles the third.

2.4 System Model

There are $T \geq 1$ threads, numbered 0 to $T - 1$, which communicate through shared memory. The number of threads is fixed before a search, and every thread knows it and its own number. Threads are asynchronous: any thread can be delayed for any length of time between any two of its steps, or stop for good, and we assume nothing about their relative speeds. The graph has $n \geq 1$ vertices, the source of a search is one of them, and the graph does not change during a search.

The shared memory is made of words of up to eight bytes, each aligned to its size, and every value that the method keeps in a word, such as a vertex, a distance, a position in a bucket or a count of edges, fits in it. Every shared variable of the method, such as a distance, a flag, the size of a bucket or a pointer, is a word, and a plan takes a few words. A thread accesses a word with a load, a store or a CAS, each of which is atomic, and in our algorithms every read of a shared variable is one load, every write one store, and every call of CAS one CAS. Memory follows total store order (TSO), the memory model of x86 processors [1, 28, 50]. The stores of a thread become visible to all other threads at the same time, and in the order in which the thread made them, and the loads of a thread take effect in program order. The one reordering that TSO allows is that a load can take effect before an earlier store of the same thread to another location becomes visible, while the store still waits in the thread's store buffer. A CAS waits until every earlier store of its thread is visible, and then reads and writes its word in one atomic step; a *fence* only waits. Both therefore order the accesses of their thread around them. Our method relies on the program order that TSO keeps between two accesses of a thread, a store and a later store, a load and a later load, or a load and a later store, and uses a CAS, or in the method of Section 3 one fence per level, where it needs a store to be visible before a later load. Each load, store, CAS or fence is one step of its thread. The method and its proofs use these operations only through this model, and Section 7 describes how our implementation realizes them.

Each thread allocates the memory it needs during a search from an arena of its own, and we assume that it can allocate memory in a bounded number of its own steps, and draw a random number from a generator of its own in a constant number. The bounds of Section 5 count an allocation and a random draw as a constant number of steps each; an allocator whose cost grows with n or T would leave

the method wait-free, but would add its cost to these bounds. As wait-freedom usually is, our guarantee is stated for this model of primitive operations, not for the allocator, the operating system or the hardware below them. A method that may need unbounded memory loses wait-freedom when memory runs out [44]; a search, by contrast, needs a bounded amount of memory for a fixed graph and number of threads (Section 6.3), so an implementation can meet the assumption by reserving that memory before the search starts (Section 7).

2.5 The BFS Method

Every thread calls the method BFS(s) once per search, with the same source s. Before the first call, $dist[s] = 0$ and $dist[v] = -1$ for every other vertex v, the method's other arrays hold their initial values, and the first level holds the source; Sections 3.1 and 4 give this initial state in full, and Section 6.1 that of the variant of Section 6. Setting up this state, and resetting it between searches, is not part of the method.

When any call returns, $dist[v] = \delta(v)$ for every vertex v, and no step that any thread takes afterwards changes *dist*. A search is therefore complete when its first call returns, and that is when we stop timing it (Section 8.1). From then on, its result can be used while delayed calls may still be running, since they can only write the distances that are already there, and a thread whose call has returned is free to do other work, or to join another search that uses separate state. What waits for the last call is only the reuse or reclamation of the state of the search, which a delayed call may still access: the method guarantees the progress of its calls, not the reclamation of the state that they use, as nonblocking methods generally do, and wait-free reclamation is a subject of its own [45]. A pool of threads can keep several copies of the state, so that a thread that stops for good withholds only one. Section 5 proves these properties, and that every call returns after a bounded number of the calling thread's own steps.

3 A Wait-Free Method for Top-Down BFS

This section presents our method with top-down steps only; Section 4 adds bottom-up steps. In place of the barrier that ends a level, every thread decides on its own when a level is complete, and finishes any part of it that the others have left undone (Figure 1). Algorithm 1 gives the method as thread t runs it.

3.1 Levels and Buckets

The levels of a search form a linked list, whose node N_d describes level d (Figure 2). A node holds its depth d, a pointer *next* to the node of level $d + 1$, which is null until some thread links that node, and one *entry* for each thread. Entry t of N_d holds the *bucket* $B_{d,t}$, a list of vertices of level d that thread t has discovered, and two fields that record how far the expansion of the bucket has got: $last_{d,t}$, the position of the last vertex of $B_{d,t}$ that thread t itself has finished, which starts at -1, and the flag $done_{d,t}$, which a thread sets once it has finished the vertices of the bucket. Only thread t appends to $B_{d,t}$, and only thread t writes $last_{d,t}$.

Besides the distances, the method keeps a flag $expanded[u]$ for every vertex u: a thread sets it once it has examined every neighbor of u, and other threads then skip u. A vertex can be in several buckets of a level, since several threads can discover it at the same time, but every copy is in the right level: a vertex in $B_{d,t}$ is at distance d. Before a search, as Section 2.5 requires, N_0 holds the source s in the bucket of thread 0 and every other bucket of N_0 is empty, every *last* is -1, every flag *done* and *expanded* is unset, $N_0.next$ is null, $dist[s] = 0$, and every other distance is -1.

3.2 An Example

Figure 2 shows a search from A with two threads. Thread 1, whose bucket of level 0 is empty, helps thread 0 with A, and both discover C, so level 1 holds it twice. In level 1, thread 0 reads $dist[F] = -1$ while it expands C, and stalls; thread 1 expands its own bucket, which discovers F, finds the rest of the bucket of thread 0 expanded, expands level 2, finds the buckets of N_3 empty, and returns. The search is complete although thread 0 is still stalled; when it resumes, it appends F to $B_{2,0}$, a late copy that changes nothing (Section 3.4).

3.3 Expanding a Level

A thread expands level d by visiting the T entries of N_d, its own first and then those of the other threads in a random order, drawn anew for every level (ExpandLevel, line 10). It skips an entry whose bucket is empty or whose flag *done* is set; otherwise it reads the size of the bucket, expands the vertices before that position, and sets *done*. In its own bucket, it expands the vertices in order, publishing its position in $last_{d,t}$ after each (line 19); in the bucket of another thread o, it expands only the vertices after position $last_{d,o}$, which o had not finished when the helper read it, in a random order (line 23). Expanding a vertex u does nothing if $expanded[u]$ is set (Expand); otherwise, the thread appends every neighbor v with $dist[v] = -1$ to its own bucket in N_{d+1} and sets $dist[v] = d + 1$, and sets $expanded[u]$ once it has examined every neighbor (line 38).

A level is thus an instance of Certified Write-All (Section 2.3) whose tasks are the positions of the T buckets of N_d, to which their owners may still be appending while the others work through them; since *done* certifies all of a bucket that a thread has read, a thread certifies the whole level for itself by reading T flags, with no pass over the n vertices.

When a thread has visited all T entries of N_d, every vertex of level d has been expanded, by it or by another thread, so every vertex of level $d + 1$ is in some bucket of N_{d+1}, and the thread moves on to N_{d+1} without waiting (line 6). Before it reads the buckets of the new level, it executes a fence (line 4): under TSO, its last appends to N_{d+1} could otherwise still sit in its store buffer, where only it sees them, while others read those buckets (Section 5.6). Slower threads may still be expanding level d; Section 3.4 explains why they cannot harm the levels after it.

3.4 The Order of Writes

Threads skip work on the strength of what others have written: a vertex whose distance is set is not discovered again, one whose flag *expanded* is set is not expanded again, helpers skip the positions up to $last_{d,o}$ of a bucket, and no thread visits a bucket whose flag *done* is set. We call a write that lets other threads skip work a *certificate*, and the method rests on one rule, which Lemma 5.5 proves for each of them: *every certificate comes after the work that it certifies.*

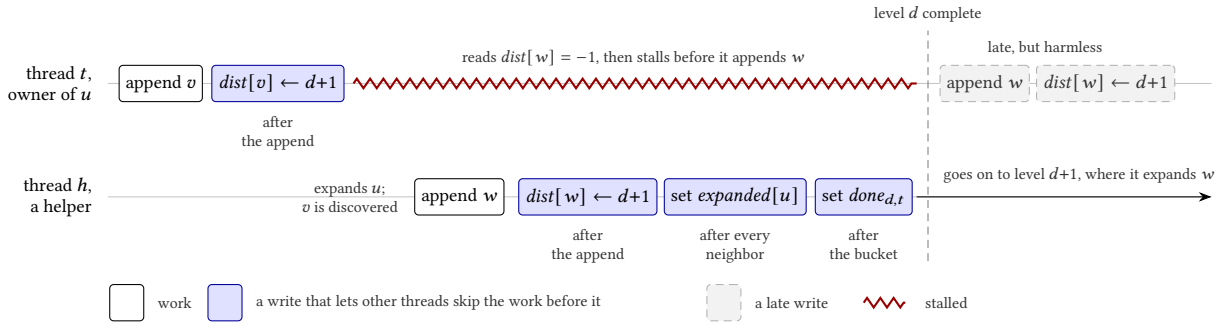


Figure 1: Helping in a top-down level d, with two threads. Thread t stalls while it expands a vertex u of its own bucket, after it reads $dist[w] = -1$. Thread h, helping with the rest of that bucket, finds u past $last_{d,t}$ and $expanded[u]$ unset, expands u again, appends w, and sets $expanded[u]$ and $done_{d,t}$. The late writes of t when it resumes are harmless (Section 3.4).

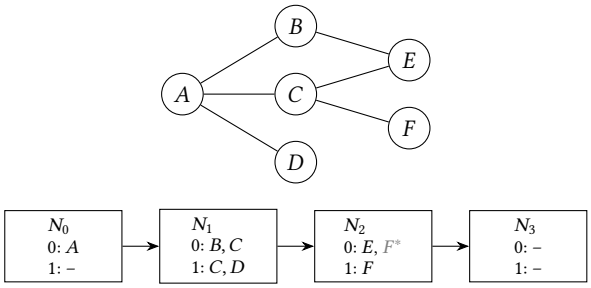


Figure 2: A search from A with two threads; each node lists the buckets of threads 0 and 1. Thread 0 appends the gray F^* after thread 1 has finished the search.

A thread that stops between any two of its steps therefore leaves no work undone that others would skip:

- Since a thread appends v to its bucket before it sets $dist[v]$ (lines 34 and 35), one that stops in between leaves $dist[v] = -1$, and another thread discovers v again. In the other order, a thread that stopped in between would leave v discovered but in no bucket, and no thread would ever expand it.
- A thread that stops in the middle of a vertex u leaves $expanded[u]$ unset (line 38), so others expand u again; they examine the neighbors that it discovered, but do not append them.
- $last_{d,t} = i$ follows the expansion of the vertex at position i, and *done* that of every vertex that the thread read in the bucket (lines 19 and 27).

Figure 1 shows these orders at work, for a thread that stops between appending one neighbor and the next. Every thread that discovers v writes the same distance, $d+1$, so the write needs no CAS. Section 4 replaces the store with a CAS, so that the direction rule counts the edges of each vertex only once.

A delayed thread can also resume long after the others have moved on. Suppose thread t stopped while expanding a vertex u of level d, after it read $dist[w] = -1$ for a neighbor w but before it appended w. Before any thread could finish level d, some thread finished expanding u, and so either appended w or found it discovered, and therefore appended, by another. When t resumes, it appends w to $B_{d+1,t}$, which the others may have expanded already, and writes $dist[w] = d + 1$ again: the distance is the one already there, and the late copy of w duplicates one that the others have expanded or will expand, so a thread that read $B_{d+1,t}$ before the late append missed nothing. Such late appends are also why a vertex can have several copies in a level, and why *done* means that the vertices a thread read in a bucket are finished, not every vertex the bucket will ever hold.

3.5 Buckets

A bucket has to let its owner append vertices, even long after other threads have read it and moved on, while any number of helpers read it by position, without locks, and without ever letting a reader see a position that has not been written or memory that has been freed. It is a dynamic array that only its owner writes (Algorithm 2), so appends need no CAS.

When the array is full, the owner copies the vertices into one twice as large from its arena, or of a small fixed size c_0 at first, and only then publishes it in *array* (line 6); it writes each vertex before it increases *size* (line 10), and only it reads *capacity*. A reader reads in the opposite order, *size* and then *array* (Algorithm 1, line 15), and then the positions before that size, in place. The array that it finds is then the one that was current when that size was published, or a later one, which holds copies of the same vertices (Lemma 5.1). A reader can miss vertices appended after it read the size, but by the time a thread reads the buckets of N_d, every vertex of level d is in them already, and the appends that come later are late copies (Section 3.4).

A replaced array cannot be freed during the search, since a delayed reader may hold its pointer for any length of time; it stays in the owner's arena, which is reset once every call has returned. Since capacities double, the old arrays of a bucket hold fewer positions than its current one, so a bucket of k vertices that has outgrown its first array takes fewer than $4k$ positions in all; an append takes constant time amortized, and a growth copies at most n vertices, since a thread appends each vertex at most once in a search (Lemma 5.12).

3.6 Linking Levels and Ending the Search

A thread that finds a bucket of N_d to expand first makes sure that N_{d+1} exists, since its discoveries go there (line 14): if $N_d.next$ is null,

Algorithm 1 Wait-free top-down BFS, as thread t runs it. For a node $N = N_d$, $N.B[o]$, $N.last[o]$ and $N.done[o]$ stand for $B_{d,o}$, $last_{d,o}$ and $done_{d,o}$.

```
1:  procedure BFS(s)
2:      N ← N_0
3:      while N ≠ null do
4:          fence
5:          ExpandLevel(N)
6:          N ← N.next
7:      end while
8:  end procedure

9:  procedure ExpandLevel(N)
10:     ρ ← t, then the other threads in a random order
11:     for k ← 0 to T − 1 do
12:         o ← ρ[k]
13:         if N.done[o] or N.B[o] is empty then continue
14:         if N.next = null then AddLevel(N)
15:         size ← N.B[o].size; A ← N.B[o].array
16:         if o = t then
17:             for i ← 0 to size − 1 do
18:                 Expand(N, A[i])
19:                 N.last[t] ← i
20:             end for
21:         else
22:             first ← N.last[o] + 1
23:             for all i from first to size − 1, in a random order do
24:                 Expand(N, A[i])
25:             end for
26:         end if
27:         N.done[o] ← true
28:     end for
29: end procedure

30: procedure Expand(N, u)
31:     if expanded[u] then return
32:     for all neighbors v of u, in the order of Section 3.7 do
33:         if dist[v] = −1 then
34:             append v to N.next.B[t]
35:             dist[v] ← N.depth + 1
36:         end if
37:     end for
38:     expanded[u] ← true
39: end procedure

40: procedure AddLevel(N)
41:     M ← the spare node of t, or a new node
42:     M.depth ← N.depth + 1
43:     linked ← CAS(N.next, null, M)
44:     the spare node of t becomes null if linked, and M if not
45: end procedure
```

Algorithm 2 Appending to a bucket B, which only its owner does, and reading it, which any thread can do.

```
1:  procedure Append(B, v)
2:      if B.size = B.capacity then
3:          c ← max(2 · B.capacity, c_0)
4:          A ← an array of c vertices from the owner's arena
5:          copy B.array[0], ..., B.array[B.size − 1] into A
6:          B.array ← A
7:          B.capacity ← c
8:      end if
9:      B.array[B.size] ← v
10:     B.size ← B.size + 1
11: end procedure

12: procedure Read(B)
13:     k ← B.size
14:     A ← B.array
15:     return A and k ▷ the reader then reads A[0], ..., A[k − 1] in place
16: end procedure
```

it prepares a node and tries to link it with a CAS from null (line 43), and if the CAS fails, it uses the node that another thread linked. A node that lost the race was never seen by another thread, so its thread keeps it as a spare for the next level; threads tend to reach a level together, so without spares most of the nodes they prepare would be wasted.

N_{d+1} is linked before any thread expands a vertex of level d or sets a flag *done*, so a thread that skips a done bucket still finds N_{d+1} linked. A level without vertices gets no successor: no thread links a node after it, and every thread that visits it finds *next* null and returns (line 6). The first such level is level $D + 1$, so every call goes through the nodes of levels 0 to $D + 1$ and returns there.

3.7 Keeping Helpers Apart

The method is correct whatever order the threads take entries and neighbors in, and helpers take positions in; the orders we use keep the threads out of each other's way. A thread visits the other entries of a node in a random order of its own; in a fixed order, threads that run out of work at about the same time would walk the same buckets in step. Owners walk their buckets in order, as *last* requires, which also keeps their reads sequential, and a helper takes the positions after the owner's published one in a random order of its own, so helpers on a bucket rarely expand the same vertex at once, or the vertex its owner is on. Likewise, an owner examines the neighbors of a vertex in order, and a helper in a random order of its own, unless the vertex has only a few (Table 1), so that threads that expand a vertex at once examine different neighbors, rather than writing the same entries of *dist* together. A neighbor that is already discovered is not appended again, but it is still examined, so a thread that enters the expansion of a vertex goes through all of its neighbors; Section 7.2 lets threads skip the neighbors of a vertex of high degree in larger parts. Section 7 describes how our implementation draws these orders, and Section 5.7 relates them to the work that threads repeat.

4 Direction Optimization That Keeps the Method Wait-Free

Adding bottom-up levels raises three problems that a blocking implementation solves at its barriers: every thread must choose the same direction for a level, although threads can count the level differently; a bottom-up level works over all undiscovered vertices rather than over buckets, so it needs its own division of work and helping; and its test of whether a vertex is in the frontier must stay exact for a thread that has fallen behind. Algorithm 3 chooses the direction of each level, and Algorithm 4 expands a bottom-up level; top-down levels are expanded as in Algorithm 1, except for how vertices are discovered.

4.1 Agreeing on a Direction

Before it expands a level, a thread must know the level's *plan*: its direction, with the counts that the direction rule of Section 2.1 carries from one level to the next, such as m_u and the size of the previous frontier. All threads must use the same plan, since the fields *last* and *done* of an entry refer to positions in its bucket in a top-down level, and to blocks of vertices in a bottom-up one, so a thread that scanned blocks while another expanded buckets would misread the other's progress and skip work that no thread has done.

A blocking implementation lets one thread decide and makes the others wait. Instead, each node N_d has a field *planner*, initially empty, and each of its entries room for a *proposal* (ChoosePlan). A thread that finds *planner* empty computes a plan with the direction rule (Rule) from the frontier size n_f, the sum of the sizes of the buckets of N_d, the count m_f of the edges of the frontier (below), and the plan of level $d-1$; it writes the plan into its own entry (line 15), and tries once to claim the level with a CAS on *planner* from empty to its own number (line 16). Every thread then adopts the proposal that *planner* names (line 18). Since a thread claims only after writing its proposal, a stopped proposer cannot leave an unpublished decision, and no thread waits (Figure 3a). The first claim comes from a thread that has finished level $d-1$, and like the fence of Algorithm 1, which it replaces, its CAS makes that thread's earlier writes visible, among them its last appends to N_d, before any thread reads those buckets to expand or scan the level, which a thread does only after ChoosePlan returns (Section 5.6); the reads of Rule before the CAS only choose the direction. Our implementation also uses the claim to leave a level early: a thread in level d that finds N_{d+1} linked with its plan claimed goes on to N_{d+1}, testing this before every owner that it visits, top-down, bottom-up or in the pass that writes frontier codes (Section 7.1), and before its own blocks; the level is over by then, since the first claim comes from a thread that has finished N_d (Lemma 5.9). The algorithms leave this test out, since it only ends loops early.

Proposals can differ, since threads read the buckets at different times and late appends change their sizes, but every thread adopts the same one, and which one wins affects only speed: the method finds the same distances in either direction, and ends the search only at a level whose buckets are empty (Section 4.3). The variant of Section 6 ends a search on a plan with $n_f = 0$ instead, and Section 6.2 shows that no level with vertices gets one. Since all threads adopt the same plan for every level, and start from the same plan before level 0, they all propose from the same previous plan.

Algorithm 3 Direction-optimizing BFS, as thread t runs it. ExpandLevel, Expand and AddLevel are those of Algorithm 1, with a call of Discover in place of lines 34 and 35 of Expand. A plan P holds a direction, the estimate $P.m_u$ of the edges not yet examined, and the size $P.n_f$ of the frontier of its level. The plan before level 0 is (top-down, m, 1), and $N_0.edges[0]$ holds the degree of s.

```
1:  procedure BFS(s)
2:      N ← N_0; P ← the plan before level 0
3:      while N ≠ null do
4:          P′ ← ChoosePlan(N, P)
5:          if P′.direction is top-down then
6:              ExpandLevel(N)
7:          else
8:              ScanLevel(N)
9:          end if
10:         P ← P′; N ← N.next
11:     end while
12: end procedure

13: procedure ChoosePlan(N, P)
14:     if N.planner is empty then
15:         N.proposal[t] ← Rule(P, N)
16:         CAS(N.planner, empty, t)
17:     end if
18:     return N.proposal[N.planner]
19: end procedure

20: procedure Rule(P, N)   ▷ the rule of GAPBS, from the plan P of the level before
21:     n_f ← the sum of the sizes of the buckets of N
22:     m_f ← 1 if P is bottom-up, and otherwise the sum of N.edges[o] over all o
23:     if n_f = 0 then return (top-down, P.m_u, 0)
24:     if P is bottom-up and (n_f ≥ P.n_f or n_f > n/β) then
25:         return (bottom-up, P.m_u, n_f)
26:     end if
27:     if m_f > P.m_u/α then return (bottom-up, P.m_u, n_f)
28:     return (top-down, P.m_u − m_f, n_f)
29: end procedure

30: procedure Discover(N, v)                ▷ in top-down levels
31:     append v to N.next.B[t]
32:     if CAS(dist[v], −1, N.depth + 1) then
33:         add the degree of v to N.next.edges[t]
34:     end if
35: end procedure
```

The rule needs m_f, the number of edges of the frontier, which the top-down level that discovered the frontier counts. With the stores of Section 3.4, a vertex discovered by two threads at once would have its edges counted twice, which would overstate m_f, so that the rule could switch to bottom-up a level too early, and run m_u down too fast. Top-down levels therefore discover a vertex with a CAS on its distance from -1 to $d+1$, still after appending it (Discover), and only the thread whose CAS succeeds adds the

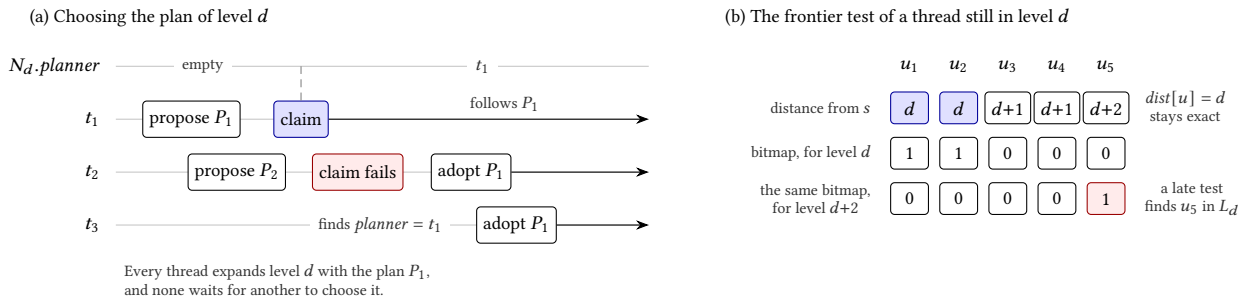


Figure 3: Direction optimization without a barrier. (a) Threads t_1 and t_2 both propose a plan for level d and try to claim it; the CAS of t_1 wins, and t_2 and the later t_3 adopt the proposal that t_1 wrote before its claim. (b) A thread still in level d tests the frontier L_d late: the reused bitmap of GAPBS may by then describe L_{d+2} and put u_5 in L_d, while the test $dist[u] = d$ stays exact.

degree of the vertex to the edge count of its entry in N_{d+1}; a thread whose CAS fails has found the vertex discovered by a thread that appended it first, so the order of Section 3.4 still holds. The count can fall short instead, if a thread stops between its CAS and adding the degree, or has not added it yet when a planner reads the count, which again changes only the direction chosen. After a bottom-up level, the rule takes $m_f = 1$, as GAPBS does.

Algorithm 4 A bottom-up level, as thread t runs it. $X_{o,k}$ is the k-th block of thread o, K_o the number of its blocks, and every stamp starts at -1.

```
1:  procedure ScanLevel(N)
2:      if N.next = null then AddLevel(N)
3:      for k ← 0 to K_t − 1, while not N.done[t] do
4:          if stamp[t][k] ≠ N.depth then ScanBlock(N, X_{t,k})
5:          N.last[t] ← k
6:      end for
7:      N.done[t] ← true
8:      for all threads o ≠ t, in a random order do
9:          for all blocks k > N.last[o], in a random order do
10:             if N.done[o] then break
11:             if k > N.last[o] and stamp[o][k] ≠ N.depth then
12:                 ScanBlock(N, X_{o,k})
13:                 stamp[o][k] ← N.depth
14:             end if
15:         end for
16:         N.done[o] ← true
17:     end for
18: end procedure

19: procedure ScanBlock(N, X)
20:     for all v ∈ X with dist[v] = −1 do
21:         if some neighbor u of v has dist[u] = N.depth then
22:             append v to N.next.B[t]
23:             dist[v] ← N.depth + 1
24:         end if
25:     end for
26: end procedure
```

4.2 Bottom-Up Levels

A bottom-up level divides the vertices into blocks of W consecutive vertices, the last possibly shorter: block b holds the vertices Wb to $\min(W(b+1), n) - 1$, so there are $\lceil n/W \rceil$ blocks (Table 1 lists W with the other parameters of our implementation). A level gives block b to thread $b \bmod T$, so the k-th block $X_{o,k}$ of thread o is block $o + kT$, and thread o has $K_o = \lceil (\lceil n/W \rceil - o)/T \rceil$ blocks, or none if $o \geq \lceil n/W \rceil$, so that the blocks of every thread spread over the whole range of vertex numbers, and a level whose undiscovered vertices lie mostly in one part of the range still gives every thread a share. Blocks take the place of buckets (ScanLevel): a thread scans its own blocks in order, publishing in $last_{d,t}$ the index k of the last one it has finished (line 5), and then helps the other threads, in a random order as in a top-down level, with the blocks after their *last*, in a random order of its own (line 9). The flag $done_{d,o}$ is set once all of the blocks of thread o are finished, by o or a helper, and the owner and helpers stop once they see it set.

A helper cannot rely on the owner's *last* alone, since a delayed owner stops moving it, and every helper would then scan all of the owner's remaining blocks. A helper therefore stamps every block that it finishes with the depth of the level (line 13), and the others skip a block whose stamp equals the current depth. A stamp left by an earlier level, or written late by a thread that has fallen behind, holds an older depth, which makes a thread scan a block again, but never skip one that needs scanning.

Scanning a block tests every undiscovered vertex v of the block (ScanBlock): if v has a neighbor in the frontier, the thread appends v to its own bucket in N_{d+1}, and only then sets $dist[v] = d + 1$ (lines 22 and 23), so a thread that finds v discovered finds it in a bucket. Threads that discover the same vertex at once write the same values, so these writes need no CAS, and since the discoveries go into buckets, the next level can go in either direction, and its plan can count its frontier.

The test of the frontier is $dist[u] = d$ (line 21). GAPBS tests a denser bitmap instead, swapping two bitmaps after every step and clearing one before the step fills it. That would not do for our method: a thread that has fallen behind in level d would test against a bitmap that the others, by then in level $d + 1$, are filling with L_{d+2}, and would give the undiscovered neighbors of those vertices the distance $d + 1$ (Figure 3b); and since a thread can fall behind in any

level, no bitmap could be reused during the search, so every bottom-up level would need n bits of its own. The test of the distance is exact for any thread, however far behind, since a distance is written only with its final value and never changes once set (Section 5.3). Our implementation tests a denser array of one byte per vertex that keeps this exactness (Section 7.1).

4.3 Ending the Search

As in Section 3, the search ends at the first level without vertices, and only a top-down level ends it: a bottom-up level always links a successor, since it cannot tell in advance whether it will discover anything, while every proposal for a level whose buckets are all empty finds $n_f = 0$, for which Rule chooses top-down (Algorithm 3, line 23), so every thread that enters that level finds its buckets empty, links no successor, and returns.

5 Correctness and Wait-Freedom

This section proves that the method of Section 4 meets the specification of Section 2.5: when any call of BFS(s) returns, *dist* holds the distances from s, and no later step changes them (Theorem 5.11), and every call returns after a bounded number of the calling thread's own steps (Theorem 5.13). The method is that of Algorithms 3 and 4, which expand top-down levels with Algorithm 1 and Discover, and keep their buckets as in Algorithm 2. We reason about the loads, stores, CASes and fences of the algorithms in the model of Section 2.4, in which every thread makes them in the order that the algorithms list them. Sections 5.1 to 5.5 assume sequential consistency: the memory accesses of all threads take effect one at a time, in an order that keeps the program order of every thread; Section 5.6 shows that the proof holds under TSO as well. The proof never assumes that a thread takes another step, so it holds whatever delays the threads suffer, even if some of them stop for good. Section 5.7 bounds the work of a search.

5.1 Definitions

In this section we write $\delta(v) = \infty$ for a vertex that s cannot reach, so that the levels $L_0, \ldots, L_D$, where D is the largest finite distance, hold exactly the vertices that s reaches. Lemma 5.2 shows that the nodes form a single list, in which N_d is the node of depth d. A thread *enters* N_d when it calls ChoosePlan(N_d), and *finishes* N_d when its call of ExpandLevel(N_d) or ScanLevel(N_d) returns; from the time its call of ChoosePlan(N_d) returns until it finishes N_d, the thread is *in level* d. A thread appends to the buckets of N_{d+1}, writes *dist*, *expanded* and stamps, and writes *last* and *done* of N_d only while it is in some level d, possibly long after the others moved on.

Let τ_d be the time of the first CAS on N_d.*planner* (Algorithm 3, line 16), if there is one; we then say that the plan of N_d is *claimed*. A thread can enter N_d and stop before its CAS, so the plan of a node that some thread enters need not be claimed. If some call of ChoosePlan(N_d) returns, though, the plan is claimed, and τ_d comes before the call returns, since a call returns after its own CAS, or after reading *planner* set by an earlier one. Every read that a thread in level d makes of the buckets of N_d thus comes after τ_d. The *reference prefix* $S_{d,t}$ is the set of vertices at positions 0 to $k-1$ of $B_{d,t}$, where k is the size of $B_{d,t}$ at time τ_d. By Algorithm 2 and Lemma 5.1, these positions keep their vertices for the rest of the search, and a thread that reads the size of $B_{d,t}$ after τ_d reads at least k.

A call of Expand(N, u) *expands* u *completely* if it examines every neighbor of u and then sets *expanded*[u] (Algorithm 1, line 38). Every call that returns has either expanded u completely or read *expanded*[u] set by a call that did. A block is *scanned in level* d once a call of ScanBlock for it by a thread in level d has returned.

5.2 Buckets, Levels, and Plans

Lemma 5.1 (Buckets). *If a thread reads k as the size of a bucket B and afterwards reads the pointer B.array, then positions 0 to $k-1$ of the array it points to hold the first k vertices appended to B, in the order of their appends, and keep them for the rest of the search.*

Proof. Only the owner of B writes the bucket. It writes each vertex into the current array before increasing *size* (Algorithm 2, line 10). When the array is full, it copies the vertices appended so far into a larger array, and only then points B.*array* to the new array (line 6). So when the size became k, the array that B.*array* pointed to held the first k vertices at positions 0 to $k-1$, and each array that replaced it later received copies of them before B.*array* pointed to it. The reader reads B.*array* after the size, so it finds a pointer to that array or to one that replaced it, and either one holds the first k vertices at positions 0 to $k-1$. No array is freed during the search, and no position of an array is written twice, so these positions keep their vertices. □

Lemma 5.2 (The level list). *Every node has at most one successor, which is linked by a CAS from null and never replaced, and a node is fully set up, with its depth set, its buckets empty and its planner empty, before it is linked. The nodes linked from N_0 thus form a single list, in which the node d steps from N_0 has depth d, and every thread walks this list from N_0, in order.*

Proof. A successor is linked only by the CAS in AddLevel (Algorithm 1, line 43), from null, so at most one CAS on the field *next* of a node succeeds, and nothing writes the field afterwards. A thread sets up its node before trying the CAS. A node whose CAS fails has been seen by no other thread, since no pointer to it was ever published, and its thread changes only its depth before trying it again at a later level. A node linked to a node of depth d has depth $d+1$ (AddLevel). Every thread starts at N_0 and moves on only along *next*. □

Lemma 5.3 (Plans). *For every node N_d, calls of ChoosePlan(N_d) that return all return the same plan, P_d, and every thread passes P_{d-1} to its call for N_d, where P_{-1} is the plan before level 0.*

Proof. Only CASes from empty write N_d.*planner* (Algorithm 3, line 16), so the first one succeeds, and *planner* never changes after it. The thread C whose CAS succeeds wrote its proposal before its CAS (line 15), and writes it only once, since it calls ChoosePlan(N_d) once. Every call reads *planner* set, and then returns the proposal of the thread it names (line 18), which is that of C. The second claim follows by induction on d, since a thread calls ChoosePlan(N_{d+1}) with the plan that its call for N_d returned. □

5.3 Safety

The proof rests on two properties. The first is that every write is *sound*:

(S1) every value $x \geq 0$ written to $dist[v]$ is $\delta(v)$;
(S2) every vertex appended to a bucket of N_d is in L_d.

The second is that every level is *ready* by the time its plan is claimed: for every node N_d whose plan is claimed, at time τ_d,

(R1) $dist[u] = \delta(u)$ for every vertex u of $L_0 \cup \cdots \cup L_d$;
(R2) every vertex of L_d is in the reference prefix $S_{d,t}$ of some thread t.

Sound writes never change a distance once it is set, since all writes to $dist[v]$ write $\delta(v)$. So as long as every write is sound, (R1) keeps holding after τ_d; (R2) does too, by Lemma 5.1. Lemmas 5.4 to 5.7 assume that every write made so far is sound, and that (R1) and (R2) hold for N_d; Lemma 5.8 then proves both properties together.

Lemma 5.4 (Discoveries). *If a thread in level d appends a vertex v to a bucket of N_{d+1}, then $\delta(v) = d + 1$, and the value it writes to $dist[v]$ is $d + 1$.*

Proof. In a top-down level, the thread appends v while expanding a vertex u taken from a bucket of N_d, so $\delta(u) = d$ by (S2), or as $u = s$ if $d = 0$, and $\delta(v) \leq d + 1$. It appends v only after reading $dist[v] = -1$ in that expansion, after τ_d, when (R1) holds; hence $\delta(v) > d$. Its CAS writes $d + 1$ (Algorithm 3, line 32), which our implementation computes as $dist[u] + 1$, equal to $d + 1$ by (R1). In a bottom-up level, the thread appends v after reading $dist[v] = -1$ after τ_d, so again $\delta(v) > d$, and after finding a neighbor u of v with $dist[u] = d$, so that $\delta(u) = d$ by (S1). So $\delta(v) = d + 1$, and the thread writes $d + 1$. Neither argument depends on how long the thread waits between its reads and its writes, since (R1) keeps holding. □

Lemma 5.5 (Certificates). *The writes that let other threads skip work come after that work:*

(a) *When a thread sets $expanded[u]$, every neighbor v of u with $\delta(v) = \delta(u) + 1$ has been appended to a bucket of $N_{\delta(v)}$ and has $dist[v] = \delta(v)$.*
(b) *In a top-down level d, when thread o sets $last_{d,o} = i$, the vertices at positions 0 to i of $B_{d,o}$ have been expanded completely, and when a thread sets $done_{d,o}$, every vertex of $S_{d,o}$ has been.*
(c) *In a bottom-up level d, when thread o sets $last_{d,o} = k$, its blocks 0 to k have been scanned in level d; when a thread stamps block k of o with d, that block has been; and when a thread sets $done_{d,o}$, every block of o has been.*

Proof. Each of these writes follows, in the program order of its thread, either the work it certifies, or the reading of an earlier certificate for that work.

(a) A thread sets $expanded[u]$ only after examining every neighbor v of u (Algorithm 1, line 38). For each v, it either read $dist[v] = -1$ and appended v before its CAS on $dist[v]$, which writes $\delta(v)$ or fails because $dist[v]$ is already set; or it read $dist[v]$ set, by a thread that had appended v before, since both Discover and ScanBlock append a vertex before writing its distance. The appends were to a bucket of $N_{\delta(v)}$, by (S2).

(b) Thread o sets $last_{d,o} = i$ after its calls of Expand on positions 0 to i, in order, have returned (Algorithm 1, line 19), and a call returns only after u has been expanded completely. A thread sets $done_{d,o}$ (line 27) after reading the size of $B_{d,o}$ after τ_d, so that the positions it read cover $S_{d,o}$, and after calling Expand on every one of them, or, as a helper, on every one after $last_{d,o}$ as it read it, the positions up to which are covered by the first claim.

(c) Thread o takes its blocks in order, and publishes $last_{d,o} = k$ (Algorithm 4, line 5) after scanning block k, or after reading its stamp equal to d; a helper stamps a block (line 13) only after scanning it. A stamp equal to d was written after a scan in level d: stamps start at -1 in every search, and a thread in a level other than d stamps with another depth. The owner sets $done_{d,o}$ after its loop over all of its blocks, which it leaves early only on reading $done_{d,o}$ set already. A helper sets it after taking every block after $last_{d,o}$ as it read it, the blocks up to which are covered by the first claim, skipping a block only if the block is now at or before $last_{d,o}$, or stamped with d, and stopping only on reading $done_{d,o}$ set already. □

Lemma 5.6 (Top-down levels). *If P_d is top-down, then when a thread finishes N_d, every vertex v of L_{d+1} has been appended to a bucket of N_{d+1} and has $dist[v] = d + 1$, and if L_d is not empty, N_{d+1} has been linked.*

Proof. Let X be the thread, let $v \in L_{d+1}$, and let u be a neighbor of v in L_d. By (R2), u is at some position i of a reference prefix $S_{d,o}$, and X visited entry o of N_d after τ_d. If it found $done_{d,o}$ set, u had been expanded completely, by Lemma 5.5(b). Otherwise, the size of $B_{d,o}$ that X read was more than i, so the bucket was not empty. If $o = X$, X called Expand on position i; if not, it either did so or found $last_{d,o} \geq i$, in which case u had been expanded completely, by Lemma 5.5(b). A call of Expand returns only after u has been expanded completely, so in every case u has been, and by Lemma 5.5(a), v has been appended to a bucket of N_{d+1} and has $dist[v] = d + 1$. Finally, if L_d is not empty, take any $u \in L_d$ and its entry o as above. Either X found $done_{d,o}$ set, by a thread that had linked N_{d+1} or found it linked before it took the bucket (Algorithm 1, line 14), or X took the bucket itself and did the same. □

Lemma 5.7 (Bottom-up levels). *If P_d is bottom-up, then when a thread finishes N_d, N_{d+1} has been linked, and every vertex v of L_{d+1} has been appended to a bucket of N_{d+1} and has $dist[v] = d + 1$.*

Proof. Let X be the thread. It linked N_{d+1}, or found it linked, before scanning (ScanLevel). Let $v \in L_{d+1}$, in block k of thread o. Thread X took every block of its own and then visited every other thread, so by Lemma 5.5(c), whichever way it passed block k of o, whether by scanning it, finding it at or before $last_{d,o}$ or stamped with d, or finding $done_{d,o}$ set, the block has been scanned in level d by some thread Y. The call of ScanBlock of Y read $dist[v]$ after τ_d. If it read a value other than -1, the thread that wrote it had appended v before, to a bucket of N_{d+1}, since by (S1) and Lemma 5.4 it was in level d. Otherwise, Y tested the neighbors of v, among them a vertex u of L_d, for which the test $dist[u] = d$ holds by (R1). The test thus succeeds, and Y appended v and set $dist[v] = d + 1$ (Algorithm 4, lines 22 and 23). □

Lemma 5.8 (Soundness and readiness). *Every write is sound, and (R1) and (R2) hold for every node N_d whose plan is claimed, from τ_d to the end of the search.*

Proof. Suppose some write is not sound, and let w be the first such write. We first show by induction on d that (R1) and (R2) hold for every node N_d with τ_d before w, from τ_d up to w. For N_0 they hold before any call starts (Section 3.1). For $d > 0$, the thread whose CAS on N_d.*planner* came first finished N_{d-1} before τ_d, after τ_{d-1}. By induction, (R1) and (R2) held for N_{d-1} in that time, and every write before w is sound, so by Lemma 5.6 or 5.7, when it finished N_{d-1}, every vertex of L_d had been appended to a bucket of N_d and had its distance. The appends came before τ_d, so they are in the reference prefixes, and (R1) for the earlier levels holds by induction. This gives (R1) and (R2) at τ_d, and they keep holding up to w, since the writes before w are sound. Now w is made by a thread in some level d, after the reads of that thread that it depends on, which came after τ_d. Lemma 5.4 then shows that w is sound, a contradiction. Every write is therefore sound, and the induction above, which then needs no bound w, gives (R1) and (R2) for every node whose plan is claimed. □

Lemma 5.9 (Leaving a level early). *Once the plan of N_{d+1} is claimed, the work of level d is done: every vertex of L_{d+1} is in a reference prefix of N_{d+1} and has its distance. A thread in level d that finds the plan of N_{d+1} claimed can therefore stop expanding or scanning N_d, or writing its frontier codes, and go on to N_{d+1}, and the lemmas above and below hold for the implementation that does so.*

Proof. By Lemma 5.8, (R1) and (R2) hold for N_{d+1} from τ_{d+1} on. The thread whose claim came first finished N_d before it without leaving early, since no claim was there to read; every thread that leaves early relies on a claim already made. To the other threads, a thread that leaves N_d early is no different from one that stops there for good, which the lemmas above allow, since every write that lets threads skip work comes after the work that it covers (Section 3.4). The thread itself enters N_{d+1} after τ_{d+1}, with the plan P_d that it adopted, as a thread that finished N_d would, and no lemma about N_{d+1} depends on how a thread got there. The level with no successor, N_{D+1}, has no plan of a later node to test, so a call still returns only after finishing it (Lemma 5.10). Each test reads *next* and *planner*, a constant number of steps for each owner that the thread visits, which changes the bounds of Section 5.5 by a constant factor at most. □

5.4 The End of the Search

Lemma 5.10 (The last level). *At any time, the list consists of the nodes N_0 to N_k for some $k \le D + 1$. A thread that finishes N_d for $d \le D$ finds N_{d+1} linked, and a thread that finishes N_{D+1} returns. A call that returns has thus gone through the nodes N_0 to N_{D+1}.*

Proof. How far the list reaches depends on how far the threads get: if all of them stop early, it can end before N_{D+1}. By (S2), every vertex appended to a bucket of N_{D+1} is in L_{D+1}, which is empty, so its buckets stay empty, at all times. Every proposal for N_{D+1} therefore finds $n_f = 0$, and P_{D+1} is top-down (line 23 of Algorithm 3). In ExpandLevel(N_{D+1}), every entry is empty, so no thread calls AddLevel(N_{D+1}), the only call that can link a successor to N_{D+1}, and the list never reaches past N_{D+1}; a thread that finishes N_{D+1} finds *next* null and returns. For $d \le D$, L_d is not empty, so by Lemmas 5.6 and 5.7, N_{d+1} has been linked when a thread finishes N_d, and the thread finds it (Algorithm 1, line 6). □

Theorem 5.11 (Correctness). *When any call of BFS(s) returns, $dist[v] = \delta(v)$ for every vertex v that s reaches, and $dist[v] = -1$ for every other vertex, and no later step of any thread changes dist.*

Proof. By Lemma 5.10, a call returns only after finishing N_{D+1}, and hence after τ_{D+1}, when (R1) holds for N_{D+1}: every vertex of $L_0 \cup \cdots \cup L_D$, which are all the vertices s reaches, has its distance. By (S1), no thread writes a value $x \ge 0$ to $dist[v]$ for a vertex v that s does not reach, so these distances stay -1. Every later write to $dist[v]$ writes $\delta(v)$, the value already there. □

The method of Section 3 is the special case in which every plan is top-down, and the same proof applies to it, with τ_d the time of the first fence that a thread executes before expanding N_d (Algorithm 1, line 4), and a thread in level d from its own such fence until it finishes N_d.

5.5 Wait-Freedom

Lemma 5.12 (Appends). *In a search, a thread appends each vertex at most once, so all the buckets together hold at most $Tn + 1$ vertices.*

Proof. A thread appends a vertex v only after reading $dist[v] = -1$, and right after appending it, the thread writes $dist[v]$, or its CAS on it fails because $dist[v]$ is already set. A distance never returns to -1, so the thread never reads $dist[v] = -1$ again. The one vertex not appended this way is the source in N_0. □

Theorem 5.13 (Wait-freedom). *Every call of BFS(s) returns after $O(Tn + (D + 1)(n + m + T))$ steps of its own thread, whatever the other threads do.*

Proof. A call walks the list once, from N_0 to N_{D+1} (Lemmas 5.2 and 5.10). No loop of the method waits for another thread: every loop runs over a range fixed when it starts, which is the T entries of a node, the positions of a bucket up to the size read, the neighbors of a vertex, or the blocks of a thread, and tests of another thread's progress only skip work or end a loop early. Each CAS is tried once. We count the steps of one call. Choosing a plan reads the T entries of a node and makes at most one CAS, and linking a successor sets up T entries and makes one CAS: $O(T)$ steps per level. Over the whole search, a thread takes every position of every bucket at most once, at most $Tn + 1$ positions in all by Lemma 5.12, and it expands each vertex at most once, since it then reads its own $expanded[u]$, so it examines at most m neighbors in top-down levels in all. In a bottom-up level, it scans each block at most once, reading each of its vertices and the neighbors of the undiscovered ones: $O(n+m)$ steps per level. Appends take constant time amortized, and the copies made as buckets grow add up to fewer positions than the buckets' final arrays, $O(Tn)$ in all. An order of $k \le 1$ items is the identity, and takes a constant number of steps. An order of $k \ge 2$ items, an affine permutation (Section 7), takes two random numbers, a constant number of steps each (Section 2.4), and a stride coprime to k, which takes $O(\sqrt{k} \log k) \subseteq O(k)$ steps to find (Lemma A.1 in Appendix A). Drawing an order of k items therefore takes $O(k+1)$ steps. A thread orders the entries of each node, the remaining positions of each bucket and, in each bottom-up level, the remaining blocks of each owner at most once, and the neighbors of each vertex at most once, so its orders cover no more items than the counts above. Every allocation takes a bounded number of steps (Section 2.4). The sum

is $O(Tn + (D+1)(n+m+T))$: $O(T)$ for each of the $D+2$ levels, $O(Tn+m)$ for the top-down levels together, and $O(n+m)$ for each bottom-up level, of which there are at most $D+1$, since level $D+1$ is top-down (proof of Lemma 5.10). It does not depend on the steps of any other thread. □

The bound is loose, and says nothing about speed; what matters is that it depends only on n, m, T and D, which are fixed for a search, and not on the other threads or the schedule, so the method is bounded wait-free (Section 2.2).

5.6 Total Store Order

In our model, memory follows TSO rather than sequential consistency (Section 2.4). We model a TSO execution in the usual way [50]: a store goes first to its thread's store buffer, and later, in program order, to memory; a load returns the newest store to its location in its thread's buffer if there is one, and the value in memory otherwise; a CAS waits until its thread's buffer is empty, and then acts on memory atomically; and a fence waits until its thread's buffer is empty. We take the time of a store to be the time it reaches memory. Two properties then hold that sequential consistency gives: the stores of a thread reach memory in program order, and its loads and stores take effect in program order, except that a load can take effect before an earlier store of its thread reaches memory.

Every step of the proof draws its conclusions in one of two ways. In the first, a thread reads its own stores, which it always sees. In the second, a thread X reads a value that another thread Y stored, and concludes that it also sees the stores that Y made, or read from other threads, before that store. Three properties of TSO make this hold. The stores of Y reach memory in program order, so its earlier stores are there by the time the one that X read is. A load of Y takes effect before the stores that follow it, so the stores of other threads that Y read had reached memory before its own store did. And the loads of X take effect in program order, so the loads it makes afterwards find all of these stores, or later ones to the same locations.

Only one step needs more. (R2) must hold in memory at τ_d, but the thread that finished N_{d-1} first may still hold some of its appends to the buckets of N_d in its store buffer. This is why τ_d is the time of the first CAS on $N_d.planner$, or, in the method of Section 3, of the first fence before N_d. The thread that makes this CAS or fence finished N_{d-1} before it, and either one empties its store buffer. At τ_d, memory therefore holds every append that this thread made or saw, which covers all of L_d, together with their distances. Every thread reads the buckets of N_d to expand or scan the level only after its call of ChoosePlan(N_d) returns, or after its own fence, so after τ_d. The reads of Rule, in ChoosePlan(N_d), can come before τ_d and miss vertices of L_d, but they only choose the direction, on which no lemma above depends: Lemma 5.3 needs only that every thread adopts the same plan, and the search ends only at a level whose buckets are empty at all times (Lemma 5.10). Every lemma above therefore holds under TSO, with "has been appended" and "has been set" read as "has reached memory, or is in the thread's own store buffer".

5.7 Work

Theorem 5.13 bounds the steps of one call. The *work* of a search, the steps of all of its calls together (Section 2.3), is at most T times that bound, no better than the work of the trivial method. This section bounds the work of a top-down level by that level and T, shows that any method of this kind can be made to repeat, once per thread, any piece of work that it certifies as a whole, and bounds what threads repeat when they take work in random orders.

We write Q_d for the number of positions in the buckets of N_d at the end of the search, and $\mathrm{vol}(L_d) = \sum_{u \in L_d} \deg(u)$ for the number of neighbors of the vertices of level d. A thread appends each vertex at most once (Lemma 5.12), only vertices of L_d go into the buckets of N_d (S2), and all of them do (R2), so $|L_d| \le Q_d \le T|L_d|$.

Top-down levels. Unlike a level run as a CWA over all n vertices (Section 2.3), a top-down level of our method costs only what it holds:

Theorem 5.14 (Work of a top-down level). *If P_d is top-down, the threads together take $O(T(Q_d + \mathrm{vol}(L_d)) + T^2)$ steps in level d, with appends counted at their amortized cost.*

Proof. Each thread plans the level and visits its T entries in $O(T)$ steps. It reads each position of each bucket of N_d at most once, and expands each vertex at most once, since afterwards it finds *expanded* set, so it examines each neighbor of each vertex of L_d at most once. Each of these accesses takes a constant number of steps, and drawing a random order of k items takes $O(k+1)$ steps (proof of Theorem 5.13). □

The bound still allows each thread to repeat all of the level, as the trivial method does, and Section 8.3 measures how much threads repeat in practice. The rest of this section asks how much of that repetition can be avoided.

Unavoidable repetition. In our method, a thread skips work only when it reads a certificate that the work has been done, and some repetition then cannot be avoided, in our method or in any other that works this way. A *piece* of work is a sequence of w steps that any thread can take, and that some thread must take in full before any call returns, such as examining the neighbors of a vertex. A *certificate* of a piece, or of a part of one, is a write that a thread makes only after the piece or part has been done, by itself or by threads whose writes it has read; a thread that reads the certificate may skip the piece or part. In our method, $expanded[u]$ certifies the expansion of u, and $dist[v]$, which a thread writes after appending v, certifies the discovery of v.

Proposition 5.15 (Certification granularity). *Consider a wait-free method in which a thread may skip a piece of work only after reading a certificate of it, and a piece of w steps.*

(a) *If the piece is certified only as a whole, then there is a schedule in which the threads together take at least $T(w-1)+1$ steps on it.*
(b) *If it is certified in parts, each part with a certificate of its own written after the part, and each thread does a part at most once, certifying it before it starts another, then a thread that stops is inside at most one part that it has started and not certified; the parts that it has not started stay whole for the*

others, who skip those certified by then. If the parts are the $\lfloor w/g \rfloor$ consecutive chunks of g steps and, if g does not divide w, a final shorter chunk, then a stopped thread leaves at most g of its steps uncertified, the piece has $\lceil w/g \rceil$ certificates, and the threads together write them at most $T\lceil w/g \rceil$ times.

Proof. (a) Until the certificate is written, a call can return only after doing the whole piece itself. Run one thread alone until it takes its first step on the piece, and stop it there. Then run another alone: it can neither skip the piece nor return without doing it, so, since the method is wait-free, it too takes a first step on the piece; stop it there, and do the same with every other thread. Then run all T threads in turn, one step on the piece each. The certificate comes only after some thread has taken all w steps of the piece, and by then every other thread has taken at least $w - 1$. (b) A thread certifies each part before it starts another, so when it stops, it is inside at most one part, and every part it has finished is certified. With the chunks of g steps, the part it is inside has at most g steps, there are $\lceil w/g \rceil$ parts, each with one certificate, and each thread writes each certificate at most once. □

For BFS, a piece is the expansion of a vertex u, which examines its $\deg(u)$ neighbors. Our method certifies the expansion as a whole, with *expanded*$[u]$. The distance of each neighbor certifies the neighbor's discovery, which is the case $g = 1$ of part (b), but these certificates save no work, since reading a neighbor's distance is the examination of that neighbor. A thread that enters the expansion of u therefore examines all of its neighbors, and appends only those not yet discovered. By part (a), a vertex of high degree can thus cost $T(\deg(u) - 1) + 1$ steps; Section 8.6 measures the effect on a graph with frontiers made of such vertices. Parts of several neighbors avoid this, at the cost of one write per part: a thread reads one certificate instead of examining a part, and a thread that stops in a vertex leaves at most one started part, of at most g examinations, uncertified, instead of up to $\deg(u)$. Section 7.2 describes how our implementation does this for vertices of high degree. At the other extreme, the trivial method treats the whole search as one piece.

Part (a) applies to every part, though: threads that take the parts in the same order can all repeat every one of them. Against such schedules, parts help only if threads take them in different orders, which is why our helpers take the positions of a bucket, and the chunks of a vertex, in random orders.

Random orders. What random orders achieve rests on the following observation.

Lemma 5.16 (Records). *Let a thread h take positions of a bucket $B_{d,o}$. Then h expands the vertex at a position x only if x is a* record *of the order in which h takes the positions: every position that h took before x has its vertex marked in expanded before the vertex at x is first marked.*

Proof. When h takes position x and finds *expanded* unset for its vertex, that vertex is marked only later. Every position that h took before x has its vertex marked by then: h found *expanded* set for it, or expanded it completely, setting *expanded* itself, before moving on. □

The lemma needs marks that are never unset: the flags *expanded* are not, and neither are the certificates of the chunks of Section 7.2, for which the lemma holds in the same way. It does not hold for the blocks of a bottom-up level. A thread that has fallen behind can overwrite the stamp of a block with the depth of its own, older level (Section 4.2), so that a block certified for the current level appears uncertified again, and a helper that finds it so scans it although it is not a record. This is the argument of Anderson and Woll for Write-All [3]: threads repeat work only at the records of their orders, relative to the order in which the work is certified. That order is up to the schedule, and an adversary can choose it to give the threads many records, but if each thread's order is chosen uniformly at random, its records are few against any schedule.

Theorem 5.17 (Repeated expansions with random orders). *Suppose that $n \geq 2$, and that memory is sequentially consistent or a thread executes a fence after every mark (see below). Suppose also that before the search, each helper h draws, for each level d and each owner o, a uniformly random permutation $\pi_{h,d,o}$ of the positions $0, \ldots, n-1$, independently of all other draws, and takes the positions of $B_{d,o}$ that it helps with, those after the owner's published position up to the size it read, in the order of $\pi_{h,d,o}$. Then for every constant c there is a constant a such that, with probability at least $1 - n^{-c}$ over these draws, the following holds for every schedule, even one chosen with knowledge of the permutations: in every level, the helpers of each bucket of R positions together expand vertices at no more than $a(T + R) \log n$ of its positions, counting a position once for each helper that expands its vertex. The theorem is not proved for the affine orders and the unfenced stores of our implementation.*

Proof. A bucket never holds a vertex twice, since its owner appends each vertex at most once (Lemma 5.12), so $R \leq n$, and the positions of a bucket hold distinct vertices. Fix a level, a bucket of R positions, the interval of positions that each of the at most $T - 1$ helpers takes, and the order σ in which the vertices at the R positions are first marked, with the positions whose vertices are never marked last, in the order of their numbers; the schedule decides all of these, and since the vertices are distinct, σ has no ties. Restricted to the interval of a helper, its permutation is a uniformly random order of the interval, independent of those of the other helpers. Take one helper, whose interval has r positions. The position that comes j-th from the end of σ among them is a record if and only if the helper takes it before the $j-1$ positions that follow it in σ. This happens with probability $1/j$, independently for $j = 1, \ldots, r$: build the helper's order by inserting the positions from the last in σ to the first, each at a random place among those inserted before it, and the j-th lands in front of all of them with probability $1/j$. The number of records of all helpers is thus a sum of independent Bernoulli variables with mean $\mu \leq T(1 + \ln R)$, which by a Chernoff bound exceeds any $z \geq 2e\mu$ with probability at most 2^{-z}. The schedule chooses among at most $R! \leq n^R$ orders σ, at most $(n+1)^{2T}$ choices of the intervals, and at most $n \cdot T \cdot (n+1)$ choices of R, the bucket and the level. A union bound over all of these choices, with $z = a(T + R) \log n$ for a large enough, shows that with probability at least $1 - n^{-c}$, none of them gives more than z records, and so neither does the one that the schedule makes, even knowing the orders. By Lemma 5.16, a helper expands a vertex only at a record. □

Under the same assumptions, with the owner of a bucket, which expands each of its positions at most once, the threads of a top-down level then expand vertices at $O((T^2 + Q_d) \log n)$ positions, against $|L_d|$ for a sequential search.

The theorem is asymptotic. With a common order, the helpers of a bucket can be made to expand all of its R positions, $(T - 1)R$ in all (Proposition 5.15). In the proof, the union bound puts at most $R \log n$ in the exponent for the orders σ, and $O(T \log n)$ for the rest, so a is a modest constant that depends only on c, and the theorem improves on $(T - 1)R$ only when T is well above $a \log n$, and not at all for buckets of fewer than about $a \log n$ positions. What it shows is that repetition per position grows as $\log n$ rather than as T, and that no schedule can force more.

The theorem bounds the vertices that threads expand, not the neighbors that they examine, since a thread that enters the expansion of a vertex examines all of its neighbors; Section 7.2 applies the same argument to the chunks of our implementation, which bounds the neighbors as well (Corollary 7.3). Uniformly random orders take R random numbers to draw; our helpers draw affine orders, which take two (Section 7), whose records we have not bounded, and Section 8.3 compares what the two kinds of order repeat in practice.

Records under TSO. Lemma 5.16 treats the marking of a vertex as a single event, as it is under sequential consistency. Under TSO, we take a vertex to be marked when the first store to its flag reaches memory, and the lemma can then fail. A thread h can mark the vertex at a position y and move on to its next position x while that store still waits in its store buffer; it finds the flag of x unset and expands x, while another thread marks x, and that mark reaches memory before the mark of y by h. Then y, which h took before x, is marked after x, so h expands x although x is not a record. A fence after each mark, which drains the store buffer, restores the lemma, and with it Theorem 5.17, under TSO. Fences cost time, so our timed builds mark with stores and no fence; for them, the lemma holds only under sequential consistency, and the theorem needs, besides, the independent uniform permutations that our affine orders are not. The counting build of Section 8.3 marks in the same way, so its counts reflect the marks of the program that we time.

Bottom-up levels. A bottom-up level is not bounded by its size. It reads every vertex, $\Theta(n)$ steps, as the sequential algorithm does, and each thread scans each block at most once, so the threads take at most T times the work of a sequential bottom-up step. The analysis of records does not apply to blocks, whose stamps a late thread can set back to an older depth (see above), so for bottom-up levels we claim only this bound. Which levels go bottom-up is decided by the rule of Section 2.1, a heuristic for which the sequential algorithm has no useful worst-case guarantee on its work either; the method stays correct and wait-free whatever the rule chooses (Section 4.1).

6 Reducing Space through Level Reuse

The method of Section 4 keeps a node for every level, and every bucket array, until the search ends, since a delayed thread may still read them, so its memory grows with the number of levels and with all the vertices appended: a search of the road network allocates a node of T entries for each of its 6,262 nonempty levels and for the empty level that ends it. This section describes a variant that keeps the entries of the threads in two nodes, one for the even levels and one for the odd, each reused two levels later, while each level adds only a record of constant size, with its plan and a link to the next level. A thread's bucket in a node keeps its array from level to level, so the memory of the variant depends on the largest levels, plus a constant per level. The difficulty is that a node is reused while threads that have fallen behind may still be working on the level it held before (Figure 4).

6.1 Reusing Nodes

The two nodes are M_0 and M_1, and level d uses $M_{d \bmod 2}$; Algorithms 5 and 6 write N for the node of a thread's level and N' for the other. Every bucket carries a *tag*, the level that its vertices belong to. A thread starts level d by emptying its own bucket in N', which held an earlier level, and tagging it with $d + 1$ (StartLevel); the thread is past level $d - 1$ by then. A thread in level d uses a bucket of N only if it is tagged d (Algorithm 6, line 6). A smaller tag means that the owner has not started level $d - 1$, and so has appended nothing to the bucket for level d. A larger tag means that the owner has started level $d + 1$, which it can do only once level $d + 1$ has been chosen, by a thread that had finished level d.

A thread that found a bucket tagged d can still be overtaken by its owner, which may empty it and refill it for a later level while the thread reads it. The reads still stay within the bucket's array: arrays only grow during a search, none is freed before the search ends, and a thread reads the size of a bucket before its array (Algorithm 6, line 7), so the array it finds has at least as many positions as the size it read, as in Section 3.5. What the positions hold is another matter, since the thread can meet vertices of later levels, or memory not written in this search. It therefore reads a vertex u of another thread's bucket once, and takes it only if u is a vertex number and $dist[u] = d$ (Valid, and Algorithm 6, line 17). Every such vertex is in L_d, so a thread expands only vertices of its own level, whatever it reads. A thread's own bucket needs no test, since only the thread itself reuses it.

The fields that record progress, and their counterparts for frontier codes, change accordingly. A thread resets its field *last* in N before it tags its bucket in N' (StartLevel), and another thread reads it for level d only after reading that tag as $d + 1$ (Progress). The two reads are not a snapshot, since the owner can move on between them, but they need not be one: a value read this way was written in level d, or the owner has started level $d + 2$ by then, and level d is over (Lemma 6.4). The flag *done* becomes the latest level at which the owner's work is known to be finished (MarkDone), and a thread in level d skips an owner whose flag is at least d; a delayed thread may write a smaller level over a larger one, which only makes other threads check finished work again. Stamps already hold levels (Section 4.2), and the certificates of chunks (Section 7.2) need no change: they hold the number of the search, and a vertex is expanded in only one level of a search, so a certificate of this search can only be one for the level of its vertex.

The plans cannot live in the reused nodes, since a thread that has fallen behind must still find the plan of every level that it passes. Each level d therefore gets a *record* R_d of its own, with its plan and a field *next*, the *slot* for the record of level $d + 1$, which starts out

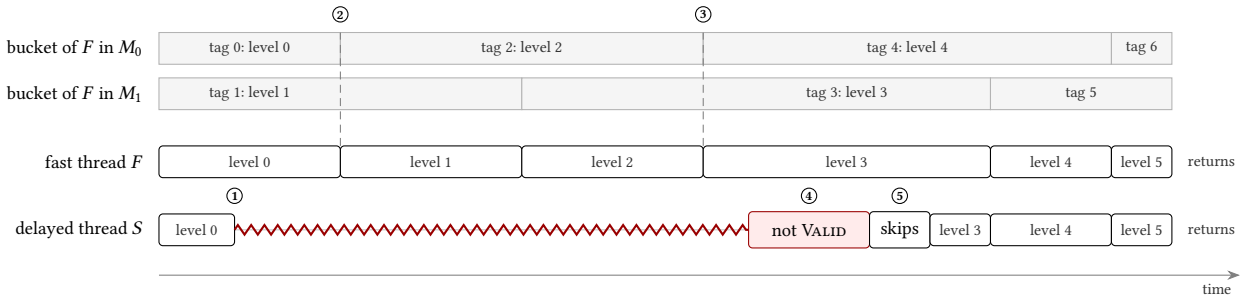


Figure 4: The two nodes of the variant over time, seen from the bucket of a fast thread F in each; level d uses $M_{d \bmod 2}$, and F empties and retags its bucket in a node when it starts the level before the one that the node holds next. (1) A delayed thread S stalls in level 0 while it reads the bucket of F, thread 0, in M_0, tagged 0. (2, 3) F retags that bucket 2, and then 4, and fills it with level 4. (4) S resumes and skips those entries, since Valid$(u, 0)$ requires $dist[u] = 0$, reading within the array, which only grows. (5) S finds level 1 chosen, takes the plans of levels 1 and 2, which are over, from their records, and joins level 3. Both threads return after level 5, which is empty.

empty; the slot of level 0 is a field S_0 of the search. A record takes the place of a proposal, and the CAS that installs it in the slot of its level, that of the claim (ChoosePlan): a thread that finds the slot of level d empty writes a plan, computed as in Section 4.1 from the buckets of N tagged d and the plan of level $d-1$, into a record whose own slot is empty, tries once to install it with a CAS from empty, and adopts whichever record is installed. Nothing changes a plan once its record is installed, so every thread that reads the slot finds the plan whole, and a thread whose CAS fails keeps its record, which no other thread has seen, for the next level that it proposes for. Since each record carries the slot of the next level, no thread links a level while it expands one, and the records form a list, as the nodes of Section 4 do. The search ends at the first level whose plan has no frontier, $n_f = 0$: that plan is top-down (Section 4.3), and every thread returns after it, without proposing for the next.

Before the first call, every bucket of both nodes is empty and tagged -1; its flag *done* and its progress field *last*, with their counterparts for the pass that writes frontier codes (Section 7.1), are -1, as are the stamps of the blocks; the bucket of thread 0 in M_0 is then tagged 0 and holds the source, with edge count $\deg(s)$; and S_0 is empty. The plan before level 0 is that of Algorithm 3.

A thread that finds the slot of level $d+1$ set knows that level d is over, since level $d+1$ has been chosen, by a thread that had finished level d. A thread that finds in the slot of level d a record whose own slot is set therefore takes the plan of level d and moves on to level $d+1$ without starting level d (Algorithm 5, line 6), a record at a time, until it reaches a level whose successor has not been chosen. It thus holds the plan of the level before when it starts a level, as every other thread does. Threads make the same test between the owners of a level, and leave a level that is over (Algorithm 6, line 4); the test only ends loops early.

6.2 Correctness and Wait-Freedom

We follow Section 5. Level d is *chosen* at τ_d, the time of the first CAS on its slot (Algorithm 5, line 22), if there is one; as in Section 5.1, a thread can reach the slot and stop before its CAS. Line numbers in this section refer to Algorithm 5. For a thread in level d, N_d stands for $M_{d \bmod 2}$, and $B_{d,t}$ for the bucket of thread t in it while the bucket is tagged d; the reference prefix $S_{d,t}$ is the first k positions of $B_{d,t}$, where k is its size at τ_d if it is tagged d then, and 0 otherwise. Unlike in Section 5, the parts of a node are reused. The lemmas below show that a part that holds level d is reused only after level $d+1$ has been chosen, so that every read on which the proof of Section 5 relies returns what it would there, while every other read is either checked or made too late to matter.

Lemma 6.1 (Slots). *Only the first CAS on the slot of level d succeeds, and every thread that reads the slot after it finds the record that it installed, with its plan unchanged; in particular, every thread that starts level d (line 7) does so after τ_d, and follows the plan of that record. The slots reached from S_0 form a single list, which every thread walks in order. A thread skips level d (line 6), or leaves it early, only after τ_{d+1}, and for $d > 0$, a thread makes a CAS on the slot of level d only after running level $d-1$ to the end, so τ_d comes after τ_{d-1}.*

Proof. Only CASes from empty write a reachable slot, so the first one succeeds, and the slot never changes after it. The thread whose CAS succeeds wrote the plan of its record before its CAS and writes it no more, so as in Lemma 5.3, every thread that reads the slot finds that plan. The slot of a record was set empty by its thread before the CAS that installs the record, and no other installed record has it, so as in Lemma 5.2, the slots reached from S_0 form a single list, and a thread moves from the slot of level d only to the slot of the record installed there. A thread skips level d, or leaves it early, only on reading the slot of level $d+1$ set, so after τ_{d+1}, and when it reads that slot again, in ChoosePlan, it finds it set and makes no CAS on it. A thread that makes a CAS on the slot of level d has therefore run level $d-1$ to the end, which it started after τ_{d-1}. □

Lemma 6.2 (Reuse). *Thread o tags its bucket in $M_{d \bmod 2}$ with d only in StartLevel$(d-1)$, or before the first call if $d = 0$, and empties and retags it next in some StartLevel(e) with $e \geq d+1$, after τ_{d+1}; in between, it appends to it only vertices that it discovers in level $d-1$. It resets its field last in $M_{d \bmod 2}$ for level d only in StartLevel(d), and next in some StartLevel(e) with $e \geq d+2$, after τ_{d+2}; in between, it writes it only in level d.*

Algorithm 5 The levels of the variant, as thread t runs them. A slot is empty until a record R is installed in it, with the plan of its level, $R.plan$, which carries the frontier size n_f it was computed from, and the slot of the next level, $R.next$, at first empty. S_0 is the slot of level 0; S holds the location of a slot, and $*S$ is the slot. In level d, $N = M_{d \bmod 2}$ and $N' = M_{(d+1) \bmod 2}$. The variables d, N, N' and R of BFS, and *spare*, which keeps a record that was not installed and starts null, are local to the thread; the other procedures, here and in Algorithm 6, use them.

```
1:  procedure BFS(s)
2:      d ← 0; S ← the location of S_0; P ← the plan before level 0
3:      while S ≠ null do
4:          N ← M_{d mod 2}; N' ← M_{(d+1) mod 2}
5:          R ← ChoosePlan(S, P, d); P ← R.plan
6:          if R.next is empty then                ▷ else level d is over
7:              StartLevel(d)
8:              if P.direction is top-down then
9:                  ExpandLevel(d)
10:             else
11:                 ScanLevel(d)
12:             end if
13:         end if
14:         S ← the location of R.next, or null if P.n_f = 0; d ← d+1
15:     end while
16: end procedure

17: procedure ChoosePlan(S, P, d)
18:     if *S is empty then
19:         R' ← spare, or a new record if spare is null
20:         R'.next ← empty
21:         R'.plan ← Rule(P, N, d)   ▷ from the buckets tagged d
22:         installed ← CAS(*S, empty, R')
23:         spare ← null if installed, and R' otherwise
24:     end if
25:     return the record in *S
26: end procedure

27: procedure StartLevel(d)
28:     N.last[t] ← −1
29:     N'.B[t].size ← 0; N'.edges[t] ← 0
30:     N'.B[t].tag ← d + 1
31: end procedure

32: procedure Valid(u, d)
33:     return 0 ≤ u < n and dist[u] = d
34: end procedure

35: procedure Progress(o, d)
36:     if N'.B[o].tag ≠ d + 1 then return −1
37:     return N.last[o]
38: end procedure

39: procedure MarkDone(o, d)
40:     if N.done[o] < d then N.done[o] ← d
41: end procedure
```

Algorithm 6 Expanding level d in the variant, top-down or bottom-up, as thread t runs it, with d, N, N' and the record R of level d as in Algorithm 5. Expand is that of Algorithm 1, with Discover of Algorithm 3, and ScanBlock that of Algorithm 4, testing $dist[u] = d$; both append to $N'.B[t]$ and write the distance $d + 1$. A thread that finds the slot $R.next$ set leaves the level, which is over.

```
1:  procedure ExpandLevel(d)
2:      ρ ← t, then the other threads in a random order
3:      for k ← 0 to T − 1 do
4:          if R.next is not empty then return
5:          o ← ρ[k]
6:          if N.done[o] ≥ d or N.B[o].tag ≠ d then continue
7:          size ← N.B[o].size; A ← N.B[o].array
8:          if size = 0 then continue
9:          if o = t then
10:             for i ← 0 to size − 1 do
11:                 Expand(A[i]); N.last[t] ← i
12:             end for
13:         else
14:             first ← Progress(o, d) +1
15:             for all i from first to size − 1, in a random order do
16:                 u ← A[i]
17:                 if Valid(u, d) then Expand(u)
18:             end for
19:         end if
20:         MarkDone(o, d)
21:     end for
22: end procedure

23: procedure ScanLevel(d)
24:     for k ← 0 to K_t − 1, while N.done[t] < d do
25:         if stamp[t][k] ≠ d then ScanBlock(X_{t,k})
26:         N.last[t] ← k
27:     end for
28:     MarkDone(t, d)
29:     for all threads o ≠ t, in a random order do
30:         if R.next is not empty then return
31:         if N.done[o] ≥ d then continue
32:         first ← Progress(o, d) +1
33:         for all blocks k ≥ first, in a random order do
34:             if N.done[o] ≥ d then break
35:             p ← Progress(o, d)
36:             if k > p and stamp[o][k] ≠ d then
37:                 ScanBlock(X_{o,k}); stamp[o][k] ← d
38:             end if
39:         end for
40:         MarkDone(o, d)
41:     end for
42: end procedure
```

Proof. Only o writes its bucket and these fields. StartLevel(e) writes the field in $M_{e \bmod 2}$ and the bucket in $M_{(e+1) \bmod 2}$, and in level e, a thread appends only to its bucket in $M_{(e+1) \bmod 2}$ and writes progress only in $M_{e \bmod 2}$. Thread o calls StartLevel for increasing

levels, each after the level has been chosen, and levels are chosen in increasing order (Lemma 6.1). □

Lemma 6.3 (Bucket reads). *Let a thread X in level d read the tag of the bucket B of thread o in $M_{d \bmod 2}$ as d, then its size as k, then its array A, and then position $i < k$ of A. The array has a position i. If X read the position before τ_{d+1}, it holds the vertex that o appended there for level d, and if X also read the size after τ_d, then k is at least the size of $S_{d,o}$. If every write so far is sound, a vertex u that X reads in A passes Valid(u, d) only if $u \in L_d$.*

Proof. Thread o publishes an array before any size larger than the array it replaces (Algorithm 2, line 6), and capacities never shrink during a search, so when o wrote k, its array had at least k positions. X read the array after the size, so it found that array or a later one, which has at least as many, and no array is freed during the search. Thread o empties B before tagging it d, and X read the size and the array after the tag. If X read the position before τ_{d+1}, all of these reads therefore came between the two calls of StartLevel in Lemma 6.2, while B held only the vertices that o appended to it for level d, and the argument of Lemma 5.1 applies. Finally, $dist[u] = d$ only if $\delta(u) = d$, by (S1). □

Lemma 6.4 (Progress). *If a thread X in level d gets $i \geq 0$ from Progress(o, d), then o wrote i in level d, after finishing its position or block i as in Lemma 5.5(b) and (c), or X read it after τ_{d+2}.*

Proof. X read the tag of the bucket of o in $M_{(d+1) \bmod 2}$ as $d + 1$ before it read $M_{d \bmod 2}.last[o]$ (Algorithm 5, line 36). Only StartLevel(d) of o writes that tag, after resetting $last[o]$ to -1 (line 28). The stores of o become visible in program order, and the loads of X take effect in program order, so X read the reset or a later value. By Lemma 6.2, the later values that o writes before it next resets the field, after τ_{d+2}, are written in level d. □

Lemma 6.5 (Flags and stamps). *If a thread in level d reads a value of at least d in $M_{d \bmod 2}.done[o]$, the value is d, written by a thread in level d at the end of its pass over o, or it was written after τ_{d+2}. A stamp equal to d was written by a thread in level d after scanning its block.*

Proof. These flags and the stamps start at -1 in every search, and a thread writes into them only the level that it is in (MarkDone and ScanLevel). The levels of $M_{d \bmod 2}$ have the parity of d, and a thread is in a level past d of that parity only after τ_{d+2}. □

Lemmas 6.2, 6.4 and 6.5 hold in the same way for the counterparts of *last* and *done* for frontier codes.

Lemma 6.6 (Levels with vertices). *Suppose that every write is sound, and that (R1) and (R2) hold for every chosen level from the time it is chosen. Then for every level $e \leq D$ that is chosen, the record installed in its slot has a plan with $n_f \geq 1$.*

Proof. The plan comes from the thread C whose CAS chose level e, which computed it from the sizes of the buckets tagged e that it read before its CAS. For $e = 0$, these include the bucket of thread 0, which holds the source. For $e > 0$, C read them after running level $e - 1$ to the end (Lemma 6.1), and as in the proof of Theorem 6.7 below, Lemma 5.6 or 5.7 holds for C, with the buckets tagged e in place of those of N_e: when it finished level $e - 1$, every vertex of L_e, of which there is at least one, had been appended to a bucket tagged e, by C itself or by a thread whose certificates C had read. Under TSO, C sees its own appends, and the appends of the other threads had reached memory before the certificates that C read (Section 5.6). No bucket tagged e is emptied before τ_{e+1} (Lemma 6.2), which comes after τ_e, and so after the reads of C. C therefore read a size of at least 1 for one of these buckets. □

Theorem 6.7. *Theorem 5.11 holds for the variant, and every call of BFS(s) returns after $O((D + 1)(Tn + m))$ steps of its own thread, whatever the other threads do.*

Proof. *Soundness and readiness.* We argue as for Lemma 5.8. Suppose that some write is not sound, and let w be the first. We first show by induction on d that (R1) and (R2) hold for every level d chosen before w, from τ_d up to w. For level 0, they hold before any call starts. For the step from d to $d+1$, let C be the thread whose CAS chooses level $d + 1$, before w. It ran level d before τ_{d+1} (Lemma 6.1), so all of its reads in level d came before τ_{d+1}. By Lemmas 6.3 to 6.5, each of them returned what the corresponding read returns in Section 5: the sizes and reference prefixes of the buckets tagged d, and progress values, flags and stamps written in level d. Each of these values was written before τ_{d+1}, after reads of its writer that came before τ_{d+1} too, and every write before w is sound, so by induction on time, each value certifies the work that Lemma 5.5 says. Lemmas 5.6 and 5.7 thus hold for C, with the buckets tagged $d + 1$ in place of those of N_{d+1}, since every thread tags its bucket in $M_{(d+1) \bmod 2}$ before it expands level d, and as in Lemma 5.8, (R1) and (R2) hold for level $d + 1$ at τ_{d+1}, and keep holding up to w. Now w is made by a thread in some level d, which started the level after τ_d (Lemma 6.1), so the reads on which w depends came after τ_d, when (R1) held. In level d, the thread expands only vertices of its own bucket, which are in L_d by (S2), and vertices that pass Valid(u, d), which are in L_d by Lemma 6.3, and its bottom-up test is $dist[u] = d$, which only vertices of L_d pass, by (S1); all of these hold, since every write before w is sound. The proof of Lemma 5.4 therefore applies to the thread that makes w, whatever else it read, and w is sound, a contradiction. Every write is therefore sound, and the induction above, which then needs no bound w, gives (R1) and (R2) for every chosen level, from the time it is chosen.

Late threads. A thread that reads a part of $M_{d \bmod 2}$ after it was reused does so after τ_{d+1}. Its writes are sound, and its certificates for level d, which may claim work that no thread did, can only make other threads in level d skip work, and no thread needs work of level d after τ_{d+1}, which is also the earliest time at which a thread leaves level d early or skips it (Lemma 6.1). Later levels ignore these certificates, by Lemmas 6.4 and 6.5.

The end. By Lemma 6.6, no level $e \leq D$ has a plan with no frontier, while the plan of level $D + 1$ has none, since its buckets stay empty by (S2). So level $D + 1$ is the first whose plan ends the search, and at any time, the records reached from S_0 are those of levels 0 to k for some $k \leq D + 1$; the list reaches the record of level $D + 1$ once some thread finishes level D. No thread skips level $D + 1$, since no thread proposes for level $D + 2$ and the slot of the record of level $D + 1$ stays empty, so every call returns after running level $D + 1$, which it starts after τ_{D+1}; the proof of Theorem 5.11 then applies.

Wait-freedom. A call visits the slots of levels 0 to $D + 1$ once each, in order (Lemma 6.1), and reads at most two slots at each before it skips the level or starts it, so it runs at most $D + 2$ levels. Choosing a plan reads the T buckets of a node, sets up a record, and makes at most one CAS. Within a level, every loop runs over a range fixed when it starts, as in Theorem 5.13, and a test of the next slot only ends one early; a size that a thread reads is one that the owner wrote, and is at most n, since a thread appends each vertex at most once in a search (Lemma 5.12). A thread thus takes at most Tn positions in each level. Unlike in Theorem 5.13, these need not add up to $Tn + 1$ over the search, since a thread that has fallen behind can find a bucket that its owner has reused, and take positions appended for a later level. The rest of the count of Theorem 5.13 carries over, and the sum is $O((D + 1)(Tn + m))$.

Under TSO, the CAS that installs the record of a level takes the place of the CAS on *planner* in Section 5.6, and the orders that Lemmas 6.3 and 6.4 use are between two stores of a thread, or two loads, which TSO keeps. □

6.3 Memory

The variant keeps two nodes of T entries each, and the bucket of a thread in a node keeps its array for every level that the node holds. The array grows by doubling, to at most $\max(c_0, 2A)$ positions, where A is the largest number of vertices that the thread appends to a level of that parity and c_0 the size of a bucket's first array (Section 3.5); its old arrays stay until the search ends, but together hold fewer positions than its current one. Each level adds a record of constant size, which also stays until the search ends, and a thread whose CAS fails keeps its record for its next proposal, so a search allocates at most $D+2+T$ records. Since a thread appends a vertex at most once (Lemma 5.12), the variant needs $O(T)$ entries, $O(D + T)$ records, and $O(\sum_t \max_d |B_{d,t}|) \subseteq O(T \max_d |L_d|)$ positions, against $O(T(D + 2) + T^2)$ entries and $O(Tn)$ positions for the method of Section 4, which has a node of T entries for each of its $D + 2$ levels, and in which each thread can also hold a spare node that lost the race to be linked (Section 3.6). Before a bottom-up level, our implementation also makes a speculative reservation, from its current estimates of the undiscovered vertices and of the degrees of the frontier, which saves copying an array that would otherwise double several times. These estimates can under- or overestimate the number of vertices that the thread appends, so the bucket still grows on demand, and the request is bounded by about $1.25n/T$ positions. Every new array at least doubles the old one, so the old arrays still hold fewer positions than the current one, and with requests bounded this way, whatever the estimates, the arrays of the variant hold $O(\sum_t \max_d |B_{d,t}| + n)$ positions, the added n being of the order of the per-vertex arrays; in the method of Section 4, whose buckets are new at every level, the reservations of a bottom-up level add up to at most about $1.25n$ positions. The measurements of Section 8.7 include them. A record of our implementation takes 256 bytes, with its plan and its slot each on a pair of cache lines of its own, so that the CAS that later installs the next record in the slot takes no line that readers of the plan need; the records of a search of the road network take about 1.6 MB. The memory of the two methods is close on graphs whose largest levels hold a large share of all vertices, and far apart on graphs such as road networks, which have thousands of small levels.

Table 1: The parameters of our implementations, with the sections that describe them.

Parameter	Value	Section
α and β of the direction rule	15, 18	2.1
Vertices per bottom-up block, W	1,024	4.2
Neighbors per chunk, g	256	7.2
Longest list a helper takes in order	3	3.7
Positions of a bucket's first array, c_0	32	3.5
Bytes per arena chunk, at least	4 MiB	7
Frontier codes	1 to 255	7.1
Random generator of a thread	`mt19937`	7

7 Implementation

An implementation keeps the guarantees of our methods if it realizes the loads, stores, CASes and fences of the model of Section 2.4 as the model describes them: each atomic, in the order that the model keeps, and in a bounded number of steps of its thread. This section describes how our implementations, written in C++ and built with GCC for x86-64 (Section 8.1), do so; Table 1 lists their parameters.

Helpers read buckets, progress fields and flags while their owners write them, and a thread that has fallen behind writes distances, codes and stamps that others are reading; with ordinary accesses, these would be data races, whose behavior C++ leaves undefined, and which let a compiler split an access, or load a location again instead of keeping a value that it loaded [10, 11]. Every access to a location that another thread may access at the same time, with one of the two a write, is therefore atomic: a `std::atomic` field (Table 2, last row), or an ordinary object accessed with `std::atomic_ref`, so that the arrays, and the code that sets them up and resets them while no other thread accesses them, stay ordinary. Each such object is a byte, a 32-bit or 64-bit integer, or a pointer, at the natural alignment that `atomic_ref` requires for it on x86-64, and lives at least until every call of the search has returned; the compiler neither splits these accesses nor repeats or invents them. Table 2 gives the order of each kind of shared access: a *release* store, which no earlier access of its thread may follow, publishes work or data, and an *acquire* load that reads its value orders after it whatever the storing thread did before the store, ordinary accesses included; a *relaxed* access orders nothing else.

Buckets and their arrays. An owner whose bucket is full copies its vertices into a larger array with ordinary stores before the release store of its address (Algorithm 2), so the copy happens before every read of the new array by a thread that has loaded the address. The copy reads the old array, and the size and address of the bucket's array, with relaxed loads, since helpers may be reading them at the same time, and C++ asks that an object referenced by an `atomic_ref` be accessed only through one while that reference exists. The other ordinary accesses are to memory that only one thread uses during a search, such as the capacity of a bucket; to memory that no thread writes during a search, such as the graph;

Table 2: The order of each kind of shared access in our implementations.

Role	Accesses	Order
Publish work or data	a flag, the certificate of a chunk, a stamp or a progress field, after the work that it covers (Section 3.4); the code and then the distance of a vertex, after the vertex is appended (Section 7.1); in Append, the new size, after the vertex, and the address of a new array, after the copy (Algorithm 2); in the variant, the tag of a bucket, after the progress of its thread is reset and the bucket emptied (Algorithm 5)	release store
Accept a publication	the loads of these flags, certificates, stamps and progress fields, and of distances, before the work that they may let a thread skip; of the size and the array of another thread's bucket, before its vertices; in the variant, of the tag of a bucket, before the size or the progress that the tag guards (Algorithms 5 and 6)	acquire load
Ordered already, or advisory	the accesses of a thread to its own bucket and fields; loads that an earlier acquire load or CAS of the thread already orders, such as those of the vertices of a bucket after its size, and the frontier tests of a bottom-up level; stores that a later release store publishes, such as the pass's codes; advisory loads, such as those of the sizes and edge counts that choose a direction	relaxed
Publish a plan, node, record or discovery	the fields *planner* and *next*, and the slots of the variant, which are `std::atomic` objects, and, with a CAS only, the distance in Discover (Algorithm 3)	sequentially consistent CAS and load

and to memory that a thread writes before publishing it with a CAS, which others read only after reading what the CAS wrote: a node before it is linked (Lemma 5.2), a proposal before its claim (Lemma 5.3), and, in the variant, a record before it is installed (Lemma 6.1).

Arenas. Each thread has two arenas, one for the arrays of its buckets and one for the nodes that it adds, or, in the variant, the records that it proposes. An arena hands out blocks from chunks of at least 4 MiB, which it keeps from one search to the next, and asks the system allocator for a new chunk only when no later chunk has room for the block. Keeping the arrays apart means that their memory only ever holds integers: allocating a chunk, with `operator new` or `calloc`, creates them implicitly [29], while memory that had held a node would need new integers created explicitly before an array could use it. In the variant, a thread that reads a reused bucket can read positions that no thread has written in the search (Section 6.1), and C++ forbids reading the indeterminate value of memory never written, so its arenas take zeroed memory from `calloc`: every position then holds 0 or a vertex written earlier, which Valid discards unless it is of the reader's level, and the zeroing, like the copy, happens before every read of an array published in that memory. The system allocator may take locks; the method assumes only that allocation takes a bounded number of steps (Section 2.4), which an implementation can meet by reserving, before any call begins, the memory that a search can allocate, bounded since a thread appends each vertex at most once (Lemma 5.12) and a search has at most $n + 1$ levels (Section 6.3). Our arenas grow on demand instead, mostly in the first search of a thread count, and are reused after it.

Instructions and steps. On x86-64, GCC compiles every relaxed, release or acquire load or store to a single `mov` or `movzx` instruction, which TSO keeps in program order with the other accesses of its thread, except a store and a later load, an order that neither release nor acquire asks for; these orders thus constrain only the compiler, and every thread makes its loads and stores in the order that the algorithms require. Every CAS is a sequentially consistent `compare_exchange_strong`, which GCC compiles to a `lock cmpxchg`, and the fence of Algorithm 1 a sequentially consistent `atomic_thread_fence`, a `lock or` of zero into the top of the stack; a locked instruction takes effect only once the earlier stores of its thread are visible [28], and GCC moves no memory access across either, so both behave as Section 2.4 assumes. Every atomic type that we use, of one, four or eight bytes, is always lock-free on x86-64 (`is_always_lock_free`), and each of our atomic operations is a single instruction, so each takes a bounded number of steps, as the proof of wait-freedom counts it. On ARM and POWER, GCC emits instructions of their own for release and acquire, but C++ promises less with these orders than TSO, since a release store need not become visible to all other threads at the same time, so our proofs do not cover such processors as they stand.

Random orders. Every random order that our implementations draw, of the entries of a node, the positions of a bucket, the neighbors of a vertex, the blocks of a thread and the chunks of a vertex (Section 7.2), is an *affine permutation* of the k items it orders, which visits item $(a + ib) \bmod k$ in step i, for a random a and a random stride b coprime to k; it takes two random numbers and no memory, while a uniformly random order of k items takes k random numbers and memory for all of them. An order of $k \leq 1$ items is the identity, and a thread skips an empty range of positions without drawing one. A thread draws its random numbers from an `mt19937` of its own, which refills its state once every 624 draws, and brings each into its range with a remainder rather than by rejection sampling, at the cost of a slight bias, so that a draw takes a bounded number of steps; the search for a stride coprime to the number of items takes at most $2\sqrt{k}$ tries (Lemma A.1). Threads repeat nearly as little work with affine orders as with uniform ones (Section 8.3), although Theorem 5.17 assumes uniform ones. Over two sockets, a thread visits the entries of threads on its own NUMA node first, in one random order, and then those on the other node, in another.

The reset and the result. Our implementation resets the arrays of a search in parallel, and no thread starts to search before every share is reset, since a thread still resetting could overwrite a distance that another has already set. This is the one step of a search at which our threads wait for each other; it lies outside the method, which starts from arrays that hold their initial values (Section 2.5), and Section 8.2 measures it apart from the search. Calls that are still running when the first returns may write the distances again, with the values already there, so a consumer that reads them before every call has returned must read them with atomic loads, or first copy them with atomic loads into memory of its own.

7.1 Frontier Codes

The test of a bottom-up level, $dist[u] = d$ (Section 4.2), is the most frequent memory access of the bottom-up step, and *dist* has four

bytes per vertex. Our implementations test a denser array: one byte per vertex, $code[v]$, cleared with the other arrays before every search, and a *code* c_d for each level d that a bottom-up level tests, so that the test becomes $code[u] = c_d$. Codes are handed out by the plans. Each plan carries c_d, or 0 if level d tests distances, and ℓ_d, the largest code handed out in the search so far; the thread that proposes the plan computes both from the plan of the level before, and the plan before level 0 has both 0. A top-down level gets $c_d = 0$ and keeps $\ell_d = \ell_{d-1}$. A bottom-up level takes $k = c_{d-1} + 1$ if level $d-1$ was bottom-up with $c_{d-1} > 0$, and $k = \ell_{d-1} + 1$ otherwise. Codes fit in a byte: if $k \le 255$, then $c_d = k$ and $\ell_d = \min(k+1, 255)$, and otherwise $c_d = 0$ and $\ell_d = \ell_{d-1}$. The discoveries of a bottom-up level d write r_d after appending a vertex and before writing its distance, where $r_d = c_d + 1$ if $0 < c_d < 255$, the code that the plan reserved, and $r_d = 0$ otherwise. A level with $c_d = 0$ tests $dist[u] = d$; a test of code 0 would be wrong, since an undiscovered vertex has code 0. Once a bottom-up level gets the code 0, $\ell = 255$, and every later level tests distances too. We say that c_d, if positive, belongs to level d, and r_d, if positive, to level $d+1$, which tests it if it is bottom-up, since then $c_{d+1} = r_d$, and leaves it unused otherwise. Each positive code belongs to only one level: ℓ never decreases, the codes that the plan of level d hands out, c_d and r_d, are at most ℓ_d, and a code $k = \ell_{d-1} + 1$ is larger than every code handed out before it. A bottom-up level with $c_d > 0$ that follows a top-down one, whose appends write no codes, starts with a pass that writes c_d for the vertices in the buckets of N_d. The pass is shared as an expansion is: a thread writes the codes of its own bucket in order, publishing how far it has got in a field *codesLast* of its entry, then writes the rest of every other bucket whose flag *codesDone* is not set, and sets that flag once the bucket's codes are all written. In the variant, a helper writes codes only for vertices that pass Valid. A thread starts testing only after the pass. Top-down levels write no codes, since threads appending nearby vertices would contend for the cache lines of *code*.

With codes, a write can also be unsound by writing a wrong code, so we add a third safety property to those of Section 5.3:

(S3) every positive value written to $code[v]$ belongs to level $\delta(v)$.

Lemma 7.1 (Frontier codes). *Let level d be bottom-up, with $c_d > 0$. (a) If every write so far is sound and a thread reads $code[u] = c_d$, then $\delta(u) = d$. (b) If every write so far is sound, and (R1) and (R2) hold for N_{d-1} and N_d from the times their plans are claimed, then when a thread in level d tests a vertex $u \in L_d$, it finds $code[u] = c_d$.*

Proof. (a) The array is cleared before the search, so the thread read a value that a write of the search stored. By (S3), that value belongs to level $\delta(u)$, and c_d belongs to level d and to no other, so $\delta(u) = d$. (b) Every write to $code[u]$ writes c_d. By (S3), a positive value written there belongs to level d, so it is c_d or r_{d-1}, and r_{d-1}, if positive, is $c_{d-1} + 1 = c_d$. A write of 0 would be a discovery of a bottom-up level $d-1$ with $c_{d-1} = 0$ or $c_{d-1} = 255$, after which $c_d = 0$. If level $d-1$ was bottom-up, $c_{d-1} > 0$, since $c_d > 0$; every vertex of L_d had been discovered in level $d-1$ by the time the first thread finished N_{d-1} (Lemma 5.7), which is before τ_d, and each discovery wrote $r_{d-1} = c_d$ before the distance. If level $d-1$ was top-down, the thread made the pass before testing, and the pass gave every vertex of L_d its code: *codesLast* and *codesDone* are written after the codes they certify, as *last* and *done* are after the expansions they certify (Lemma 5.5(b)), and every vertex of L_d is in a reference prefix by (R2), so the argument of Lemma 5.6 applies with codes in place of expansions. Under TSO, every code is stored before the distance or the field that certifies it, and the argument of Section 5.6 applies. □

The proof of Lemma 5.8 carries over to the method with codes, with (S3) among the properties of a sound write. Let w be the first write that is not sound. In the induction on d, Lemma 5.7 holds for a level that tests codes, since the test $code[u] = c_d$ holds for every vertex u of L_d by part (b) of the lemma, in place of (R1); (R1) and (R2) thus hold for every level whose plan is claimed before w, from then up to w. Then w cannot be a discovery in a bottom-up level e that tests codes, nor its append: the neighbor u that the thread found with $code[u] = c_e$ has $\delta(u) = e$ by part (a), so the proof of Lemma 5.4 applies. Nor can w be the code r_e that such a discovery writes, since the discovery is sound, so $\delta(v) = e + 1$, and r_e belongs to level $e + 1$. Nor can w be a code that the pass of a level e writes, since w writes c_e, which belongs to level e, for a vertex of a bucket of N_e, which is in L_e by (S2), or, in the variant, passes Valid and is in L_e by Lemma 6.3. All other writes are those of Section 5. Every write is therefore sound, and Theorems 5.11 and 5.13 hold for the method with codes; the pass adds a constant number of steps per position of a bucket to the count of Theorem 5.13.

7.2 Chunk Certificates

In the method, a thread that enters the expansion of a vertex examines all of its neighbors (Section 3.7), so a schedule can make every thread examine again every neighbor of a vertex of high degree (Proposition 5.15). Our implementations therefore certify the neighbors of a vertex of more than g neighbors in *chunks* of g consecutive neighbors, the last of which may be shorter, as part (b) of the proposition describes; we use $g = 256$. Chunk c of a vertex u has a *certificate* $cert[u][c]$, which a thread sets to v, the number of the current search, once it has examined every neighbor in the chunk, and other threads skip a chunk whose certificate holds v. A thread that expands u goes through its chunks, scanning each one that is not certified and certifying it afterwards, and sets $expanded[u]$ once it has been through every chunk. A thread that stops in the middle of a chunk thus leaves uncertified only the at most g neighbors of that chunk that it has examined; the chunks that it has not started stay whole for the others, which do not scan again the chunks certified before it. An owner takes the K chunks of a vertex in order from chunk $\lfloor tK/T \rfloor$, so that owners that hold copies of the same vertex start apart, and a helper takes them in a random order of its own, and the neighbors of each chunk in another. A vertex of at most g neighbors is expanded as in Algorithm 1, since $expanded[u]$ certifies the same work as a single chunk would.

The certificates are one array of 32-bit words, with the certificates of a vertex u at $\lfloor offsets[u]/g \rfloor + u$ onward, $n + m/g + 1$ words in all, about as much memory as the distances take; a vertex of at most g neighbors leaves its word unused. Offsets into the adjacency array, like edge counts, are 64-bit integers, since that of Twitter has about 2.9 billion entries; vertices are 32-bit integers. The words of two vertices never overlap: with $o = offsets[u]$ and $q = \deg(u)$, the

words of $u+1$ start $\lfloor (o+q)/g \rfloor - \lfloor o/g \rfloor + 1 \geq \lfloor q/g \rfloor + 1 \geq \lceil q/g \rceil$ words after those of u, and u has $\lceil q/g \rceil$ chunks. A search writes its number into them, and the numbers grow from search to search, so a certificate left by an earlier search holds a smaller number, and the array is cleared only before the first search of each thread count, whose numbers start again at 1. The numbers are positive 32-bit integers, since our implementation uses negative ones to stop its threads, so this holds for $2^{31}-1$ searches; a program that runs more would have to clear the array again before the numbers wrap around, once every call of the last search has returned.

Lemma 7.2 (Chunk certificates). *When a thread certifies a chunk of a vertex u in the search, every neighbor v of u in the chunk with $\delta(v) = \delta(u)+1$ has been appended to a bucket of $N_{\delta(v)}$ and has $dist[v] = \delta(v)$, and with chunks, Lemma 5.5(a) still holds.*

Proof. A thread sets $cert[u][c]$ only after examining every neighbor of u in chunk c, and for each such neighbor v, as in the proof of Lemma 5.5(a), it either appended v before writing its distance, or found the distance set by a thread that had appended v first. Certificates written in earlier searches hold smaller numbers than ν, so a certificate equal to ν was written in this search, by a thread that had scanned the chunk first. A thread sets $expanded[u]$ only after going through every chunk of u, each of which it scanned or found certified with ν, so by then every neighbor v of u with $\delta(v) = \delta(u)+1$ has been appended and has its distance, which is Lemma 5.5(a). □

With chunks, a call of Expand expands u completely if, for every chunk of u, it examines every neighbor in the chunk or reads its certificate equal to ν, and then sets $expanded[u]$. The proofs of Section 5 use complete expansions only through Lemma 5.5(a) and the fact that a complete expansion sets $expanded[u]$, so by the lemma, Theorems 5.11 and 5.13 hold for the implementation with chunks. A call reads each certificate at most once, which adds at most $n + m/g$ steps to the bound of Theorem 5.13, and a top-down level d takes $O(T(Q_d + \mathrm{vol}(L_d) + C_d) + T^2)$ steps in Theorem 5.14, where $C_d \leq |L_d| + \mathrm{vol}(L_d)/g$ counts the chunks of vertices of L_d.

The argument of Theorem 5.17 applies to chunks taken in uniformly random orders as well, if there are at most n threads: the threads that expand a vertex scan $O((k+K)\log(n+m))$ of its K chunks, where k is the number of times threads enter its expansion, instead of up to kK. Combined with the theorem, this bounds the neighbors that a level examines.

Corollary 7.3 (Neighbors examined). *Suppose that $n \geq 2$ and $T \leq n$; that the assumption of Theorem 5.17 on memory holds for the certificates of chunks as well as for the flags expanded, so that memory is sequentially consistent or a thread executes a fence after every certificate; that helpers take positions as the theorem assumes; and that before the search, each helper draws, for each vertex, an order of the vertex's chunks uniformly at random, independently of all other draws, and takes the chunks of every vertex that it expands in that order. Let $\lambda = \log(n+m+2)$. Then for every constant c, with probability at least $1-n^{-c}$ and for every schedule, the helpers of a top-down level d examine $O\big((\mathrm{vol}(L_d) + gQ_d + gT^2)\lambda^2\big)$ neighbors; if each owner also draws its orders of chunks in this way, then so do all threads of the level together.*

Proof. Let k_u be the number of times threads enter the expansion of $u \in L_d$, counting only helpers for the first claim. By Theorem 5.17, and since an owner expands each of its positions at most once, $\sum_u k_u = O((T^2+Q_d)\log n)$. The proof of Theorem 5.17 applies to the $K_u = \lceil \deg(u)/g \rceil$ chunks of u and the expansions of u, with Lemma 5.16 for certificates, which needs the assumption on memory, and with the union bound taken also over which threads enter them. There are at most $T^{k_u} \leq n^{k_u}$ such choices, since $T \leq n$, and at most $K_u! \leq m^{K_u}$ orders in which the chunks are certified, since $K_u \leq \deg(u) \leq m$; the degree has no bound in terms of n alone, since our graphs keep repeated edges (Section 8.1). So the union bound adds $O((k_u+K_u)\lambda)$ to the exponent, and with $z = a(k_u+K_u)\lambda$ for a large enough, it bounds the chunks of u that these expansions scan by $O((k_u+K_u)\lambda)$, whatever order the other threads take them in. A chunk holds at most g neighbors, so these expansions examine $O(g\lambda \sum_u (k_u+K_u)) = O\big(g\lambda\,((T^2+Q_d)\log n + |L_d| + \mathrm{vol}(L_d)/g)\big)$ neighbors, which is the bound, since $|L_d| \leq Q_d$ and $\log n \leq \lambda$. □

Our owners take the chunks of a vertex in order, from a start that depends on the thread, which keeps their scans sequential. For a hypothetical implementation whose helpers draw independent uniform orders, and whose marks are ordered as Theorem 5.17 requires, the corollary would still bound the neighbors that helpers examine, but not those that owners examine, since a vertex can be in the buckets of several owners, and with fixed orders the argument does not apply to them. Our helpers draw affine orders, and our marks are stores without a fence, so the corollary does not cover our implementation, whose repeated work Section 8.3 measures instead, and Section 8.6 what chunks save.

8 Experiments

We compare our method with three of the fastest BFS implementations for shared memory, GAPBS, GBBS and PASGAL, on five standard graphs and on a sixth that makes vertices of high degree costly, without stalls (Section 8.2) and with threads stalled on purpose (Section 8.8); we also count the work that our method repeats (Section 8.3), and split the gap between it and GAPBS, which runs the same algorithm, into a traversal term and bookkeeping (Section 8.9). We call the method of Section 4 WF-DO, and the variant of Section 6 WF-DO-mem.

8.1 Setup

Machine and programs. We ran every experiment on an otherwise idle machine with two Intel Xeon Platinum 8592V processors, each with 64 cores, one hardware thread per core, and 256GB of memory, under Linux 4.18. We compare against the direction-optimizing BFS of GAPBS [6] (commit 2972aeb), which uses OpenMP, and the BFS implementations of GBBS [16] (commit bd48872) and PASGAL [17] (commit f7817ba), which use the work-stealing scheduler of ParlayLib [8], at commit 352f536 for GBBS and in the copy at c655b8f that PASGAL includes. The sequential baseline is the direction-optimizing BFS of GAPBS without threads. Each program is a single file that holds the BFS of its library, with the parts of the library's support code that the BFS uses. We changed only how the programs read graphs, place their threads, and time searches, and what they output and return: GAPBS and GBBS store each vertex's depth where they stored its parent, so

Table 3: The graphs, with their numbers of vertices and of input edges, each of which the searches store at both ends, and the number of levels of a search from vertex 0 that hold vertices, $D + 1$.

Graph	Vertices	Edges	Levels
LiveJournal [39]	4,847,571	68,475,391	12
Twitter [37]	41,652,230	1,468,365,182	17
Road-USA [49]	23,947,347	28,854,312	6,262
uk-2002 [12, 15]	18,520,486	298,113,762	29
Kronecker [6, 43]	33,554,432	523,609,147	8
Hubs (Section 8.6)	25,200,513	50,400,008	128

that every program computes distances, as PASGAL does already. All programs, ours included, read graphs with the same loader, and search the same CSR, in which every edge of the input is stored at both of its ends, duplicates included.

Builds and schedulers. We built every program with GCC 15.2.1, C++23, `-O3 -march=native -DNDEBUG`, frame pointers, and loops and jump targets aligned to 32 bytes. GAPBS uses the system libgomp, whose default is `OMP_WAIT_POLICY=PASSIVE` with a spin count of zero; we set `GOMP_SPINCOUNT=300000`, upstream libgomp's default, so that a worker at a barrier spins for a bounded time and then sleeps. Every program pins each worker to a CPU itself, so `OMP_PROC_BIND` does not affect the placement. GBBS and PASGAL use ParlayLib's scheduler defaults, including elastic parallelism and its 10 ms steal timeout.

Graphs. Table 3 lists the graphs: the social networks LiveJournal and Twitter, the web graph uk-2002, the road network of the USA, and a Kronecker graph with 2^{25} vertices that the generator of GAPBS builds with the parameters of Graph500, drawing its default of 16 edges per vertex and dropping self-loops and repeats; if vertex 0 is outside its largest component, we swap its label with that of a vertex inside. We call these five the *standard graphs*; all but the road network have low diameter. With D the largest distance from vertex 0, a search has $D + 1$ levels that hold vertices, which Table 3 counts; GAPBS takes one step for each, and our methods also go through the empty level $D + 1$. The sixth graph, of hubs, joins a root to the first of 64 layers of 8 hubs each, and puts between consecutive layers 400,000 spokes, each joined to a random hub of each of the two layers, so that after the root, levels of 8 hubs of about 50,000 to 100,000 neighbors alternate with levels of 400,000 spokes of degree two. Section 8.5 adds a path of 10,000 vertices.

Placement and method. Each run uses both NUMA nodes: its threads are split evenly between them, each pinned to an idle CPU of its own, and memory is interleaved over both, even with one thread, which runs on one socket. Our threads visit the entries of other threads on their own NUMA node first (Section 7). We run 1, 2, 4, 8, 16, 24, 32, 40, 48, 56, 64, 80, 96, 112 and 128 threads; at 128, the workers take every core and leave none to the operating system. Where a program's code lands in memory changes its time by up to 5%, so we build every program five times, shifting its code by 0 to 256 bytes in steps of 64. A build loads its graph once and, for each thread count in turn, rests for one to three seconds and runs 20 searches from vertex 0; the mean of the 20 is the build's time, each plotted point is the median of the five builds, and its error bar spans the fastest and slowest builds. Every build's distances were those of the sequential baseline: a build compares the distances of each thread count with those of the first, and our scripts compare those of its last with the baseline's.

Timing. The threads of every program live across searches. For GAPBS and GBBS, a search is a call of their BFS, which allocates and fills its output and returns after its last parallel loop has joined; PASGAL's BFS reuses arrays that it set up once. A search of our methods starts when the main thread begins to set up its first level, includes the reset of the arrays, which every thread waits for, and ends when the first call returns, when the distances are final (Section 2.5). Our threads pin themselves to their CPUs just before the first search, which can therefore wait for the last of them; the first of the 20 searches is included in every program's mean.

8.2 Without Stalls

Table 4 gives each program's best time, and Figure 5 its time at every thread count up to 112; Appendix B lists all the times. Our two methods are the fastest programs on every graph. Over the five standard graphs, the best time of GAPBS is 1.58× that of WF-DO in the geometric mean, that of GBBS 2.77×, and that of PASGAL 2.73×, and at its best, WF-DO is 23 to 74 times faster than the sequential baseline on the graphs of low diameter. The exceptions are mostly at small thread counts: at one thread, where WF-DO takes 1.00 to 1.39× as long as the sequential baseline on the standard graphs, GAPBS is faster than WF-DO on LiveJournal, uk-2002 and Road-USA, by 9, 1 and 17%, and on LiveJournal also at two threads, by 0.5%, and up to 32 threads it is up to 11% faster on Twitter; on the graph of hubs, GAPBS is faster up to 8 threads, by up to 35%, and GBBS up to 16, by up to 15%. From 40 threads on, WF-DO is faster than GAPBS on every graph, and GBBS and PASGAL are slower than WF-DO at every count up to 112 threads on the graphs of low diameter. On Road-USA, with its 6,262 levels, both of our methods are at their fastest at 48 threads, 7.5 and 9.0 times faster than the sequential baseline, and get slower beyond, as the cost of the levels takes over (Section 8.5); PASGAL keeps improving up to 112 threads, at which it is faster than WF-DO but not than WF-DO-mem, and GBBS is slower than the sequential baseline at every count.

The variant with two nodes. WF-DO-mem takes about as long as WF-DO on the graphs of low diameter, within 1.9% through 112 threads, or up to 5.3% longer on LiveJournal; on Road-USA, where WF-DO allocates a node for each of its 6,263 levels and WF-DO-mem reuses two, it takes 11 to 28% less time at every count.

All cores and the reset. Table 4 takes each program's best median over all counts, 128 threads included, at which every core has a worker, so that any other work on the machine, even that of the operating system, takes a core from some worker. The slowest of the five builds is then 15.5% slower than the fastest in the median over programs and graphs, against 4.4% at 112, and in builds that report how long a worker is ready to run but has no core, in the median of three runs per graph, that time grows from 112 to 128 threads, to up to 1.9 ms per search for WF-DO, 23 ms for GAPBS

Table 4: Each program's best median time, in ms per search, over 1 to 128 threads on two sockets, with the thread count in parentheses. The fastest program on each graph is in bold; on uk-2002, our two methods are within 0.1% of each other. The best thread counts, and the ratios in the text, come from the unrounded medians.

Graph	Sequential	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
LiveJournal	96.2	**4.2** (112)	4.5 (112)	5.4 (112)	6.7 (112)	9.2 (128)
Twitter	3038.2	43.6 (128)	**43.1** (128)	55.7 (112)	89.0 (112)	121.3 (128)
Road-USA	851.3	113.7 (48)	**94.4** (48)	273.4 (24)	1273.3 (32)	147.2 (112)
uk-2002	1183.7	**25.4** (112)	**25.4** (96)	37.1 (112)	40.1 (96)	84.3 (112)
Kronecker	985.5	13.3 (128)	**13.1** (128)	23.2 (112)	38.0 (112)	77.0 (128)
Hubs	1463.9	64.5 (128)	**61.8** (128)	332.7 (48)	90.1 (112)	141.5 (112)

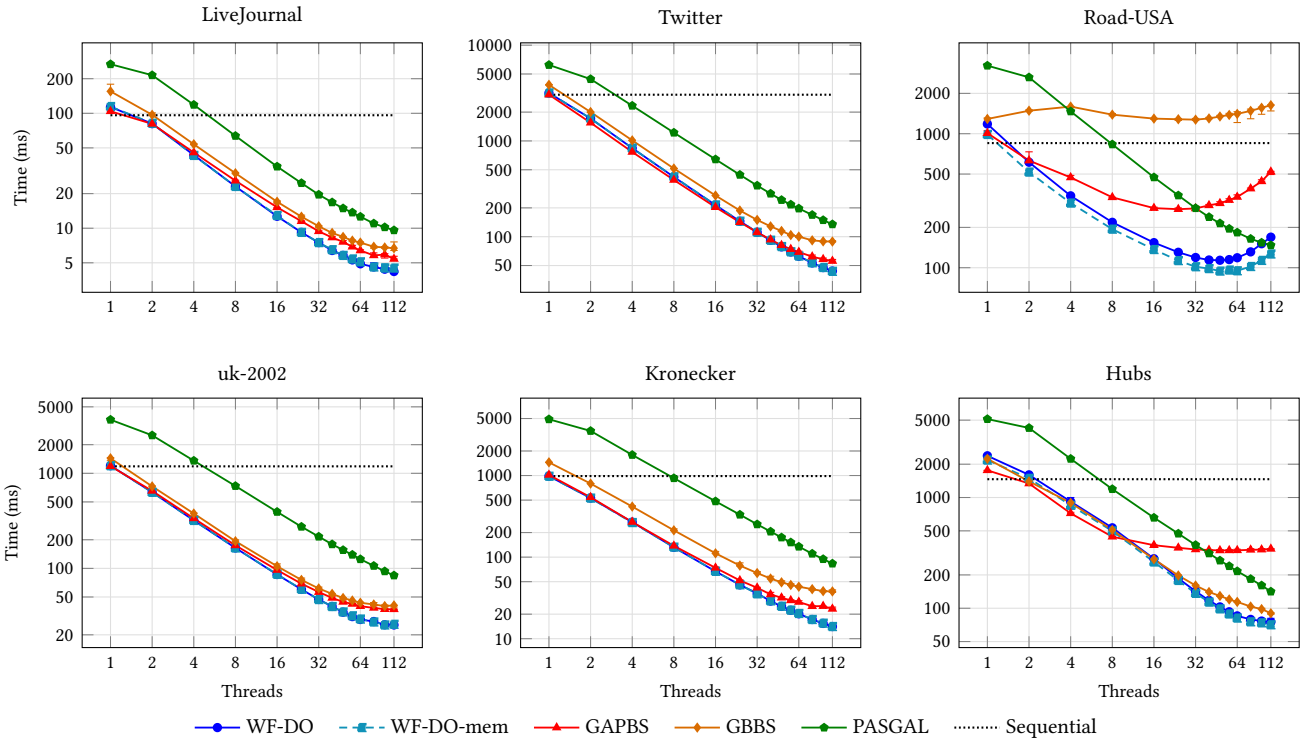


Figure 5: Median time per search against the number of threads, from 1 to 112, with threads and memory spread over both sockets, on the five standard graphs and on the graph of hubs of Section 8.6. Both axes are logarithmic; the dotted line is the sequential baseline, and each error bar spans the fastest and slowest of the five builds. The times at 128 threads, at which every core has a worker, are in Appendix B.

and 41 ms for GBBS, and unevenly: on LiveJournal, the worker that waits the longest waits 2.4 ms per search, against a mean of 0.06 ms for WF-DO. GAPBS, each of whose barriers waits for any worker that has lost its core, is slower at 128 threads than at 112 on every graph, by 1.31 to 1.53 times, or 2.69 on Road-USA; GBBS is too, by at most 1.12 times. Our methods are slower at 128 threads only on LiveJournal and uk-2002, by up to 1.38 times, and WF-DO by 1.5% on Road-USA. Table 5, from five more runs of instrumented builds, splits their searches into the reset of the arrays, which every thread waits for (Section 7), and the search proper, the wait-free method, each the mean over the 20 searches of a run. From 112 to 128 threads, the search proper of WF-DO changes by −16 to +5%, and that of WF-DO-mem by −11 to +17%, since the others take over the work of a thread that loses its core, while the reset grows from 0.4–1.7 ms to 1.8–3.4 ms, since a thread without a core when a search begins holds up all the others; on LiveJournal, it is a third of a search of WF-DO at 128 threads, and most of why our methods take longer there than at 112. The search thus tolerates delayed workers, while the reset, which lies outside the method, waits for all of them. These runs are separate from those of Table 4, and their totals need not match its medians, since the builds differ and the background work of the host varies from run to run.

The other programs also set up before their first step, allocating and filling distances, allocating a queue and clearing bitmaps, or

Table 5: A search of our methods in its two phases, on two sockets, in ms per search: the reset, from when the main thread starts to set up the search to when the first thread starts to search, and the search proper, from then to when the first thread returns. Each column is a median of five runs, taken on its own, so the phases need not add up to the total.

		WF-DO			WF-DO-mem		
Graph	T	Reset	Search	Total	Reset	Search	Total
LiveJournal	112	0.4	4.0	4.4	0.4	4.3	4.7
	128	1.9	3.6	5.6	1.8	5.1	7.1
Twitter	112	1.4	42.7	44.1	1.5	42.2	43.8
	128	2.9	39.5	42.1	2.6	40.3	44.7
Road-USA	112	1.1	162.7	163.8	1.3	115.9	117.2
	128	3.0	169.1	171.9	2.3	120.7	122.8
uk-2002	112	0.7	24.9	25.6	0.7	25.0	25.7
	128	3.1	26.2	29.3	2.9	27.9	30.8
Kronecker	112	1.2	13.5	14.8	1.2	13.4	14.7
	128	2.1	11.7	14.1	3.2	12.7	16.9
Hubs	112	1.6	74.2	75.8	1.7	69.3	70.9
	128	3.4	62.5	65.9	2.7	61.8	64.8

refilling arrays that they keep. In instrumented builds of the three, in the median of three other runs per graph alongside ours, this setup takes 0.3 to 14.7% of a search at 112 threads, our reset 0.5 to 7.6% of ours, and leaving both out changes no comparison between our methods and the other three: the search proper of the fastest of the three takes 1.39 to 1.62 times as long as that of WF-DO on the four graphs of low diameter and 1.25 times on the graph of hubs, against 1.45 to 1.76 times for whole searches on the four, and on Road-USA, that of PASGAL takes 0.95 times as long as that of WF-DO and 1.35 times as long as that of WF-DO-mem.

8.3 Work

A wait-free method can always fall back on doing all of the work in every thread (Section 2.3); the question is how much of it threads actually repeat. A counting build of WF-DO counts, per search, the neighbors that threads examine, the bucket positions and vertices that they read, and the neighbors that they repeat, and builds of GAPBS and the sequential baseline count the same, in runs separate from those of Table 4. The counting build marks vertices and certifies chunks with the stores of our timed builds, and differs from them only in per-thread counters and timers. Since every vertex of a top-down level is expanded completely at least once, it counts as repeated, after each search, the neighbors that threads examined in top-down levels beyond the degrees of the vertices of those levels. Table 6 gives the work of WF-DO relative to that of the sequential baseline, which examines 28.0, 706, 57.7, 802, 36.8 and 136 million neighbors per search on the six graphs, in the order of the table; GAPBS examines exactly the neighbors, and reads exactly the vertices, of the baseline, since it makes the same choices of direction and expands every vertex once.

On the five standard graphs, WF-DO examines at most 2.1% more neighbors than the sequential baseline at 16 threads and at most 23.9% more at 112, or 25.5% in the worst of the three runs, against the T-fold work of the trivial method, although it reads up to 65% more vertices. Where threads repeat a large share of the neighbors of top-down levels, those levels are small: on Kronecker, they repeat

Table 6: The work of a search of WF-DO from vertex 0 on two sockets, relative to that of the sequential baseline: the neighbors examined; the vertices read, in bucket positions, including those of the passes that write frontier codes (Section 7.1), and in bottom-up scans, against the frontier vertices and bottom-up scans of the baseline; and the share of the neighbors examined in top-down levels that are repeated. Medians of three runs of the counting build; in the rows marked s, one of the 112 threads stalls for 10 ms at the start of every level, or for 1 ms on Road-USA.

Graph	Threads	Neighbors	Vertices	Repeated
LiveJournal	16	1.02	1.08	1.7%
	64	1.06	1.40	8.7%
	112	1.11	1.60	15.1%
	112s	1.15	1.65	15.2%
Twitter	16	1.00	1.01	0.7%
	64	1.01	1.02	2.8%
	112	1.02	1.03	4.2%
	112s	1.03	1.04	4.1%
Road-USA	16	1.02	1.04	1.6%
	64	1.07	1.31	6.1%
	112	1.09	1.45	8.5%
	112s	1.10	1.41	8.8%
uk-2002	16	1.01	1.03	0.6%
	64	1.14	1.10	3.3%
	112	1.24	1.27	8.9%
	112s	1.20	1.28	8.7%
Kronecker	16	1.00	1.01	19.2%
	64	1.01	1.02	45.0%
	112	1.02	1.03	55.4%
	112s	1.02	1.04	55.0%
Hubs	16	1.12	1.07	6.2%
	64	1.55	1.11	15.1%
	112	2.04	1.15	20.6%
	112s	2.02	1.16	20.6%

up to 55% of them, but the baseline examines only 0.2 million of its 36.8 million neighbors there, and on LiveJournal nearly all of the repeats lie in the two levels right after the source, which hold 94 and 20,685 vertices, few for 112 threads (Section 5.7). Most of the extra work at 112 threads on LiveJournal and uk-2002 lies in bottom-up levels instead: in blocks that a helper scans again, and in an extra bottom-up level, since the frontier size that the planner counts includes copies of vertices (Section 4.1), mostly appended by two threads scanning the same block, so that with many threads the rule stays bottom-up in most searches for a level that the baseline expands top-down, and a bottom-up level reads every vertex. A build that counts each vertex once, with a CAS on the distance of every bottom-up discovery, runs as many bottom-up levels as the baseline on these two graphs, and at 112 threads cuts the extra vertices read from 60 to 30% on LiveJournal and from 27 to 11% on uk-2002, and the extra neighbors on uk-2002 from 24 to 14%, while on LiveJournal they grow by three points; but its CAS costs more than the work it saves: in the median of five builds, it is 7 to 20% slower than WF-DO on the four graphs of low diameter at 16, 64, 96 and 112 threads, and within 1.1% on Road-USA and

the graph of hubs. On Road-USA, most of the extra work lies in its thousands of top-down levels, each shared by all threads. On the graph of hubs, most of it lies in bottom-up levels, where WF-DO examines 3.8 times as many neighbors as the baseline although only 3.7% of its scans of blocks are repeats: a block is certified only as a whole, and the blocks likeliest to be scanned again, since they take longest, hold hubs of the layers not yet reached, each of which examines all of its roughly 50,000 to 100,000 neighbors without finding one in the frontier. A thread stalled at every level changes the neighbors examined by at most 4.1%, since the others take over its share instead of repeating it.

Orders. Our helpers take the positions of a bucket in affine orders (Section 7), a cheap stand-in for the uniformly random permutations that Theorem 5.17 assumes, rather than an approximation of their distribution: at 112 threads, the share of the neighbors of top-down levels that threads examine again is within 1.5 points of what it is with uniform orders in place of all of ours, on every graph.

8.4 Other Roots

The searches above start from vertex 0. To see how much the root matters, we repeat the runs of the six graphs from 16 other roots, at 16 and 112 threads over both sockets. The roots are drawn uniformly at random, with a fixed seed, among the vertices whose search reaches at least half of the graph; on each graph, all of them reach the same vertices as vertex 0, from 50.8% of Kronecker to all of Twitter, Road-USA and the graph of hubs. Each program is built once, with its code unshifted, and runs 20 searches from each root, with the programs taking turns from root to root; every run gave the distances of a sequential BFS from its root.

Our two methods are faster on average than GAPBS, GBBS and PASGAL in every setting of Table 7 but one, Road-USA at 112 threads, where PASGAL is faster than WF-DO but not than WF-DO-mem, as from vertex 0; elsewhere, the fastest of the three takes 1.05 to 1.79× as long as WF-DO at 16 threads, and 1.30 to 1.91× at 112. Vertex 0 is within the range of the other roots for every program, except on uk-2002, where it is slower than every other root for WF-DO at 16 threads, for GAPBS at 112, and for PASGAL at both counts, and on the graph of hubs, where it is the root of the layers and has the deepest search, and GAPBS takes longer from it than from any other root at both counts, and PASGAL less, as does GBBS at 16 threads.

8.5 The Per-Level Cost of Our Methods

Every thread takes part in every level: it plans the level or adopts its plan, and visits up to T entries of its node, and in WF-DO, one thread also links a node of T entries for the next level. A level therefore costs every thread $O(T)$ steps, however few vertices it holds, and the $D + 2$ levels of a search cost it $O(T(D + 2))$ steps besides its share of the work (Theorem 5.14). On graphs of low diameter, this cost is small next to the work of their few levels; on a path, it is nearly all that a search does. This is where our methods are weakest.

Table 8 gives the cost of a level on a path of 10,000 vertices. With two threads, on different sockets, a level of WF-DO takes 1.64 μs, against 0.07 μs with one, since every level moves the cache lines of its node, its plan and its one bucket between the sockets, and every further thread adds an entry that every thread visits; WF-DO-mem, which reuses its two nodes, grows more slowly. GAPBS grows faster still, since every one of its steps ends at a barrier of all of its threads. GBBS stays flat, since it runs a frontier of one vertex as a single block on the calling thread, and PASGAL is nearly as fast as the sequential baseline, since a task follows the vertices it discovers through a local queue of up to 128 entries, setting distances with an atomic minimum that a later, shorter path could still lower, so that one round covers up to 128 levels [17, 52]. The road network, with 6,262 levels of about 3,800 vertices, lies in between, and our methods are at their fastest there at 48 threads (Section 8.2).

On the road network, WF-DO-mem, whose best time is 17% below that of WF-DO, does the same work with less new memory: WF-DO puts every level in a new node with new buckets, 42 to 51 KB per level on this graph (Section 8.7), so its appends write lines that the cores do not hold, while the two nodes of WF-DO-mem keep their buckets in the caches. Hardware counters are consistent with this, although they cover each search until every call has returned, and so include calls that return after the first and the spinning of threads that have returned: over two runs, WF-DO takes 10 to 34% more cycles than WF-DO-mem at 1, 16, 40 and 112 threads, for 4 to 5% more instructions, and its requests miss the last-level cache 18 to 30% more often, while its stores miss the private caches far more often only at one thread.

These costs follow from our design, not from wait-freedom itself. Every thread goes through every level and visits all T entries of its node, so that it could finish the search alone, although a wait-free thread may skip a level that it finds certified complete, as our late threads do (Sections 4.1 and 6.1); and our methods never change a published distance to a different value, which rules out the corrections that let PASGAL run ahead of the levels. On graphs whose levels hold few vertices, blocking implementations that leave a small level to one thread, or run past it, are faster than ours.

8.6 A Graph of Hubs

Proposition 5.15 says that a vertex that is certified only as a whole can cost T times its degree. The graph of hubs (Section 8.1), every other frontier of which consists of 8 such vertices, shows what certifying chunks instead buys. With whole lists, threads repeat 94% of the neighbors that they examine in top-down levels at 112 threads, so that they examine 17 times as many as the sequential baseline does there, and a search takes 1.4 times as long as with chunks at 16 threads and 3.4 times as long at 112, in three runs of one build of each. With chunks, they repeat 21%, and our two methods keep improving through 128 threads (Figure 5 and Appendix B), to 19.5 and 20.6 times faster than the sequential baseline at 112, while GAPBS, which expands every vertex of a top-down level in a single thread, so that at most 8 of its threads work in a top-down level of hubs, improves little past 16 threads.

8.7 Memory

Table 9 gives the memory of our two methods at 1, 16 and 112 threads. We count the bytes that the arenas of both kinds hand out in a search, which hold all that a search of WF-DO allocates, its nodes and its bucket arrays, and the bucket arrays and level records of WF-DO-mem, whose two nodes take under 0.1 MB more;

Table 7: Searches from 16 random roots, on two sockets: each program's mean time over the roots, in ms, with its fastest and slowest root in parentheses. Ratios in the text come from the unrounded means.

Graph	Threads	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
LiveJournal	16	13.6 (12.2–15.8)	13.7 (10.9–18.4)	15.8 (13.1–20.4)	20.0 (17.4–30.7)	66.3 (32.5–126.9)
	112	5.3 (4.4–7.7)	5.0 (4.3–6.5)	8.7 (5.1–41.8)	8.1 (6.7–10.3)	16.1 (8.8–26.8)
Twitter	16	182.4 (136.2–350.0)	181.8 (136.1–347.1)	223.3 (158.9–408.8)	226.4 (170.1–437.2)	493.4 (319.7–787.0)
	112	41.4 (31.6–80.4)	41.6 (32.2–79.5)	113.0 (52.2–335.7)	79.3 (62.6–131.7)	97.8 (63.1–159.7)
Road-USA	16	159.4 (136.6–181.8)	141.0 (119.7–150.5)	285.8 (250.7–307.6)	1296.6 (1062.8–1435.3)	475.0 (420.8–556.4)
	112	180.1 (143.0–204.8)	134.1 (102.6–158.0)	552.7 (425.1–764.5)	1648.8 (1323.5–1789.7)	155.7 (128.0–178.9)
uk-2002	16	67.7 (56.5–87.2)	67.1 (56.4–86.4)	79.3 (68.4–93.9)	90.1 (74.3–119.9)	193.9 (133.2–393.8)
	112	24.1 (20.5–28.6)	24.3 (19.7–27.8)	35.8 (30.8–40.4)	45.2 (38.4–57.1)	43.2 (30.9–84.3)
Kronecker	16	72.4 (60.4–82.3)	72.7 (63.5–83.6)	80.4 (73.2–93.2)	114.9 (93.6–129.1)	358.4 (162.7–646.7)
	112	15.8 (13.9–19.6)	16.2 (13.6–20.7)	27.8 (22.9–40.2)	41.1 (34.4–48.9)	65.9 (29.8–116.7)
Hubs	16	279.6 (276.0–285.6)	263.5 (257.3–268.8)	306.2 (249.9–368.0)	293.4 (286.4–301.2)	700.5 (684.2–726.4)
	112	77.9 (74.4–81.2)	73.3 (69.1–78.5)	277.0 (203.3–346.8)	101.7 (96.1–105.7)	154.2 (147.5–160.2)

Table 8: The time of a level on a path of 10,000 vertices, whose search has 10,000 levels of one vertex each: the time per search divided by 10,000, in μs, in the median of five builds of each program. The sequential baseline takes 0.04 μs per level.

Threads	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
1	0.07	0.04	0.26	0.50	0.04
2	1.64	1.33	1.92	0.51	0.05
16	3.10	2.32	5.15	0.51	0.04
64	5.01	3.42	13.71	0.52	0.05
112	12.35	5.69	24.60	0.51	0.05

nothing is freed during a search, so a search holds the most at its end. We also measure how much the resident size of the process, file-backed and shared pages included, grows over the 20 searches of a thread count, from its size once the graph and the per-vertex arrays are loaded, which is the same for both methods. An arena's pages become resident when first written, unless the process already held them, and the arenas keep their chunks from one search to the next, so this growth can be smaller or larger than what one search takes.

The two methods differ the most on graphs with many small levels. On the road network, a search of WF-DO takes 265 to 317 MB, 63 to 75% of what the graph takes, and one of WF-DO-mem 1.9 to 2.2 MB, 142 to 149 times less, since each of its buckets holds one level at a time, a few thousand vertices there; most of it is the record of each of its 6,263 levels (Section 6.3). The graph of hubs, with 128 levels, is similar, with 16 to 24 times less. On LiveJournal, Twitter and Kronecker, whose largest levels hold a large share of their vertices, the two take about the same, as Section 6.3 expects, and uk-2002, with 29 levels, lies in between. The resident size can grow by more than a search takes, by about 1.5 times as much for WF-DO at 112 threads on the road network, uk-2002 and the graph of hubs.

8.8 Stalled Threads

A program that waits for all of its threads at the end of every level or round is held up by each delayed thread, while in our methods

Table 9: The memory of WF-DO and WF-DO-mem, from vertex 0 on two sockets, in MB of 10^6 bytes: the most that a search takes from the arenas, and how much the resident size of the process grows over 20 searches. The graphs themselves take 0.59, 12.08, 0.42, 2.53, 4.46 and 0.60 GB of 10^9 bytes, and with the per-vertex arrays and the rest of the process, the resident size before the searches is 0.70, 12.72, 0.76, 2.84, 4.95 and 0.96 GB. Ratios in the text come from the unrounded values.

		WF-DO		WF-DO-mem	
Graph	Threads	Search	Resident	Search	Resident
LiveJournal	1	39.8	24.0	38.1	24.3
	16	40.0	41.2	38.1	47.3
	112	53.9	62.2	43.9	59.3
Twitter	1	306.5	167.0	299.3	164.9
	16	306.2	356.5	299.2	231.3
	112	323.6	362.8	301.0	377.0
Road-USA	1	265.0	256.3	1.9	0.1
	16	277.6	287.5	1.9	2.2
	112	317.1	466.8	2.2	18.7
uk-2002	1	250.0	76.3	172.6	50.1
	16	248.8	325.4	172.2	184.6
	112	250.2	377.6	171.2	97.0
Kronecker	1	300.3	72.8	270.4	70.0
	16	300.3	132.7	270.3	169.2
	112	301.3	307.0	269.8	93.1
Hubs	1	286.1	286.1	12.4	7.0
	16	298.6	321.5	12.5	11.0
	112	317.6	479.7	19.5	36.8

the others take over the work of a delayed thread and go on. To measure the difference, with 112 workers over both sockets, which leaves 16 cores for the host and for the thread that sends signals, we stall on purpose: one worker at the start of every level or round; every worker at random points, varying the length of a stall and the share of time stalled; and our workers where a stall leaves the most work to the others. The text gives numbers for Twitter, the largest graph, and Road-USA, each of whose 6,262 levels ends in a

synchronization in GAPBS and GBBS, while a round of PASGAL can span many (Section 8.5), and summarizes the other four; on Road-USA, the stalls last at most 1 ms, since with stalls of 1 ms at every level a search of GAPBS already takes 6.4 s. The programs are built with the code that stalls them, unshifted, and each setting runs five times, a run being 20 searches, with the programs taking turns; each point is the median of the five runs, and every run gave correct distances. The experiments run apart from each other and from those of Section 8.2, so the times without stalls differ between them: on Twitter, GAPBS takes 58.9 ms without stalls in the first experiment, 58.1 ms in the second, and 55.7 ms in the benchmark.

One thread per level. In the first experiment, one worker busy-waits for a set time when it starts its work in a level or round, so that it holds that work while it waits, as a thread preempted in the middle of its share would: our methods stall at the start of every level (WF-DO-mem only if it is not over), GAPBS at the start of every top-down and bottom-up step, and GBBS and PASGAL at the first vertex or block of edges that a worker takes in a round. The stalled worker is the last of the 112, so that the threads that run the sequential parts of GAPBS, GBBS and PASGAL never stall. Figure 6 varies the stall from 10 μs to 10 ms, and to 1 ms on Road-USA. Our methods stay flat on every graph. GAPBS grows by about one stall per level, since every one of its steps waits for the stalled thread: on Twitter, to 4.0 times its time without stalls at 10 ms, and on Road-USA to 6.4 s with stalls of 1 ms, about 0.94 ms more for each of the 6,262 levels. GBBS and PASGAL grow less on these two graphs, since a stalled worker delays a round only when it holds part of it: on Twitter, they take 1.22 and 1.26 times as long at 10 ms as without stalls, and on Road-USA, PASGAL takes 2.7 times as long at 1 ms, while GBBS, whose rounds on this graph seldom give the stalled worker any work, changes by at most 8%.

The variation of runs. The runs of a setting vary by more than short stalls cost: on Twitter, the five runs of WF-DO without stalls take 44.0 to 48.7 ms per search. For Twitter, we therefore also give the mean of the five runs, each the mean of its 20 searches, with its 95% interval by Student's t, and the difference between the runs with and without stalls, with its 95% interval by Welch's method. With one thread stalled for 10 ms at every level, WF-DO takes 47.9 ms (46.3–49.4), 5.2% longer than without stalls, and the interval of the difference, −0.1 to 10.5%, just includes zero; for WF-DO-mem, it is −0.2 to 17.3%. The interval includes zero for both of our methods at every length, and excludes it for each of the other programs at the longer stalls; but it also excludes zero where PASGAL takes 3 to 4% less time with short stalls than without, which no stall explains: the runs of different settings also differ by more than those of one setting.

The other graphs. With stalls of 10 ms on LiveJournal, uk-2002, Kronecker and the graph of hubs, our methods take at most 7% longer than without stalls, while GAPBS, GBBS and PASGAL take up to 19.5, 14.6 and 7.8 times as long, GAPBS and PASGAL the most on LiveJournal and GBBS on the graph of hubs. GAPBS grows by a smaller factor on Twitter than on LiveJournal because its levels are longer, not because it waits less: it waits about once per level on both, its searches growing by 123 ms over the 12 levels of LiveJournal and by 179 ms over the 17 of Twitter, but its searches take 0.55 ms per level on LiveJournal and 3.5 ms on Twitter. On the graph of hubs, GBBS also waits about once per level, since its rounds there spread over all of its workers: its searches grow by 1.25 s over the 128 levels.

Table 10: Median time per search of our methods with 112 threads on two sockets, in ms, with threads stalled right after a winning CAS or halfway through a chunk of a hub, as often as stalls at random points for 2% of the time would be, except in hubs on Twitter at 10 and 100 μs, where every point stalls, which is less often. Road-USA has no vertex of 1,024 or more neighbors, and its stalls last at most 1 ms. The stalls of 0 μs cost what the points themselves cost.

	Twitter				Road-USA	
	After a CAS		In a hub		After a CAS	
Stall	WF-DO	WF-DO-mem	WF-DO	WF-DO-mem	WF-DO	WF-DO-mem
0	43.9	44.1	43.9	44.2	166.4	119.2
10 μs	44.8	44.6	44.1	44.0	171.8	121.8
100 μs	44.8	44.5	44.6	44.3	165.8	117.6
1 ms	44.4	44.6	44.3	44.1	165.5	117.5
10 ms	45.0	45.2	45.3	44.9	–	–

The stalled threads themselves return late. With stalls of 10 ms on Twitter, the stalled thread of WF-DO returns 182.5 ms after the search starts, and that of WF-DO-mem after 68.6 ms, since the build of WF-DO stalls it again at every level it walks through, even one that is over, while the build of WF-DO-mem skips the stall in the levels that are over (Section 6.1); on Road-USA, with stalls of 1 ms, they return after 6.27 s and 124.1 ms. Neither delays the result, which is final once the first call returns, but until it returns, a stalled thread is not free for other work, and the state of its search cannot be reused (Section 2.5).

Random stalls. Delays in a real system fall wherever the threads happen to be. In the second experiment, every worker, independently of the others, alternates between running and stalling: it runs for a random time, is then stopped by a signal wherever it is, and busy-waits in the signal handler for a set time d before it runs on. To stall it for a share s of its time, we draw its running times from an exponential distribution of mean $d(1-s)/s$: with stalls of 1 ms for 2% of the time, a worker runs for 49 ms on average between stalls. The share is that of a worker's whole run: the signals also arrive while it resets its arrays and while it waits between searches, and a signal that arrives while an earlier one is still pending merges with it, so the share of a search that a worker spends stalled can differ. Busy-waiting in a handler models a worker delayed while it holds work; it does not reproduce every effect of preemption or of page faults, such as the loss of cached data. Figure 7 keeps the share at 2% and varies the length of a stall from 10 μs to 10 ms, so that the workers lose the same time in many short stalls or in a few long ones, and Figure 8 keeps the stalls at 1 ms and varies the share from 0.5 to 50%; on Road-USA, the stalls last at most 1 ms and take at most 5% of the time. On Twitter, with stalls of 10 ms for 2% of the time, WF-DO takes 1.16 times as long as without stalls, and GAPBS, GBBS and PASGAL 2.85, 1.56 and 1.38 times as long. As the share grows, the time of WF-DO grows slowly, to 1.67 times its time without stalls with half of every worker's time stalled, since

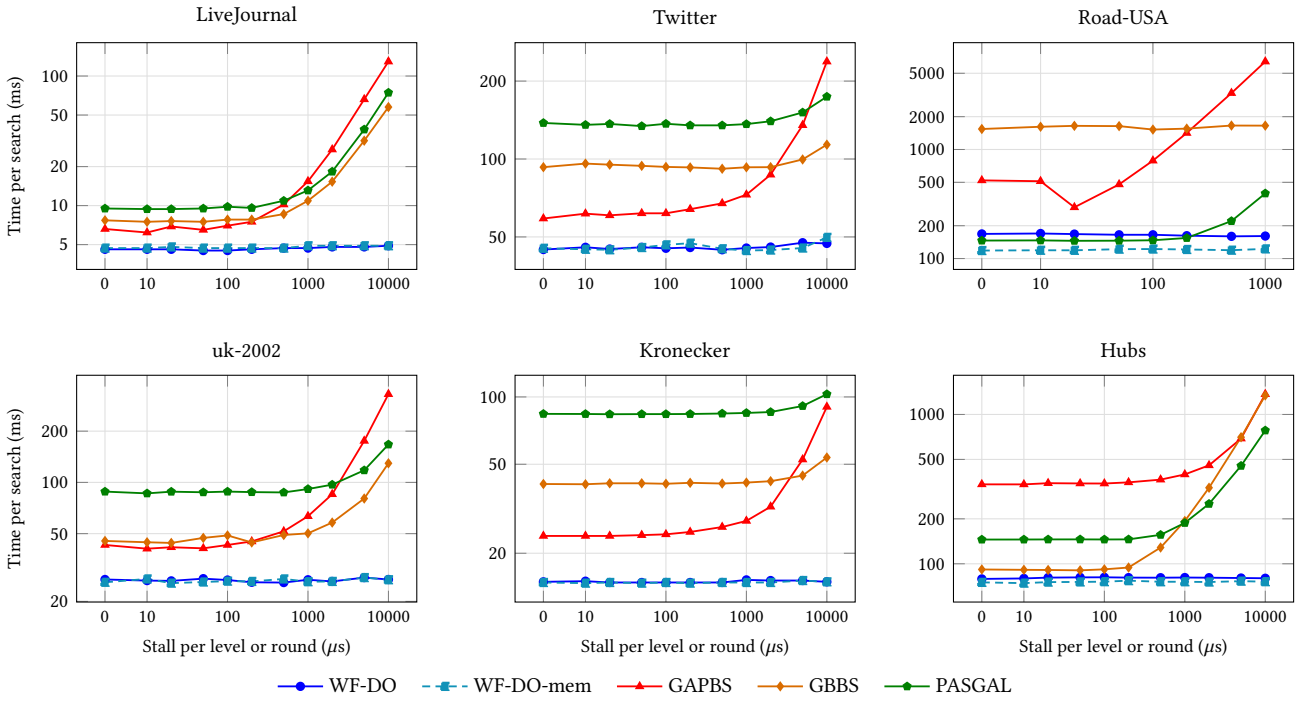


Figure 6: Median time per search with 112 threads on two sockets, against how long one of the threads stalls at the start of every level or round, on the five standard graphs and on the graph of hubs; on Road-USA, the stalls last at most 1 ms. Both axes are logarithmic, apart from the points at 0, which are the runs without stalls. Appendix C lists the times.

the workers that run take over the work of those that are stalled, while GAPBS, GBBS and PASGAL then take 2.28, 3.00 and 2.34 times as long. On Road-USA, our methods take within 4% of their times without stalls at every point, while GAPBS takes 3.3 s with stalls of 1 ms for 2% of the time and 7.8 s with 5%, PASGAL 2.0 and 3.1 times as long as without stalls, and GBBS within 12%; on the other four graphs, with half of every worker's time stalled in stalls of 1 ms, WF-DO takes 1.45 to 1.77 times as long as without stalls, and the other programs 2.7 to 23.2 times, the most for GBBS on the graph of hubs. What random stalls of 10 ms add to the time of WF-DO falls mostly in the reset before a search, which waits for every thread, rather than in the search proper: in three further runs on Twitter, the reset took 7.5 ms per search in the median, against 1.5 ms without stalls, while the search proper took 43.8 ms, against 42.4 ms.

Where a stall falls. The third experiment stalls our workers where a stall leaves the most to the others, at points of two kinds: right after the winning CAS of a discovery, and in the middle of the expansion of a *hub*, a vertex with at least 1,024 neighbors, halfway through one of its chunks, which a helper scans again in full. A worker stalls at each point of the kind with the probability that gives a search as many stalls as a share of 2% does at random points, so that the stalls differ only in where they fall, except where a graph has too few points for that, as in hubs on Twitter at 10 and 100 μs and on Kronecker at 10 μs, where every point stalls. On Twitter and Road-USA, our methods take within 4% of their times with stalls of 0 μs at every point and length (Table 10), and on the other four graphs, with stalls of 10 ms, within 4% as well, or 12% on LiveJournal, whose searches take about 5 ms.

8.9 Accounting for the Performance Gap

GAPBS and WF-DO both run direction-optimizing BFS with the same rule for switching directions, but they need not choose the same direction for every level, since the frontier sizes that WF-DO counts include copies of vertices (Section 8.3). We split the time of each to describe where the gap between them appears; the split does not measure what helping alone causes. For every parallel loop of GAPBS, we time how long each thread takes to start it, to do its share, and then to wait at the barrier that ends it, and we time its work outside the traversal: allocating and filling its distances, clearing its bitmaps, converting between its queue and its bitmap, and fixing its distances at the end. The three times of a loop add up to its wall time for each thread, and so do their means over the threads, which we sum over the steps of a search and average over its searches, so that each is a share of the wall time of a search. For WF-DO, we time the reset and the search proper of each search, the latter to the first return, as in Table 5, and in every level each thread's time to plan it, to write frontier codes, on its own share and helping others, with means over the threads in the same way. Both run in three rounds on the six graphs at 16, 64 and 112 threads, next to their builds without timers, and we use the round of each whose time is the median. These runs are separate from those of

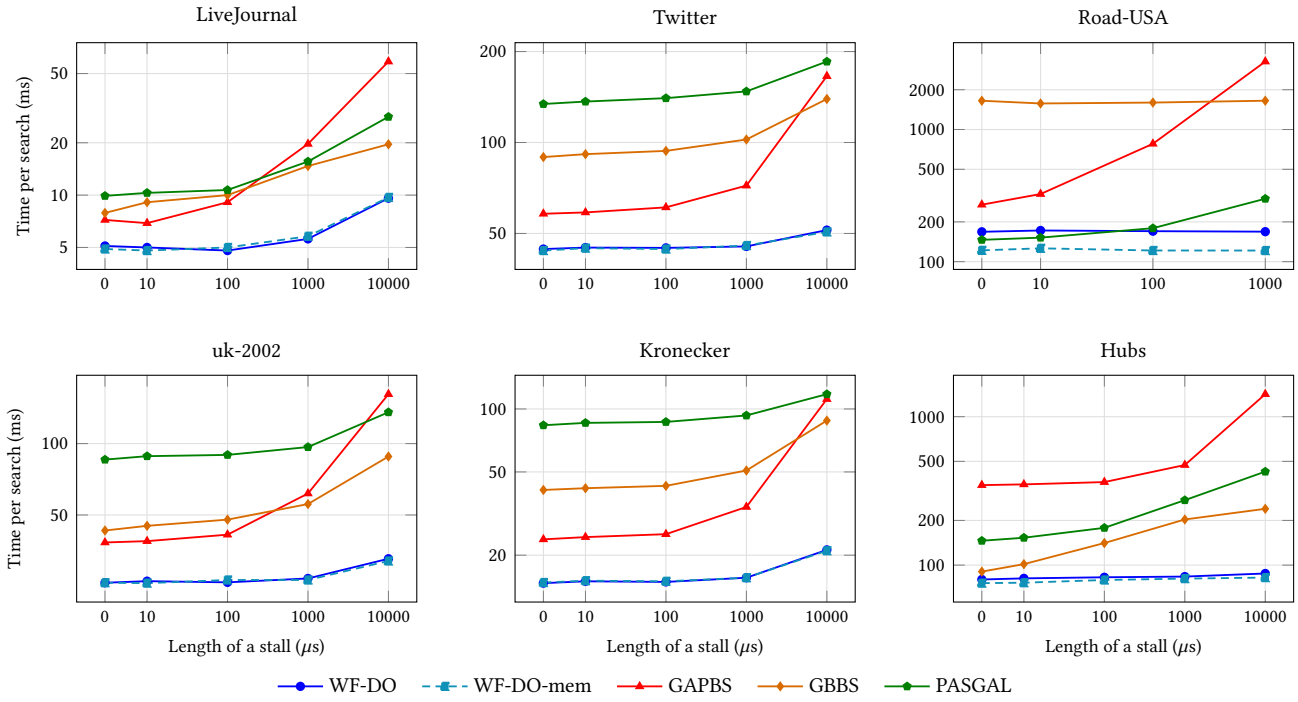


Figure 7: Median time per search with 112 threads on two sockets, with every thread stalled at random points for 2% of its time, against the length of a stall, on the five standard graphs and on the graph of hubs; on Road-USA, the stalls last at most 1 ms. Both axes are logarithmic, apart from the points at 0, which are the runs without stalls. Appendix C lists the times.

Table 4: in the mean of the three rounds, the builds without timers took from 15% less to 8% more time than in the benchmark, and the timed builds within 3% of the time of the builds without timers.

Table 11 splits the gap, the time of a search of GAPBS less that of WF-DO to its first return, in two. Let W_G be the time that the threads of GAPBS spend starting traversal steps and waiting at their ends, C_G the time that they spend working in those steps, and B_G the rest of its time, its setup and bookkeeping, with the starts and waits of their loops; let C_F be the search proper of WF-DO less an average thread's time writing frontier codes, and B_F the rest of its time, its reset and frontier codes. Then the gap is

$$(W_G + C_G + B_G) - (C_F + B_F) = (W_G + C_G - C_F) + (B_G - B_F).$$

The first term, which we call the *traversal* term, is the waiting of GAPBS less how much longer the levels of WF-DO, without frontier codes, take than the threads of GAPBS spend working, a residual rather than a time that we measure directly; the second is bookkeeping. Helping is what lets WF-DO do without that waiting, but it is not free: helpers repeat work (Section 8.3), and the levels of WF-DO have bookkeeping of their own. The split accounts for the gap, rather than removing one cause at a time: the differences between the two programs in their choices of direction, in how they represent frontiers, allocate memory and schedule threads, and in the chunks of WF-DO, all fall into its terms.

Uneven work and many steps. Where the work of many steps is uneven, or a search has thousands of steps, nearly all of the gap is

Table 11: The gap between GAPBS and WF-DO on two sockets, in ms per search: the time of each; the waiting W_G, the time that the threads of GAPBS spend starting the steps of its traversal and waiting at their ends; and the gap split into the traversal term, the waiting less how much longer the levels of WF-DO take than the threads of GAPBS spend working, and bookkeeping, the setup and bookkeeping of GAPBS beyond the reset and frontier codes of WF-DO, with their shares of the gap. From builds with timers, in the median round of three.

Graph	T	GAPBS	WF-DO	Waiting	Traversal	Bookkeeping
LiveJournal	16	15.1	11.6	1.6	0.9 (26%)	2.6 (74%)
	64	5.9	4.6	1.0	0.1 (11%)	1.1 (89%)
	112	5.2	4.1	1.1	0.1 (9%)	1.0 (91%)
Twitter	16	187.7	181.8	0.7	−3.9 (−67%)	9.7 (167%)
	64	72.5	57.4	0.8	7.6 (51%)	7.4 (49%)
	112	58.9	42.3	1.1	9.1 (55%)	7.5 (45%)
Road-USA	16	259.3	138.8	76.0	111.8 (93%)	8.8 (7%)
	64	333.5	111.6	134.7	211.2 (95%)	10.7 (5%)
	112	514.8	150.5	203.6	336.3 (92%)	28.1 (8%)
uk-2002	16	86.1	73.7	4.2	7.1 (57%)	5.3 (43%)
	64	40.9	27.1	5.1	9.0 (65%)	4.8 (35%)
	112	38.5	24.5	6.3	8.7 (62%)	5.3 (38%)
Kronecker	16	73.0	64.8	1.5	2.2 (27%)	6.0 (73%)
	64	29.0	19.5	2.1	4.5 (47%)	5.0 (53%)
	112	24.9	14.0	2.2	5.4 (49%)	5.5 (51%)
Hubs	16	330.7	275.8	139.8	50.4 (92%)	4.6 (8%)
	64	300.3	82.2	228.7	212.8 (98%)	5.4 (2%)
	112	294.8	68.1	242.0	221.7 (98%)	5.0 (2%)

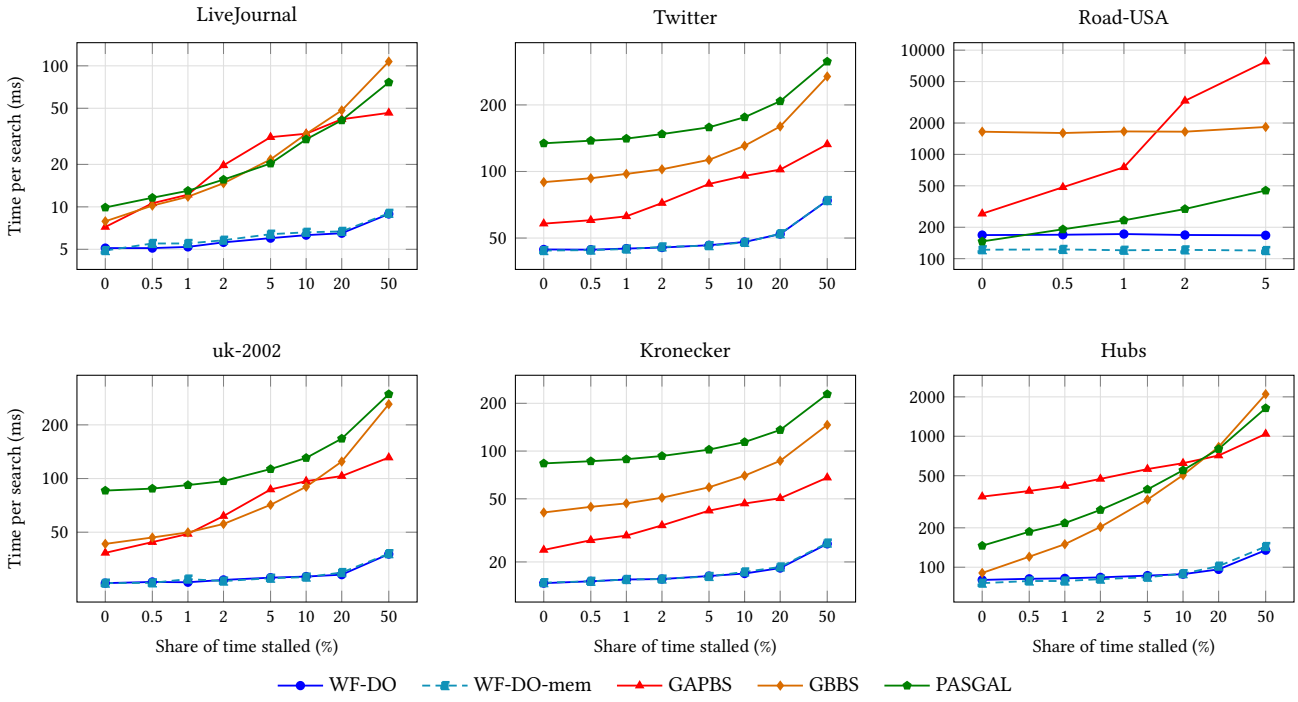


Figure 8: Median time per search with 112 threads on two sockets, with every thread stalled at random points in stalls of 1 ms, against the share of its time stalled, on the five standard graphs and on the graph of hubs; on Road-USA, up to 5%. Both axes are logarithmic, apart from the points at 0, which are the runs without stalls. Appendix C lists the times.

Table 12: The work that helpers do in WF-DO on LiveJournal on two sockets with 112 threads, one of which stalls at the start of every level, in percent: their share of the vertices expanded and of the neighbors examined in top-down levels, of the blocks scanned in bottom-up levels, and of the blocks of the stalled thread; and the share of the neighbors examined, and of the scans of blocks, that repeat work that another thread also finished.

	Done by helpers				Repeated	
Stall	Vertices	Neighbors	Blocks	Stalled	Neighbors	Blocks
0	52.0	62.1	25.8	24.0	15.2	11.2
100 μs	54.3	61.5	26.0	56.8	15.2	11.3
500 μs	51.0	60.9	27.6	95.3	15.0	12.2
1 ms	51.1	60.5	30.0	99.6	15.0	12.8
2 ms	52.8	61.6	29.1	100.0	15.1	12.8
10 ms	54.2	61.4	28.5	100.0	14.9	12.6

in the traversal term at every count. On the graph of hubs, GAPBS expands each vertex of a top-down step in a single thread, so a step of 8 hubs keeps at most 8 threads busy while the others wait; on Road-USA, the 6,259 top-down steps of GAPBS, of its 6,262 in all, give each thread a few dozen to a few hundred vertices on average, so that its threads spend 44 to 72% as long starting and ending steps as working in them, and its bookkeeping there includes the cost of the timers between steps. On uk-2002, the traversal term is more than half of the gap at every count, and at 16 and 64 threads, 35 to 50% of the waiting of GAPBS lies in one top-down step, the last before its first bottom-up one, in which one thread works for 2.0 to 2.6 ms while the mean is 0.2 to 0.5 ms, most likely on a vertex of high degree that the static schedule of GAPBS leaves to one thread.

Few levels. On the other graphs, whose levels are few, the gap is mostly bookkeeping on LiveJournal at every count and on Kronecker at 16 threads, and about half bookkeeping elsewhere, except on Twitter at 16 threads: there the threads of GAPBS wait only 0.7 ms per search in traversal steps, less than the levels of WF-DO take beyond the time that the threads of GAPBS spend working, so that the traversal term is negative and bookkeeping accounts for more than the whole gap. GAPBS clears its bitmaps serially, two before the first step and one before every bottom-up step; lighter bookkeeping, unlike the absence of waiting, is something that a blocking implementation could also have.

How helping spreads the work. Table 12, from a build of WF-DO that counts the work that each thread does for its own share and for those of others, shows what helpers do on LiveJournal at 112 threads. Without stalls, they expand half of the vertices of top-down levels and examine 62% of their neighbors, about 1.5 times as many per vertex as owners, since helping happens where a bucket takes long, and they scan 26% of the blocks of bottom-up levels; 15% of the neighbors examined in top-down levels, and 11% of the scans of blocks, repeat work that another thread also finished. When one thread stalls at the start of every level, the others take over its share,

Table 13: Ways to run a parallel BFS, and what they give: whether a search is wait-free, the work of a top-down level d, in terms of its vertices L_d, their neighbors $\mathrm{vol}(L_d)$ and the positions $Q_d \le T|L_d|$ of its buckets, or of the graph, and the work that threads repeat. The bound for WF-DO holds for every schedule. If helpers take positions in independent uniformly random orders, and marks are sequentially consistent or fenced, then with high probability they also expand vertices at only $O((Q_d + T^2)\log n)$ positions (Theorem 5.17); our implementation's affine orders and unfenced marks do not meet these assumptions.

Approach	Wait-free	Work of a level	Repeated work
Trivial (Section 2.3)	yes	$T(\|L_d\| + \mathrm{vol}(L_d))$	all of it, $T-1$ times
All-vertex CWA per level [40, 41]	yes	at least n	within the CWA's bound
Work stealing [9, 16, 17]	no	$O(\|L_d\| + \mathrm{vol}(L_d))$	no task runs twice; PASGAL revisits vertices
Idempotent stealing [42]	no	$O(\|L_d\| + \mathrm{vol}(L_d))$	stolen tasks, at times
Asynchronous BFS [47]	no	$O(\|L_d\| + \mathrm{vol}(L_d))$	vertices revisited
WF-DO (this paper)	yes	$O(T(Q_d + \mathrm{vol}(L_d)) + T^2)$	measured: 0.3–24% more neighbors on the standard graphs, up to 104% on hubs (Table 6)

scanning more than half of its blocks with stalls of 100 μs and nearly all of them from 1 ms on, and from 500 μs on, the stalled thread has nothing to expand in top-down levels, since it reaches every level late and the others discover every vertex; the repeated work grows little with the stall.

9 Related Work

Do-All and asynchronous PRAMs. Our method belongs with the work on Write-All, Do-All and Certified Write-All [21, 32, 41] (Section 2.3). Some algorithms for these problems let processors walk the tasks in permutations of their own and record their progress in shared memory, as our threads walk buckets in random orders and publish their progress and certificates, and they bound the work that processors repeat by the contention of the permutations [3], which our analysis adapts (Section 5.7). A CWA per step runs any PRAM program on asynchronous processors [33, 40], so these results give a wait-free BFS, most simply with one CWA over all n vertices per level, at work $\Omega(Dn)$, a bound of this mapping rather than of every construction built on CWA; to our knowledge, none of these constructions was implemented to compete with tuned parallel BFS. Our method differs from them in that its tasks are discovered during the computation, by the same threads, and grouped by the thread that discovered them, so that a thread certifies a level by reading T flags rather than n cells; in that its tasks are weighted by degrees, so that what the others redo for a stopped thread depends on how finely the work is certified (Proposition 5.15); and in that its threads agree on a plan for each level with one CAS. Table 13 compares these approaches: among the wait-free ones, ours is the only one whose threads share the work of a top-down level at a cost that depends on the level rather than on the graph.

Wait-free objects and tasks. Wait-freedom is studied mostly where inputs are distributed, for concurrent objects [23, 25, 35] and for decision tasks such as consensus [18] and set agreement, whose wait-free solvability Herlihy and Shavit characterize [24]; in a search, it is a question of work, not of solvability (Section 2.3). Wait-freedom has also been studied for concurrent graph data structures. Kallimanis and Kanellou [30] provide wait-free graph objects with consistent dynamic traversals, using helping to complete pending operations, and Bhardwaj et al. [7] use multiversioning to take wait-free snapshots of a lock-free dynamic graph, on which queries such as BFS can run during concurrent updates. These works address consistent operations and traversals of a changing graph, each run by one thread, while our threads cooperate on a single search of a graph that does not change.

Parallel BFS. The parallel BFS methods for shared memory are mostly level-synchronous, from the fine-grained multithreaded BFS of Bader and Madduri for the Cray MTA-2 [4] and the work-efficient BFS of Leiserson and Schardl, which builds frontiers with bags and Cilk++ reducers [38], to the direction-optimizing BFS of Beamer et al. [5], which GAPBS implements [6] and whose direction rule our method uses, and the vertex subsets of Ligra [51] and GBBS [16], built on Ligra. PASGAL [17] addresses the synchronization and scheduling overhead of large-diameter graphs with the vertical granularity control and hash bags of Wang et al. [52]: a task explores several hops locally, which reduces the number of global rounds, and vertices may be revisited when shorter distances are found. All of these are blocking: they end every step at a barrier or a join.

BFS without barriers. Other parallel BFS methods relax the order between levels. Pearce et al. [47] traverse graphs asynchronously in shared memory, for BFS, shortest paths from a source, and connected components: threads process visits to vertices from queues, with no barriers between levels, which hides the latency of memory and of solid-state storage, and visit a vertex again when they find a shorter path to it. On GPUs, XBFS [19] makes the bottom-up step asynchronous, so that one iteration can discover vertices of more than one level, but each iteration still starts once the one before has ended. Unordered BFS gives up levels altogether, at the cost of potentially more work; Holst and Selin [27] schedule it with a relaxed FIFO queue, and report speedups of 1.6 to 2.4 over a level-synchronous BFS on sparse, high-diameter graphs. None of these makes a search wait-free: XBFS waits for its slowest thread at the end of every iteration, and in an asynchronous or unordered BFS without helping, a thread that is suspended after taking a vertex or a visit from a queue holds work that no other thread will do, so that the search cannot complete until it resumes.

Work stealing. Work stealing [9] balances load by letting a thread take tasks from the queues of others, and hands out every task exactly once, so a thread that stops after taking a task holds it for good. Idempotent work stealing [42] hands a task out at least once,

which lets the owner's operations avoid atomic instructions and store-load fences, for applications that tolerate or detect repeated tasks, such as transitive closure and spanning trees. Our methods tolerate repeated work as well, but that alone does not make a search wait-free: a task that a stopped thread has taken may be handed out again, but nothing ensures that it will be. What our methods add is an order of publishing work and progress in which a stopped thread leaves nothing that the others cannot redo: the work stays in its bucket or block, where the others redo what has not been certified, and each thread decides alone when a level is complete.

10 Conclusion

We presented a direction-optimizing BFS that is wait-free under TSO, given that a thread can allocate memory in a bounded number of its own steps: every vertex receives its BFS distance, final once any call returns, and the work of a top-down level depends only on that level and on T. A variant also bounds its memory by the largest levels, plus a constant per level. On the five standard graphs, our implementation examined at most 30% more neighbors than a sequential search at 112 threads, its best observed times were below those of GAPBS, GBBS and PASGAL, and with a thread stalled at every level it lost little, while GAPBS, which waits at every level, lost about one stall per level.

Since every thread takes part in every level, the method is weakest on graphs with many levels of few vertices, where blocking implementations that leave such a level to one thread are faster; and the reset before a search, outside the method, still waits for every thread. The main open questions are running ahead of small levels without changing a published distance, cheap orders with provable contention bounds, and reclaiming and resetting the state of a search without waiting for a stopped thread; others concern direction choices at high thread counts, hubs, and weaker memory models. Finally, nothing in the structure of the method, a list of levels whose per-thread buckets any thread can finish, is specific to BFS, and it could make other frontier-based graph algorithms wait-free, such as connectivity, or single-source shortest paths, whose buckets of tentative distances play the role of levels.

Acknowledgments

I thank Dr. Sathya Peri and Dr. Sandeep Kulkarni for their comments and suggestions on an earlier manuscript arising from this project.

References

[1] Advanced Micro Devices, Inc. 2022. AMD64 Architecture Programmer's Manual, Volume 2: System Programming. Publication No. 24593, Revision 3.39, October 2022.

[2] Richard J. Anderson and Heather Woll. 1991. Wait-Free Parallel Algorithms for the Union-Find Problem. In *Proceedings of the 23rd Annual ACM Symposium on Theory of Computing (STOC '91)*. ACM, New York, NY, USA, 370–380. doi:10.1145/103418.103458

[3] Richard J. Anderson and Heather Woll. 1997. Algorithms for the Certified Write-All Problem. *SIAM J. Comput.* 26, 5 (1997), 1277–1283. doi:10.1137/S0097539794319126

[4] David A. Bader and Kamesh Madduri. 2006. Designing Multithreaded Algorithms for Breadth-First Search and st-connectivity on the Cray MTA-2. In *Proceedings of the 2006 International Conference on Parallel Processing* (Columbus, OH, USA) *(ICPP '06)*. IEEE, 523–530. doi:10.1109/ICPP.2006.34

[5] Scott Beamer, Krste Asanović, and David Patterson. 2012. Direction-Optimizing Breadth-First Search. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis* (Salt Lake City, UT, USA) *(SC '12)*. IEEE, Article 12, 10 pages. doi:10.1109/SC.2012.50

[6] Scott Beamer, Krste Asanović, and David Patterson. 2015. The GAP Benchmark Suite. arXiv:1508.03619 [cs.DC]

[7] Gaurav Bhardwaj, Ayaz Ahmed, and Sathya Peri. 2024. Concurrent Wait-Free Graph Snapshots Using Multi-versioning. In *Networked Systems: 12th International Conference, NETYS 2024* (Rabat, Morocco) *(Lecture Notes in Computer Science, Vol. 14783)*. Springer, Cham, Switzerland, 32–49. doi:10.1007/978-3-031-67321-4_2

[8] Guy E. Blelloch, Daniel Anderson, and Laxman Dhulipala. 2020. Brief Announcement: ParlayLib – A Toolkit for Parallel Algorithms on Shared-Memory Multicore Machines. In *Proceedings of the 32nd ACM Symposium on Parallelism in Algorithms and Architectures* (Virtual Event, USA) *(SPAA '20)*. ACM, New York, NY, USA, 507–509. doi:10.1145/3350755.3400254

[9] Robert D. Blumofe and Charles E. Leiserson. 1999. Scheduling Multithreaded Computations by Work Stealing. *J. ACM* 46, 5 (1999), 720–748. doi:10.1145/324133.324234

[10] Hans-J. Boehm. 2011. How to Miscompile Programs with "Benign" Data Races. In *Proceedings of the 3rd USENIX Workshop on Hot Topics in Parallelism (HotPar '11)*. USENIX Association, Berkeley, CA, USA.

[11] Hans-J. Boehm and Sarita V. Adve. 2008. Foundations of the C++ Concurrency Memory Model. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '08)*. ACM, New York, NY, USA, 68–78. doi:10.1145/1375581.1375591

[12] Paolo Boldi and Sebastiano Vigna. 2004. The WebGraph Framework I: Compression Techniques. In *Proceedings of the 13th International Conference on World Wide Web* (New York, NY, USA) *(WWW '04)*. ACM, New York, NY, USA, 595–602. doi:10.1145/988672.988752

[13] Bogdan S. Chlebus and Dariusz R. Kowalski. 2005. Cooperative Asynchronous Update of Shared Memory. In *Proceedings of the 37th Annual ACM Symposium on Theory of Computing (STOC '05)*. ACM, New York, NY, USA, 733–739. doi:10.1145/1060590.1060698

[14] Richard Cole and Ofer Zajicek. 1989. The APRAM: Incorporating Asynchrony into the PRAM Model. In *Proceedings of the First Annual ACM Symposium on Parallel Algorithms and Architectures (SPAA '89)*. ACM, New York, NY, USA, 169–178. doi:10.1145/72935.72954

[15] Timothy A. Davis and Yifan Hu. 2011. The University of Florida Sparse Matrix Collection. *ACM Trans. Math. Software* 38, 1, Article 1 (2011), 25 pages. doi:10.1145/2049662.2049663

[16] Laxman Dhulipala, Guy E. Blelloch, and Julian Shun. 2021. Theoretically Efficient Parallel Graph Algorithms Can Be Fast and Scalable. *ACM Trans. Parallel Comput.* 8, 1, Article 4 (2021), 70 pages. doi:10.1145/3434393

[17] Xiaojun Dong, Yan Gu, Yihan Sun, and Letong Wang. 2024. Brief Announcement: PASGAL: Parallel And Scalable Graph Algorithm Library. In *Proceedings of the 36th ACM Symposium on Parallelism in Algorithms and Architectures* (Nantes, France) *(SPAA '24)*. ACM, New York, NY, USA, 439–441. doi:10.1145/3626183.3660258

[18] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. 1985. Impossibility of Distributed Consensus with One Faulty Process. *J. ACM* 32, 2 (1985), 374–382. doi:10.1145/3149.214121

[19] Anil Gaihre, Zhenlin Wu, Fan Yao, and Hang Liu. 2019. XBFS: eXploring Runtime Optimizations for Breadth-First Search on GPUs. In *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing* (Phoenix, AZ, USA) *(HPDC '19)*. ACM, New York, NY, USA, 121–131. doi:10.1145/3307681.3326606

[20] Chryssis Georgiou, Alexander Russell, and Alex A. Shvartsman. 2004. The Complexity of Synchronous Iterative Do-All with Crashes. *Distrib. Comput.* 17, 1 (2004), 47–63. doi:10.1007/s00446-003-0099-3

[21] Chryssis Georgiou and Alex A. Shvartsman. 2008. *Do-All Computing in Distributed Systems: Cooperation in the Presence of Adversity*. Springer, New York, NY, USA. doi:10.1007/978-0-387-69045-2

[22] Phillip B. Gibbons. 1989. A More Practical PRAM Model. In *Proceedings of the First Annual ACM Symposium on Parallel Algorithms and Architectures (SPAA '89)*. ACM, New York, NY, USA, 158–168. doi:10.1145/72935.72953

[23] Maurice Herlihy. 1991. Wait-Free Synchronization. *ACM Trans. Program. Lang. Syst.* 13, 1 (1991), 124–149. doi:10.1145/114005.102808

[24] Maurice Herlihy and Nir Shavit. 1999. The Topological Structure of Asynchronous Computability. *J. ACM* 46, 6 (1999), 858–923. doi:10.1145/331524.331529

[25] Maurice Herlihy and Nir Shavit. 2012. *The Art of Multiprocessor Programming, Revised Reprint* (1st ed.). Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.

[26] Torsten Hoefler, Timo Schneider, and Andrew Lumsdaine. 2010. Characterizing the Influence of System Noise on Large-Scale Applications by Simulation. In *Proceedings of the 2010 ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis* (New Orleans, LA, USA) *(SC '10)*. IEEE, 1–11. doi:10.1109/SC.2010.12

[27] Simon Holst and Johan Selin. 2025. *A Faster Breadth-First Search on Sparse Graphs*. Master's thesis. Chalmers University of Technology, Gothenburg, Sweden. https:

//hdl.handle.net/20.500.12380/310782
[28] Intel Corporation. 2022. Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3 (3A, 3B, 3C & 3D): System Programming Guide. Order Number 325384-078US, December 2022. https://cdrdv2-public.intel.com/671447/325384-sdm-vol-3abcd.pdf
[29] International Organization for Standardization. 2024. *ISO/IEC 14882:2024, Programming Languages – C++*. The C++23 standard.
[30] Nikolaos D. Kallimanis and Eleni Kanellou. 2016. Wait-Free Concurrent Graph Objects with Dynamic Traversals. In *19th International Conference on Principles of Distributed Systems (OPODIS 2015)* (Rennes, France) *(Leibniz International Proceedings in Informatics (LIPIcs), Vol. 46)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 27:1–27:17. doi:10.4230/LIPIcs.OPODIS.2015.27
[31] Paris C. Kanellakis and Alex A. Shvartsman. 1991. Efficient Parallel Algorithms on Restartable Fail-Stop Processors. In *Proceedings of the 10th Annual ACM Symposium on Principles of Distributed Computing (PODC '91)*. ACM, New York, NY, USA, 23–36. doi:10.1145/112600.112603
[32] Paris C. Kanellakis and Alex A. Shvartsman. 1992. Efficient Parallel Algorithms Can Be Made Robust. *Distrib. Comput.* 5, 4 (1992), 201–217. doi:10.1007/BF02277667
[33] Paris C. Kanellakis and Alex A. Shvartsman. 1997. *Fault-Tolerant Parallel Computation*. The Springer International Series in Engineering and Computer Science, Vol. 401. Kluwer Academic Publishers, Boston, MA, USA. doi:10.1007/978-1-4757-5210-6
[34] H. J. Kanold. 1967. Über eine zahlentheoretische Funktion von Jacobsthal. *Math. Ann.* 170, 4 (1967), 314–326. doi:10.1007/BF01350607
[35] Alex Kogan and Erez Petrank. 2012. A Methodology for Creating Fast Wait-Free Data Structures. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (New Orleans, LA, USA) *(PPoPP '12)*. ACM, New York, NY, USA, 141–150. doi:10.1145/2145816.2145835
[36] Dariusz R. Kowalski and Alex A. Shvartsman. 2008. Writing-All Deterministically and Optimally Using a Nontrivial Number of Asynchronous Processors. *ACM Trans. Algorithms* 4, 3 (2008), 33:1–33:22. doi:10.1145/1367064.1367073
[37] Haewoon Kwak, Changhyun Lee, Hosung Park, and Sue Moon. 2010. What is Twitter, a Social Network or a News Media?. In *Proceedings of the 19th International Conference on World Wide Web* (Raleigh, NC, USA) *(WWW '10)*. ACM, New York, NY, USA, 591–600. doi:10.1145/1772690.1772751
[38] Charles E. Leiserson and Tao B. Schardl. 2010. A Work-Efficient Parallel Breadth-First Search Algorithm (or How to Cope with the Nondeterminism of Reducers). In *Proceedings of the 22nd ACM Symposium on Parallelism in Algorithms and Architectures* (Thira, Santorini, Greece) *(SPAA '10)*. ACM, New York, NY, USA, 303–314. doi:10.1145/1810479.1810534
[39] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. http://snap.stanford.edu/data.
[40] Charles U. Martel, Arvin Park, and Ramesh Subramonian. 1992. Work-Optimal Asynchronous Algorithms for Shared Memory Parallel Computers. *SIAM J. Comput.* 21, 6 (1992), 1070–1099. doi:10.1137/0221063
[41] Charles U. Martel and Ramesh Subramonian. 1994. On the Complexity of Certified Write-All Algorithms. *J. Algorithms* 16, 3 (1994), 361–387. doi:10.1006/jagm.1994.1017
[42] Maged M. Michael, Martin T. Vechev, and Vijay A. Saraswat. 2009. Idempotent Work Stealing. In *Proceedings of the 14th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Raleigh, NC, USA) *(PPoPP '09)*. ACM, New York, NY, USA, 45–54. doi:10.1145/1504176.1504186
[43] Richard C. Murphy, Kyle B. Wheeler, Brian W. Barrett, and James A. Ang. 2010. Introducing the Graph 500. In *Cray User's Group (CUG)* (Edinburgh, UK).
[44] Ruslan Nikolaev and Binoy Ravindran. 2022. wCQ: A Fast Wait-Free Queue with Bounded Memory Usage. In *Proceedings of the 34th ACM Symposium on Parallelism in Algorithms and Architectures* (Philadelphia, PA, USA) *(SPAA '22)*. ACM, New York, NY, USA, 307–319. doi:10.1145/3490148.3538572
[45] Ruslan Nikolaev and Binoy Ravindran. 2024. A Family of Fast and Memory Efficient Lock- and Wait-Free Reclamation. *Proc. ACM Program. Lang.* 8, PLDI, Article 235 (2024), 25 pages. doi:10.1145/3658851
[46] Naomi Nishimura. 1990. Asynchronous Shared Memory Parallel Computation. In *Proceedings of the Second Annual ACM Symposium on Parallel Algorithms and Architectures (SPAA '90)*. ACM, New York, NY, USA, 76–84. doi:10.1145/97444.97672
[47] Roger A. Pearce, Maya B. Gokhale, and Nancy M. Amato. 2010. Multithreaded Asynchronous Graph Traversal for In-Memory and Semi-External Memory. In *Proceedings of the 2010 ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis* (New Orleans, LA, USA) *(SC '10)*. IEEE, 1–11. doi:10.1109/SC.2010.34
[48] Fabrizio Petrini, Darren J. Kerbyson, and Scott Pakin. 2003. The Case of the Missing Supercomputer Performance: Achieving Optimal Performance on the 8,192 Processors of ASCI Q. In *Proceedings of the 2003 ACM/IEEE Conference on Supercomputing* (Phoenix, AZ, USA) *(SC '03)*. ACM, New York, NY, USA, 55. doi:10.1145/1048935.1050204
[49] Ryan A. Rossi and Nesreen K. Ahmed. 2015. The Network Data Repository with Interactive Graph Analytics and Visualization. In *Proceedings of the 29th AAAI Conference on Artificial Intelligence (AAAI '15)*. doi:10.1609/aaai.v29i1.9277
[50] Peter Sewell, Susmit Sarkar, Scott Owens, Francesco Zappa Nardelli, and Magnus O. Myreen. 2010. x86-TSO: A Rigorous and Usable Programmer's Model for x86 Multiprocessors. *Commun. ACM* 53, 7 (2010), 89–97. doi:10.1145/1785414.1785443
[51] Julian Shun and Guy E. Blelloch. 2013. Ligra: A Lightweight Graph Processing Framework for Shared Memory. In *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Shenzhen, China) *(PPoPP '13)*. ACM, New York, NY, USA, 135–146. doi:10.1145/2442516.2442530
[52] Letong Wang, Xiaojun Dong, Yan Gu, and Yihan Sun. 2023. Parallel Strong Connectivity Based on Faster Reachability. *Proc. ACM Manag. Data* 1, 2, Article 114 (2023), 29 pages. doi:10.1145/3589259

A Drawing Affine Orders

An affine order of $k \geq 2$ items (Section 7) needs a stride b coprime to k. Our implementation obtains a candidate stride in $1, \ldots, k-1$ from a random number by a remainder, which is close to uniform but slightly biased, and if it is not coprime to k, tries the next one up, until it finds one.

Lemma A.1 (Affine orders). *For $k \geq 2$, the search for a stride coprime to k stops after at most $2\sqrt{k}$ tries, of $O(\log k)$ steps each, so it takes $O(\sqrt{k} \log k) \subseteq O(k)$ steps.*

Proof. Euclid's algorithm tests a stride in $O(\log k)$ steps. The search never passes $k-1$, which is coprime to k, so it stops within $j(k)$ tries, where $j(k)$, Jacobsthal's function, is the least number such that any $j(k)$ consecutive integers include one coprime to k. Kanold showed that $j(k) \leq 2^{\omega(k)}$, where $\omega(k)$ is the number of distinct prime factors of k [34], and $2^{\omega(k)}$ is at most the number of divisors of k, which is at most $2\sqrt{k}$. □

B Times per Thread Count

Tables 14 to 19 list the median times per search that Figure 5 plots, and those at 128 threads, for every graph and program; Section 8.1 describes how we measured them.

Table 14: Median time per search on LiveJournal, in ms, at each number of threads T. The sequential baseline takes 96.2 ms.

T	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
1	113.5	114.5	104.1	155.3	267.4
2	81.3	81.9	80.9	97.0	214.6
4	43.3	43.0	45.6	53.8	118.6
8	23.1	23.1	25.9	30.1	63.6
16	12.7	12.9	15.2	17.0	34.4
24	9.2	9.2	11.5	12.6	24.7
32	7.5	7.5	9.4	10.4	19.6
40	6.4	6.5	8.3	9.1	16.8
48	5.8	5.8	7.6	8.4	14.9
56	5.3	5.4	6.9	7.8	13.7
64	4.9	5.1	6.4	7.5	12.6
80	4.6	4.6	5.8	6.9	11.0
96	4.4	4.5	5.9	6.8	10.2
112	4.2	4.5	5.4	6.7	9.6
128	5.5	6.2	7.8	6.8	9.2

Table 15: Median time per search on Twitter, in ms, at each number of threads T. The sequential baseline takes 3038.2 ms.

T	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
1	3167.5	3169.7	3044.0	3853.7	6214.7
2	1715.8	1718.0	1549.7	2002.8	4423.2
4	841.1	841.0	767.1	1013.4	2332.5
8	420.4	422.2	391.2	515.3	1220.5
16	213.5	214.5	203.2	269.1	642.4
24	145.0	145.3	141.6	187.7	443.0
32	111.3	111.3	111.1	149.7	341.9
40	91.6	91.9	93.2	128.2	281.8
48	78.3	78.9	81.4	114.7	241.7
56	69.3	69.5	74.1	104.0	216.6
64	62.5	62.8	69.1	100.2	197.1
80	53.2	53.4	61.9	91.6	168.9
96	47.6	47.7	58.1	89.4	149.0
112	43.8	43.5	55.7	89.0	134.8
128	43.6	43.1	73.1	94.8	121.3

Table 16: Median time per search on Road-USA, in ms, at each number of threads T. The sequential baseline takes 851.3 ms. GAPBS ran at 128 threads in runs of its own.

T	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
1	1181.8	987.5	1009.5	1288.8	3220.5
2	612.7	517.2	630.3	1484.6	2634.2
4	344.0	303.7	471.6	1593.7	1467.8
8	218.2	194.0	335.2	1387.2	832.9
16	153.9	136.7	278.5	1296.3	472.8
24	130.4	113.0	273.4	1282.1	346.4
32	119.2	101.7	276.9	1273.3	278.4
40	114.2	98.5	292.0	1300.3	238.8
48	113.7	94.4	302.5	1344.0	214.5
56	115.0	96.1	319.6	1380.8	195.5
64	118.7	94.9	337.2	1406.0	183.2
80	131.3	102.1	389.4	1480.0	164.4
96	151.0	113.4	441.0	1561.6	153.9
112	169.1	126.6	520.2	1632.3	147.2
128	171.6	123.5	1400.6	1698.6	165.2

Table 17: Median time per search on uk-2002, in ms, at each number of threads T. The sequential baseline takes 1183.7 ms.

T	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
1	1203.0	1201.0	1188.2	1437.7	3662.7
2	633.3	631.3	653.1	728.7	2508.7
4	319.6	319.4	335.7	377.5	1357.2
8	164.0	164.0	174.5	193.1	735.4
16	86.2	86.6	96.1	105.3	393.1
24	60.1	60.3	69.0	75.6	274.2
32	47.1	47.0	56.4	61.5	215.7
40	39.8	39.7	48.9	53.9	179.3
48	34.6	34.8	44.4	48.8	155.8
56	31.1	31.7	42.3	45.9	139.2
64	29.0	29.3	40.0	43.5	124.9
80	27.6	27.3	38.4	41.7	106.3
96	25.5	25.4	37.2	40.1	93.2
112	25.4	25.8	37.1	40.8	84.3
128	27.7	27.5	51.1	45.6	85.3

Table 18: Median time per search on Kronecker, in ms, at each number of threads T. The sequential baseline takes 985.5 ms.

T	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
1	984.9	980.3	1017.6	1452.1	4896.8
2	530.0	529.0	541.7	797.0	3522.7
4	268.7	265.3	269.5	416.5	1790.6
8	131.9	132.6	137.6	213.2	928.9
16	66.9	67.0	73.9	111.4	483.4
24	45.7	45.8	51.8	78.8	331.6
32	36.0	35.5	42.1	63.8	252.5
40	28.9	29.1	34.9	54.6	205.7
48	24.9	25.0	31.8	49.1	174.3
56	22.3	22.4	29.5	45.6	151.6
64	20.2	20.4	28.2	43.4	134.4
80	17.2	17.2	25.0	40.5	110.3
96	15.4	15.5	25.1	38.3	94.6
112	14.1	14.1	23.2	38.0	83.4
128	13.3	13.1	32.3	40.7	77.0

Table 19: Median time per search on the graph of hubs (Section 8.6), in ms, at each number of threads T. The sequential baseline takes 1463.9 ms.

T	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
1	2381.9	2178.7	1761.3	2237.5	5112.0
2	1602.7	1490.2	1338.2	1398.0	4249.4
4	918.9	850.7	720.3	895.6	2237.1
8	532.8	498.5	442.5	509.3	1191.8
16	280.2	261.8	371.0	277.2	657.4
24	186.2	178.3	351.4	197.8	472.3
32	142.2	136.3	339.7	160.5	372.6
40	117.6	113.6	335.6	140.1	313.1
48	102.8	98.7	332.7	129.1	269.9
56	92.9	88.8	333.2	120.1	240.1
64	85.3	81.5	333.5	114.1	216.1
80	79.3	75.4	337.0	103.6	183.8
96	76.6	73.8	338.4	98.2	160.8
112	75.1	70.9	343.7	90.1	141.5
128	64.5	61.8	527.0	94.9	157.2

C Times with Stalled Threads

Tables 20 to 37 list the median times per search with 112 threads that Figures 6, 7 and 8 plot, for every graph, program and stall; the rows at 0 are the runs without stalls, and Section 8.8 describes how we measured them.

Table 20: Median time per search on LiveJournal, in ms, by the length of the stall of one thread per level or round (Figure 6).

Stall (μs)	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
0	4.6	4.7	6.6	7.7	9.5
10	4.6	4.7	6.2	7.5	9.4
20	4.6	4.8	6.9	7.6	9.4
50	4.5	4.7	6.5	7.5	9.5
100	4.5	4.7	7.0	7.8	9.8
200	4.6	4.7	7.5	7.8	9.6
500	4.7	4.7	10.2	8.6	10.9
1000	4.7	4.9	15.4	10.9	13.1
2000	4.8	4.9	27.1	15.3	18.3
5000	4.8	4.9	66.1	31.7	38.7
10000	4.9	4.9	129.3	57.6	74.4

Table 21: Median time per search on Twitter, in ms, by the length of the stall of one thread per level or round (Figure 6).

Stall (μs)	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
0	44.7	45.2	58.9	93.0	137.8
10	45.6	44.8	61.5	96.0	135.5
20	44.9	44.6	60.6	95.1	136.6
50	45.6	45.5	61.7	94.1	134.0
100	45.2	46.6	61.7	93.2	136.8
200	45.5	47.3	64.0	92.8	134.9
500	44.6	45.0	67.4	91.7	134.9
1000	45.3	44.3	72.8	92.9	136.4
2000	45.7	44.5	87.0	93.0	139.9
5000	47.5	45.3	135.1	99.6	151.3
10000	47.2	49.8	237.9	113.6	174.1

Table 22: Median time per search on Road-USA, in ms, by the length of the stall of one thread per level or round (Figure 6).

Stall (μs)	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
0	168.2	118.4	521.0	1541.5	146.2
10	169.7	119.1	511.7	1616.6	146.7
20	167.6	119.0	295.0	1647.7	145.4
50	165.2	121.9	478.5	1638.3	145.8
100	165.2	122.2	790.4	1523.6	147.1
200	162.0	121.1	1417.1	1553.7	154.4
500	159.8	119.3	3289.9	1658.0	220.7
1000	160.8	122.6	6403.1	1655.0	395.7

Table 23: Median time per search on uk-2002, in ms, by the length of the stall of one thread per level or round (Figure 6).

Stall (μs)	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
0	26.9	25.9	42.9	45.3	88.3
10	26.5	27.1	40.8	44.5	86.2
20	26.4	25.6	41.6	44.1	88.2
50	27.2	26.0	41.0	47.2	87.5
100	26.7	26.3	42.9	48.8	88.3
200	25.9	26.2	45.0	44.4	87.7
500	25.8	27.1	51.6	49.1	87.3
1000	26.8	26.1	63.3	50.2	91.4
2000	26.2	26.2	85.2	58.1	97.0
5000	27.6	27.6	175.3	80.4	117.8
10000	26.9	26.8	329.8	129.5	167.0

Table 24: Median time per search on Kronecker, in ms, by the length of the stall of one thread per level or round (Figure 6).

Stall (μs)	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
0	14.9	14.8	23.9	40.8	84.0
10	15.0	14.7	23.9	40.7	83.9
20	14.8	14.8	23.9	41.1	83.7
50	14.8	14.7	24.1	41.1	83.8
100	14.8	14.8	24.3	40.9	83.8
200	14.8	14.7	24.9	41.3	83.9
500	14.8	14.8	26.2	41.0	84.3
1000	15.2	14.8	27.9	41.4	84.8
2000	15.1	14.8	32.3	42.0	85.6
5000	15.1	15.1	52.5	44.5	91.1
10000	14.9	14.9	90.3	53.6	102.9

Table 25: Median time per search on the graph of hubs, in ms, by the length of the stall of one thread per level or round (Figure 6).

Stall (μs)	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
0	79.2	75.1	340.4	91.6	145.3
10	79.8	74.2	340.2	91.1	145.4
20	80.8	75.4	346.6	90.9	145.6
50	81.1	75.6	345.2	90.4	145.7
100	81.2	75.9	344.8	91.8	145.5
200	80.8	77.1	351.1	94.4	145.7
500	80.7	75.8	366.1	128.5	156.3
1000	80.9	75.8	396.8	193.5	187.8
2000	80.7	75.5	456.4	323.0	252.0
5000	80.3	76.5	689.6	704.0	453.0
10000	80.0	75.9	1367.5	1341.6	781.9

Table 26: Median time per search on LiveJournal, in ms, with random stalls for 2% of the time, by stall length (Figure 7).

Stall (μs)	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
0	5.1	4.9	7.2	7.9	9.9
10	5.0	4.8	6.9	9.1	10.3
100	4.8	5.0	9.1	10.0	10.7
1000	5.6	5.8	19.7	14.7	15.6
10000	9.6	9.7	58.6	19.6	28.2

Table 27: Median time per search on Twitter, in ms, with random stalls for 2% of the time, by stall length (Figure 7).

Stall (μs)	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
0	44.4	43.9	58.1	89.5	134.1
10	44.9	44.7	58.7	91.5	136.6
100	44.8	44.4	61.0	93.8	140.2
1000	45.3	45.6	72.0	102.3	147.6
10000	51.3	50.6	165.8	139.3	185.3

Table 28: Median time per search on Road-USA, in ms, with random stalls for 2% of the time, by stall length (Figure 7).

Stall (μs)	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
0	168.3	121.8	269.9	1651.1	146.3
10	172.2	126.2	324.9	1574.2	152.0
100	170.1	121.5	780.0	1599.1	179.2
1000	168.7	121.4	3266.6	1651.7	299.4

Table 29: Median time per search on uk-2002, in ms, with random stalls for 2% of the time, by stall length (Figure 7).

Stall (μs)	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
0	25.9	25.9	38.3	43.0	85.6
10	26.3	25.8	38.8	45.0	88.5
100	26.0	26.6	41.3	47.8	89.6
1000	27.0	26.6	61.6	55.6	96.7
10000	32.7	32.0	161.6	88.2	135.6

Table 30: Median time per search on Kronecker, in ms, with random stalls for 2% of the time, by stall length (Figure 7).

Stall (μs)	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
0	14.7	14.8	23.8	41.0	83.6
10	15.0	15.1	24.4	41.8	85.8
100	14.9	15.0	25.2	42.9	86.7
1000	15.6	15.6	34.0	50.8	93.1
10000	21.2	21.0	111.5	88.0	117.7

Table 31: Median time per search on the graph of hubs, in ms, with random stalls for 2% of the time, by stall length (Figure 7).

Stall (μs)	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
0	80.0	75.6	345.9	90.4	145.9
10	81.4	76.1	349.9	101.5	152.9
100	82.8	79.4	362.6	140.8	178.3
1000	83.7	81.1	472.4	203.0	273.9
10000	87.8	82.4	1418.4	239.5	426.6

Table 32: Median time per search on LiveJournal, in ms, with random stalls of 1 ms, by the share of time stalled (Figure 8).

Stalled (%)	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
0	5.1	4.9	7.2	7.9	9.9
0.5	5.1	5.5	10.6	10.2	11.6
1	5.2	5.5	12.2	11.8	13.0
2	5.6	5.8	19.7	14.7	15.6
5	6.0	6.4	31.2	21.7	20.3
10	6.3	6.6	33.0	32.9	30.1
20	6.5	6.7	41.8	48.3	41.1
50	8.9	9.0	46.4	107.2	76.3

Table 33: Median time per search on Twitter, in ms, with random stalls of 1 ms, by the share of time stalled (Figure 8).

Stalled (%)	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
0	44.4	43.9	58.1	89.5	134.1
0.5	44.3	44.1	60.2	93.2	137.8
1	44.8	44.6	62.7	97.5	140.8
2	45.3	45.6	72.0	102.3	147.6
5	46.4	46.2	87.8	112.9	158.4
10	48.0	47.8	95.5	130.7	176.0
20	52.1	52.3	102.0	159.8	207.9
50	73.9	73.9	132.6	268.9	314.1

Table 34: Median time per search on Road-USA, in ms, with random stalls of 1 ms, by the share of time stalled (Figure 8).

Stalled (%)	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
0	168.3	121.8	269.9	1651.1	146.3
0.5	169.6	122.6	484.2	1603.9	191.2
1	172.2	120.4	752.8	1661.5	233.0
2	168.7	121.4	3266.6	1651.7	299.4
5	167.5	119.6	7774.5	1833.4	450.6

Table 35: Median time per search on uk-2002, in ms, with random stalls of 1 ms, by the share of time stalled (Figure 8).

Stalled (%)	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
0	25.9	25.9	38.3	43.0	85.6
0.5	26.3	26.0	44.0	46.7	87.8
1	26.2	27.3	48.9	49.9	91.9
2	27.0	26.6	61.6	55.6	96.7
5	27.8	27.6	86.8	71.2	113.0
10	28.2	28.0	96.7	89.9	130.5
20	28.9	29.7	103.2	124.5	167.4
50	37.7	37.9	131.1	261.2	297.3

Table 36: Median time per search on Kronecker, in ms, with random stalls of 1 ms, by the share of time stalled (Figure 8).

Stalled (%)	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
0	14.7	14.8	23.8	41.0	83.6
0.5	15.1	15.1	27.4	44.5	86.3
1	15.5	15.5	29.3	46.8	88.8
2	15.6	15.6	34.0	50.8	93.1
5	16.3	16.2	42.1	59.1	102.1
10	16.9	17.3	46.7	70.0	114.0
20	18.3	18.6	50.4	86.8	135.8
50	26.0	26.3	68.0	146.1	228.4

Table 37: Median time per search on the graph of hubs, in ms, with random stalls of 1 ms, by the share of time stalled (Figure 8).

Stalled (%)	WF-DO	WF-DO-mem	GAPBS	GBBS	PASGAL
0	80.0	75.6	345.9	90.4	145.9
0.5	81.6	78.5	382.0	120.3	186.7
1	82.2	78.5	417.1	149.9	217.2
2	83.7	81.1	472.4	203.0	273.9
5	86.3	83.9	561.6	328.2	392.6
10	88.5	89.3	623.0	505.3	550.0
20	96.3	101.9	713.0	829.6	799.7
50	134.9	143.9	1041.9	2096.5	1639.2